



European Trustworthy
AI Association



KAA for explainability

User and Integration Manual

**KAA
v25.12**



Contributors

	Name	Organisation
Responsible for the deliverable	Philippe Dejean	IRT Saint Exupéry
Co-author	Mohamed-Chafik Bakey	IRT Saint Exupéry
	Alice Petit	Sopra Steria
	Caroline Gardet	Sopra Steria
	Antoine Coppin	Naval Group
	Philippe Dejean	IRT Saint Exupéry



Contents

Introduction	7
Versions	8
Presentation	10
.1 Definitions	10
.1.1 Evolution	10
.1.2 Four notions for a human-centered AI	10
.2 Scope of KAA	12
.3 Value proposal description	12
.3.1 Benchmark of explanations without KAA	12
.3.2 Benchmark of explanations with KAA	13
.4 Available methods and metrics	14
 I User manual	 16
Introduction	17
A Kaa's TUI	18
A.1 TUI widgets	18
A.1.1 Switchable tabs	18
A.1.2 Drop-down lists	18
A.1.3 Editable areas	19
A.1.4 Spin buttons	19
A.1.5 Check boxes	19
A.1.6 Groups of checkboxes	19
A.1.7 Radio buttons	19
A.1.8 Action buttons	19
A.1.9 Help	19
A.2 Kaa's TUI	21
A.2.1 Tab "USE CASE"	21
A.2.1.1 Column "MODEL"	21
A.2.1.2 Column "DATASET"	21
A.2.2 Tab "EXPLAINABILITY"	22
A.2.2.1 Column "METHODS"	22
A.2.2.2 Column "METRICS"	23
A.2.3 Tab "REPORTS"	23
A.2.3.1 Column "VISUALISATION"	23
A.2.3.2 Column "FULL REPORT"	24
A.2.4 Tab "EXECUTION"	24
A.2.4.1 Column "ACTIONS"	25



A.2.4.2	Column "MANAGEMENT"	25
A.3	Remark about method and metric parameter keys	26
B	Principles of use	27
B.1	First use	27
B.1.1	Configuration	27
B.1.1.1	The environment variables	27
B.1.1.2	The paths to the data and models	27
B.1.2	Launching	27
B.2	Kaa quick start	28
B.2.1	Selection of the use case and the model to test	28
B.2.2	Selection of the data to treat	29
B.2.3	Selection of the explainability library	31
B.2.4	Selection of the explainability method	32
B.2.5	Selection of the report	33
B.2.6	Set path extension and traces in log	34
B.2.7	Launch actions	36
B.2.8	Results through the log file	36
B.2.9	Selection of another explainability method	37
B.2.10	Selection of explainability metric	38
B.2.11	Launch actions	39
B.2.12	Results through the log file	40
B.3	Kaa command line	41
B.3.1	Help option	41
B.3.2	Usage of command file	41
B.3.2.1	Principle	41
B.3.2.2	Usage	42
B.3.3	Activate or deactivate a Use Case	43
B.3.4	Plugin paths	43
B.3.4.1	Library plugins	43
B.3.4.2	Use case plugins	43
II	Integration manual	44
Introduction		45
A	Library integration	46
A.1	Plugin declaration in the <library_name>.py file	46
A.1.1	Import libraries and other plugin files	46
A.1.2	The class and its initialisation	47
A.1.3	Usable methods and metrics	48
A.1.4	Absence of metrics	49
A.1.5	Method selection for metrics	49
A.2	Plugin methods	49
A.2.1	XAI_computeExplanations()	49
A.2.2	XAI_plotExplanations()	50
A.2.3	XAI_computeMetrics()	51
A.2.4	XAI_plotMetrics()	51
A.2.5	XAI_writeReport()	52
A.2.6	Compute Explanations methods in the <library_name>_computeExplanations.py file	52



A.2.6.1	Import libraries	52
A.2.6.2	Functions in the file	53
A.2.7	Plot Explanations methods in the <library_name>_plotExplanations.py file . . .	55
A.2.7.1	Import libraries	55
A.2.7.2	Functions in the file	55
A.2.8	Compute Metrics methods in the <library_name>_computeMetrics.py file	57
A.2.8.1	Import libraries	57
A.2.8.2	Functions in the file	57
A.2.9	Plot Metrics methods in the <library_name>_plotMetrics.py file	59
A.2.9.1	Import libraries	59
A.2.9.2	Functions in the file	59
B	Existing pluginsXAI	60
B.1	AIX360	60
B.1.1	Main methods	60
B.1.2	Specifial methods and functions	61
B.1.2.1	<i>Same training prediction</i> function	61
B.1.2.2	Model method: predic_proba()	61
B.1.2.3	Model method: megaPixels() and reduceTo1Band()	61
B.1.2.4	Model method: predict_from_text()	62
B.2	Alibi	62
B.2.1	Specifial methods and functions	63
B.3	Captum	63
B.3.1	Specifial methods and functions	63
B.3.1.1	Model method: megaPixels() and reduceTo1Band()	63
B.3.1.2	Model method: predict_torchtensor()	63
B.4	PAIR saliency	64
B.4.1	Specifial methods and functions	64
B.4.1.1	<i>Call Model Function To Gradient</i> function	64
B.5	Shap	64
B.5.1	Specifial methods and functions	65
B.5.1.1	Model method: predict_from_textlist()	65
B.6	Xplique	65
B.6.1	Specifial methods and functions	66
B.6.1.1	Model method: predic_proba()	66
B.6.1.2	Model method: megaPixels() and reduceTo1Band()	66
C	Use case integration	67
C.1	Use case definition in <nameUC>.py	67
C.1.1	Specific functions	67
C.1.2	Model class and methods	68
C.1.2.1	__init__()	68
C.1.2.2	fctPrepareModel()	69
C.1.2.3	fctPrepareData()	69
C.1.2.4	__call__()	69
C.1.2.5	convertDataToFramework()	70
C.1.2.6	convertPredictionToFramework()	70
C.2	Use case definition in UC<nameUC>.py	70
C.2.1	Specific functions	70
C.2.1.1	UC_createModel()	71
C.2.1.2	UC_computeInference()	71
C.2.1.3	UC_ImageDataToPlot()	72



C.2.1.4	UC_ExplanationToPlot()	73
C.2.1.5	getModelParams()	73
C.2.2	Model class and methods	73
C.2.2.1	__init__()	73
C.2.2.2	setInferenceCible()	74
C.2.2.3	savePrediction()	74
C.3	Data files declaration in UC<nameUC>.json	75
C.4	Plugin declaration in UC<nameUC>_<nameLib>.py	76
C.4.1	The class and its initialization	76
C.5	Plugin methods	77
C.5.1	UCXAI_createModel()	77
C.5.2	UCXAI_computeInference()	77
C.5.3	UCXAI_computeExplanations()	77
C.5.4	UCXAI_plotExplanations()	78
C.5.5	UCXAI_computeMetrics()	78
C.5.6	UCXAI_plotMetrics()	78
C.5.7	UCXAI_writeReport()	78
C.6	Quick howto integration	79
C.6.1	Model instantiation	79
C.6.2	Data preparation as model input	79
C.6.3	Inference	79
D	Checkpoints	80
D.1	PluginUCXAI checkpoints	80
D.1.1	FctPrepareDataOutput checkpoint	81
D.1.2	controlInferenceInput checkpoint	81
D.1.3	controlInferenceOutput checkpoint	81
D.1.4	UC_computeInferenceOutput checkpoint	81
D.2	PluginXAI checkpoints	81
D.2.1	XAI_computeExplanationsInput checkpoint	81
D.2.2	XAI_computeExplanationsOutput checkpoint	82
D.2.3	_loadExplanationsOutput checkpoint	82
D.2.4	pFctExplainToPlotOutput checkpoint	82
D.2.5	_plotExplanationInput checkpoint	82
D.2.6	XAI_computeMetricsInput checkpoint	82
E	Documentation	83
E.1	Global parameter dictionary	83
E.2	Templates	84
F	Explainability toolboxes	85
F.1	AIX360	85
F.1.1	The library	86
F.1.2	The metrics	88
F.1.2.1	Metrics of AIX360	88
F.1.2.2	Metrics of external libraries	89
F.2	Alibi	90
F.2.1	Description	90
F.2.2	The explanation methods	90
F.3	Captum	91
F.3.1	Description	91
F.3.2	Primary attribution	91



F.3.3	Layer attribution	92
F.3.4	Neuron attribution	93
F.3.5	”Noise Tunnel” function	94
F.3.6	Metrics	94
F.4	PAIR Saliency	95
F.4.1	Description	95
F.4.2	The attribution methods	95
F.4.2.1	(Vanilla) Gradients and SmoothGrad	95
F.4.2.2	Grad-CAM	95
F.4.2.3	Occlusion	95
F.4.2.4	Integrated Gradients	95
F.4.2.5	Guided IG (Guided Integrated Gradients)	95
F.4.2.6	Blur IG (Blur Integrated Gradients)	95
F.4.2.7	XRAI (eXplanation with Ranked Area Integrals)	96
F.4.3	The metrics	96
F.4.3.1	PIC (Performance Information Curve)	96
F.5	SHAP	97
F.5.1	Description	97
F.5.2	The explanation methods	97
F.5.3	The metrics	97
F.6	Xplique	98
F.6.1	The attribution methods	98
F.6.1.1	White-box attribution methods	98
F.6.1.1.1	DeconvNet	98
F.6.1.1.2	Guided Backpropagation	98
F.6.1.1.3	Saliency	98
F.6.1.1.4	Gradient x Input	98
F.6.1.1.5	Integrated gradients	98
F.6.1.1.6	Smoothgrad	98
F.6.1.1.7	SquareGrad	99
F.6.1.1.8	VarGrad	99
F.6.1.1.9	Grad-CAM	99
F.6.1.1.10	Grad-CAM++	99
F.6.1.2	Black-box explanation methods	99
F.6.1.2.1	LIME	99
F.6.1.2.2	KernelSHAP	99
F.6.1.2.3	Occlusion	99
F.6.1.2.4	RISE	99
F.6.1.2.5	HSIC	100
F.6.1.2.6	Sobol	100
F.6.2	The explanation metrics	100
F.6.2.1	MuFidelity	100
F.6.2.2	Deletion	100
F.6.2.3	Insertion	100
F.6.2.4	Average Stability	100
F.6.2.5	MeGe	100
F.6.2.6	ReCo	100

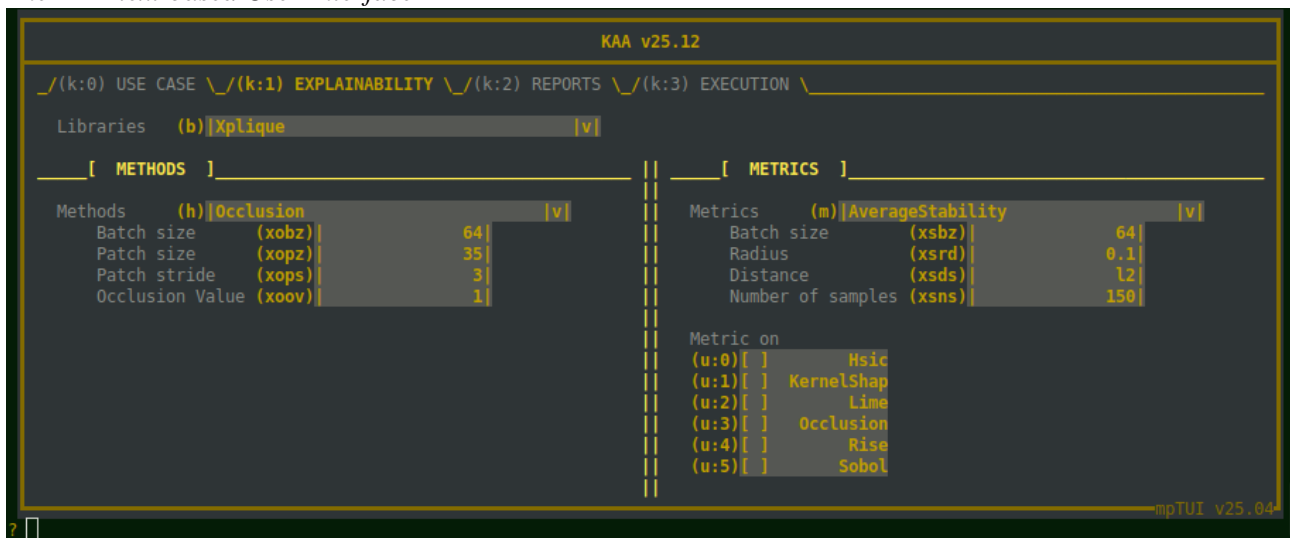


Introduction

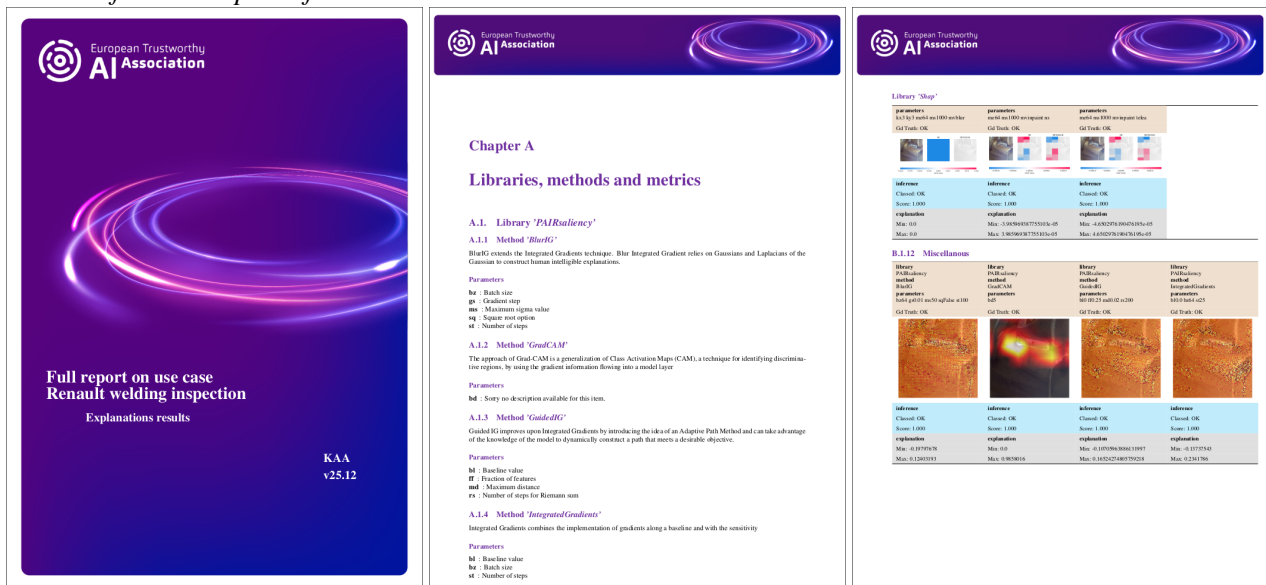
This document consists of two separate parts:

1. a user manual for the handling of the platform and its operation for the explainability of AI models
2. an integration manual for the enrichment of KAA by:
 - the creation of plugins dedicated to use cases
 - creation of new explainability library plugins

The KAA text-based User Interface



Extract of a KAA report of a benchmark





Versions

Version 25.12

First open-source version!

This version is tagged 25.12 for December 2025.

It contains the following features:

- Upgrade versions of XAI libraries in pluginsXAI:
 - AIX360 v0.3.0
 - Alibi v0.9.6
 - Captum v0.7.0
 - PAIR-Saliency v0.2.1-c (fork of v0.2.1 to extend capabilities)
 - Shap 0.44.1
 - Xplique v1.4.0

Warning: Due to dependancies of these libraries, the Python version must to be v3.8.

This is the last version of Kaa with this problem. A major evolution will allow to use libraries with separate environments.

Version 24.09

This version is tagged 24.09 for September 2024.

It contains the following features:

- Enrichment of explanation results for better analysis (with command `t1`) (see §A.2.3.1).
- Creation of a complete experimentation report that includes all the results of a benchmark (with new action "full report") (see §A.2.3.2)
- Color map harmonisation for object detection and semantic segmentation (Visualisation tab) (see §A.2.3.1)

Version 24.07

This version is tagged 24.07 for July 2024.

It contains the following features:

- create sub-directories for results according to the sub-directories in data path (see §A.3).
- reformatting name of result files (see §A.3).
- enrich "*planche contact*" LaTeX report (with adding command `nc`)(see §A.2.3.1).

Version 24.06

This version is tagged 24.06 for June 2024.

It contains the following features:

- `modelcode` removal in the json file of use case plugins.
- plot explanation with original data and/or model input data shap with command `tw` (see §A.2.3.1).
- adding model parameters in the TUI interface and command files to adjust them (see §A.2.1.1).
- split the pluginXAI file into multiple functionality files for easy integration (see §A).
- save the inference image for all use cases in the "*dataInference*" folder with the associated *.json.



- code cleaning.

Version 24.05

This version is tagged 24.05 for May 2024.

It contains the following features:

- ability to use `-b` option in command line instead of environment variable `USECASE_BASE` (see §B.1.1).
- ability to use `-r` option in command line instead of environment variable `RESULT_PROD` (see §B.1.1).
- ability to add your own directory path for the `pluginXAI` with the command line option `-x` (see §B.3.4.1).
- ability to use your own directory path for the `pluginUCXAI` instead of the KAA one with the command line option `-u` (see §B.3.4.2).
- ability to save commands to a file for later use with the KAA TUI command `!r:<file>` (press `!` to see usage).
- restructuring of the use case plugins with isolation of their specific functions (see §C.1).
- activation/deactivation of type and shape verifications for handled elements in ten critical points with TUI command `K` (see §D).
- rename `dataplot` result directory to `dataPlotExplanations`.
- plot image results according to data origin size or input model size with the TUI command `tw` (see §A.2.3.1).
- define plot image results height (in inch) with the TUI command `th` (see §A.2.3.1).

Version 24.04

This version is tagged 24.04 for April 2024.

It contains the following features:

- command line enriched. Use `"kaa.py -help"` (see §B.3.1).
Warning: the command lines `"kaa.sh <command file>"` or `"kaa.py <command line>"` have to be changed as: `"kaa.sh -c <command file>"` or `"kaa.py -c <command line>"` respectively (see §B.3.2).
- enable/disable use case with the `-a` or `-d` option in command line (see §B.3.3).
- tooltips can be available in TUI with `!i:<action>` command. Use the `!` command to learn more.



Presentation

.1. Definitions

.1.1 Evolution

The major shift towards human-centric AI marks a turning point in the design, deployment and evaluation of artificial intelligence systems.

Until the 2020s, AI has largely developed around increasingly efficient but opaque models (deep learning, large language models). These systems, although effective, posed major problems:

- Lack of transparency: inability for users to understand how a decision was made.
- Risk of bias and errors: without explainability, difficult to detect and correct biases or malfunctions.
- User mistrust: opacity fueled distrust, especially in sensitive sectors (health, justice, finance).

As early as 2016, programs such as DARPA (US) launched research on explainable AI (XAI), aiming to make models more transparent and understandable. The objective being to allow humans to understand, criticize, and validate algorithmic decisions.

The adoption of the AI Act in 2024-2025 marked a turning point. This regulation imposes:

- Transparency and explainability requirements for high-risk systems.
- A situated explainability: the explanations must be adapted to the context and to the end user, and not only technical.
- Increased responsibility of designers and users, with recourse mechanisms in case of a questionable decision.

.1.2 Four notions for a human-centered AI

Explainability and interpretability are often confused, but explainability concerns the transparency of the system, while interpretability pertains to the human capacity to give meaning to explanations.

Comprehensibility and understanding emphasize the importance of human mediation and criticism of explanations, especially in high-risk areas.

The European AI Act (2024-2025) and recent work in XAI insist on the need for a situated explainability, adapted to the context and to the user, for a responsible and trustworthy AI.

These four concepts can be summarized and defined as follows:

Adapted and contextualized explainability

Explanations for different audiences: a data scientist who develops an AI model, a quality technician, or a end-user do not have the same needs in terms of explanations. AI tools must provide justifications adapted to each profile.



Explainability : Explainability in AI refers to the ability to relate and make understandable the elements considered by an AI system to produce a result. It aims to provide a technical or logical justification for an algorithmic decision, often in response to a requirement for transparency or regulation.



Explainability is essential for users (expert or not) to be able to understand, critique and, if necessary, challenge AI decisions.

Recent developments: With the entry into force of the European AI Act (2024-2025), explainability has become a legal obligation for high-risk systems, requiring explanations adapted to the context and the end user.

Understanding and criticism

Avoid passive acceptance: the AI Act and XAI work highlight the risk that users accept AI decisions without understanding them, through over-confidence or habit. Encourage critical thinking: train users to assess the relevance and robustness of explanations provided by AI.



Understanding : Understanding is the faculty of grasping concepts and logical relationships, allowing one to judge the validity or coherence of an explanation. It is about an ability to integrate and criticize information, including those from AI, by linking them to conceptual categories.

Risks: Several studies in XAI highlight that users can accept AI decisions without understanding them, especially if the system has a good reputation. This can lead to a false sense of understanding or uncritical addiction. The European AI Act seeks to avoid this passive acceptance by requiring understandable and critiquable explanations.

Comprehensibility and appropriation

Understanding goes beyond explanation: it is about allowing the user to appropriate the meaning of the decision, criticize it, and act accordingly.



Comprehensibility : Comprehensibility is the cognitive outcome by which a human grasps the meaning and implications of an algorithmic explanation or decision. It involves the ability to reexpress meaning understood in another setting (e.g., translate a technical explanation into accessible terms) and to become aware of the context.

Challenge: Comprehensibility goes beyond the mere reception of an explanation: it requires active appropriation by the user, often facilitated by mediation or reformulation tools.

Interpretability as a human process

Interpretation is not passive: it involves an active mediation between the machine and the human. The user must be able to link the explanations to their own frame of reference, knowledge and experience.



Interpretability : Interpretability is defined as the ability to explain or provide the meaning of the results of an AI model in human-understandable terms. It is not limited to the technical analysis of internal mechanisms (weight, features, etc.), but mainly refers to the ability for a human to make sense of the model's decision, by linking the explanations provided to their own context, business knowledge or reference framework.

Human dimension: Interpretation is an active cognitive process, where the user gives meaning to explanations based on their expertise and expectations. For example, a doctor interprets a risk score based on their medical knowledge and the patient's history.



These definitions reflect a major shift towards human-centric AI, where technical transparency is no longer enough. It is also necessary that the explanations are useful, reviewable and adapted to the real needs of the users.

.2. Scope of KAA

KAA must be considered as a comprehensibility tool. It is part of these mediation tools allowing to implement explainability methods in a guided way, to facilitate the adjustment of their parameters by adding knowledge of their influence. It then offers appropriate and relevant interfaces and visualizations of results to select the methods and metrics that are most suited to the user's problem.

Kaa is a framework allowing to apply several methods and metrics from several dedicated libraries on AI model to verify them.

It is built around a kernel offering a TUI (*Text-based User Interface*) and hosting two sets of plugins:

- the plugins of the explainability libraries called `pluginsXAI`
- the plugins of the use cases called `pluginsUCXAI`

In this version of Kaa, the AIX360, Alibi, Captum, PAIR saliency, Shap and Xplique plugins are provided. A chapter is dedicated to their specificities in the part "Integration manual" §II.

.3. Value proposal description

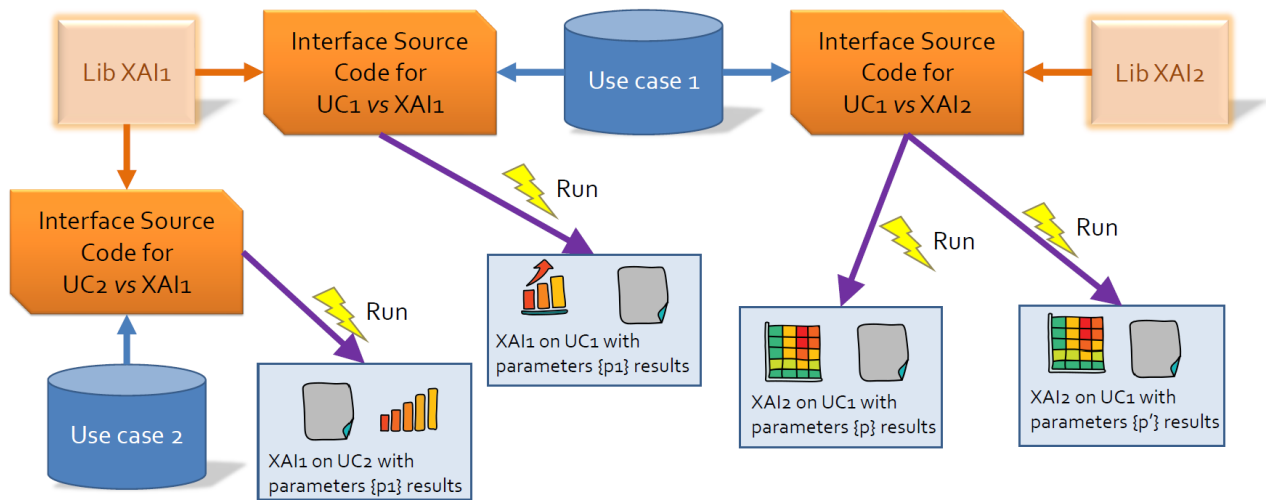
.3.1 Benchmark of explanations without KAA

Actually, when you want to use explainability methods or metrics provided by a dedicated library to explain your model, you need to:

- know how the library provides its API and how to adjust your source code to interface with your data and your model to have good explanations and efficiency plots,
- known the use case source code to load data, load model, run an inference and plot its result,
- implement this interface between library, method, metric and your model and data -Interface Source Code (ISC)- for each method and each metric,
- run each triplet ISC(parameters) you need to test.

Then, you need to display efficiently all results with ground-truth to compare explanation each other on several data.

These steps are resumed in this graph:



.3.2 Benchmark of explanations with KAA

With KAA, you need to know only the use case source code to development of the Function Source Code (FSC) for

- model loading (FSC m.l)
- data loading (FSC d.l)
- inference (FSC inf)

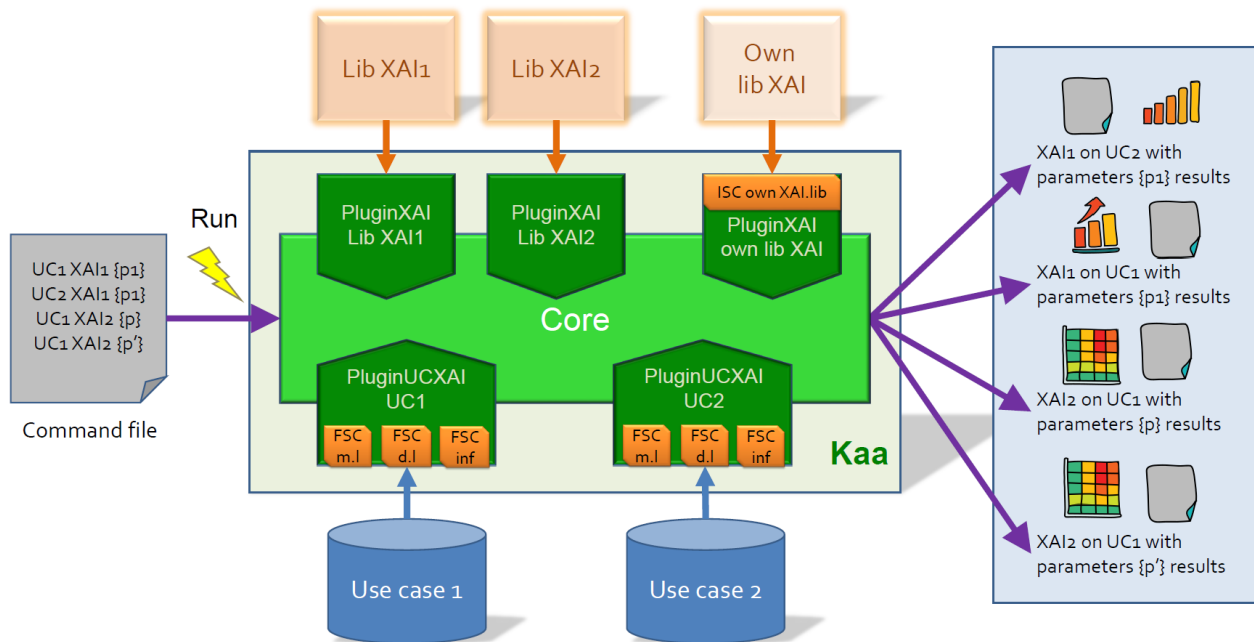
in a plugin dedicated to your use case.

In a single run for a set of triplets (Use Case, XAI methods or metrics, parameters), grouped in a command file, you run a KAA session of benchmark and obtain explanation results as *contact sheet* for each sub-run, a full report of the benchmark in pdf format -if LaTeX is installed on your system- are all results (inference result, explanation file, plots saved in separate directories.

A Text-based User Interface is provided to pilot KAA session.

Go to further, you can develop a plugin for your own explainability library.

KAA architecture is resumed in this graph:



.4. Available methods and metrics

At the end of this document, two chapters are dedicated to the provided plugins for use explainability libraries:

- §F gives some descriptions of the methods and metric excerpt from their official repository,
- §B describes the plugins of these libraries in Kaa.

A table of references is available in chapter §F.6.2.6.

The table bellow, resume the usability of the methods and metrics according to the kind of model and problem as implemented in the provided plugins. It was established on private use case during the national french program *Confiance.ai* and indicate according the available use cases and for each tested library some characteristics in terms of:

1. Supported tasks: Classification, Regression, Object detection, Object segmentation, NLP
2. Data type supported: Tabulars, Images, Text

Library	Method and <i>metric</i>	Classif/Regr Tabular	Classif Image	Obj.Detect-Segm Image	NLP Text
AIX360 (IBM)	BRCGE	👉			
	Lime	👉	👉		👉
	LinearRule Regression	👉			
	LogisticRule Regression	👉			
	Protodash	👉			👉
	Shap	👉	👉		
	<i>Faithfulness</i>	👉			
	<i>Monotonicity</i>	👉	👉		
	<i>Sklearn-Scores</i>	👉			
Captum (Meta)	Anchors				👉
	Feature Ablation		👉		👉
	Feature Permutation		👉		👉



	Shapley Value Sampling		👉		👉
	Occlusion		👉		👉
	KernelSHAP		👉		👉
	Lime		👉		👉
PAIR Saliency (Google)	BlurIG		👉		
	BlurIG (SmoothGrad)		👉		
	GradCAM	👉	👉		
	GuidedIG	👉	👉		
	GuidedIG (SmoothGrad)	👉	👉		
	IntegratedGradients	👉	👉		
	Occlusion		👉		
	VanillaGradients	👉	👉		
	VanillaGradients(SmoothGrad)	👉	👉		
Shap (Microsoft)	Partition	👉	👉		👉
Xplique (ANITI)	DeconvNet	👉	👉		
	GradientInput	👉	👉		
	GuidedBackProp	👉	👉		
	Hsic		👉	👉	
	IntegratedGradients	👉	👉		
	KernelShap	👉	👉	👉	
	Lime	👉	👉	👉	
	Occlusion	👉	👉	👉	
	Rise		👉	👉	
	Saliency	👉	👉		
	SmoothGrad	👉	👉		
	Sobol		👉	👉	
	SquareGrad	👉	👉		
	VarGrad	👉	👉		
	<i>Deletion</i>		👉	👉	
	<i>Insertion</i>		👉	👉	
	<i>Average tability</i>		👉	👉	
	<i>Mu Fidelity</i>		👉	👉	



Part I

User manual



Introduction

This part is divided into two chapters:

1. The first one is dedicated to the presentation of the use of the widgets in KAA text-based user interface (TUI) and the TUI itself.
2. The second one is dedicated to the use of KAA with an example based on the provided use case plugin.



Chapter A

Kaa's TUI

The Kaa Text-based User Interface allows you to control the explainability activities and explainability metrics according to the plugins present.

A.1. TUI widgets

The TUI consists of the following widgets:

- switchable tabs
- drop-down lists
- editable areas
- check boxes
- groups of checkboxes
- action buttons

A.1.1 Switchable tabs

The tabs are switchable by entering the letter and number in parentheses.

```
_(k:0) USE CASE \_(k:1) EXPLANABILITY \_(k:2) REPORTS \_(k:3) EXECUTION \_
```

A.1.2 Drop-down lists

Drop-down lists allow you to select an element from a defined set. They are identified by |v| and usable by the letter in parentheses.

```
Libraries (b)|AIX360|v|
```

The letter followed by :v allows you to scroll down the list:

```
Libraries (b)|AIX360|v|
(b:0)|>AIX360
(b:1)|Captum
(b:2)|PAIRsaliency
(b:3)|Shap
(b:4)|Xplique
```

This shows the selected item indicated by '>' and the commands in parentheses to select a new item from the list.

If you want to choose a new value of item, you can use the command in parentheses (b:2) or you can use the command followed by ''' and the name of the item. Be careful the second option does not work if the name contains a ':' (b:'PAIRsaliency').



A.1.3 Editable areas

Editable areas are used to fill in requests. To do this, simply enter the command consisting of the key indicated in parentheses, two points ':' followed by the value to be filled in. In the case below the command is `algkw:25`.

```
Kernel Width (algkw)| 25|
```

A.1.4 Spin buttons

The spin buttons allow to adjust numeric values. The command in parenthesis followed by ':' and + or - increase or decrease the current value with the first widget value (default 1), while commands ++ or - increase or decrease the current value with the second widget value. You can also change the value like in editable areas case.

```
Quick report: Num.Columns (nc)| 3|+-|
```

A.1.5 Check boxes

Check boxes allow to select or not operating choices. They are indicated by brackets '[']' with or without checkmark 'x'. To activate them, simply use the command in parentheses.

```
Check types and shapes (K)[ ]
```

A.1.6 Groups of checkboxes

Groups of checkboxes allow to select or not a set of choices for a given treatment. The operation is the same as for simple checkboxes.

```
Possible actions
(a:0)[ ] Inference
(a:1)[ ] Explanations
(a:2)[ ] Plot explanations
(a:3)[ ] Metrics
(a:4)[ ] Plot metrics
(a:5)[ ] Quick report
(a:6)[ ] Full report
```

Note that the `<letter>:s`, `<letter>:d` and `<letter>:i` commands allow you to select all, deselect all, and reverse the selection, respectively.

A.1.7 Radio buttons

As check boxes, radio buttons allow to select or not operating choices. With only and always one option can be selected.

```
Plot size
(tw:0)(o) model size
(tw:1)( ) data size
```

A.1.8 Action buttons

Action buttons make it possible to launch actions. They are materialized by brackets '[']' with inside the command in parentheses and the action label.

```
[(!)Launch actions ]
```

A.1.9 Help

If you need help you can write the command '!'. This command will show every helpful commands you need.

- '!d!': to activate debug log



- '!i:<action>': to see more information about a method or a parameter, it can be helpful to define the correct value
- '!c!': to collapse all tabs
- '!uc!': to uncollapse all tabs
- '!p:<file>': charge palette - !p:kw, !p:y, !p:b, !p:yb, !p:yg, !p:gg : charge predefined palettes
- '!cmd:<file>': to execute commands from file
- '!r:<file>': to record commands on file
- '!r!': to pause/stop or restart recording



A.2. Kaa's TUI

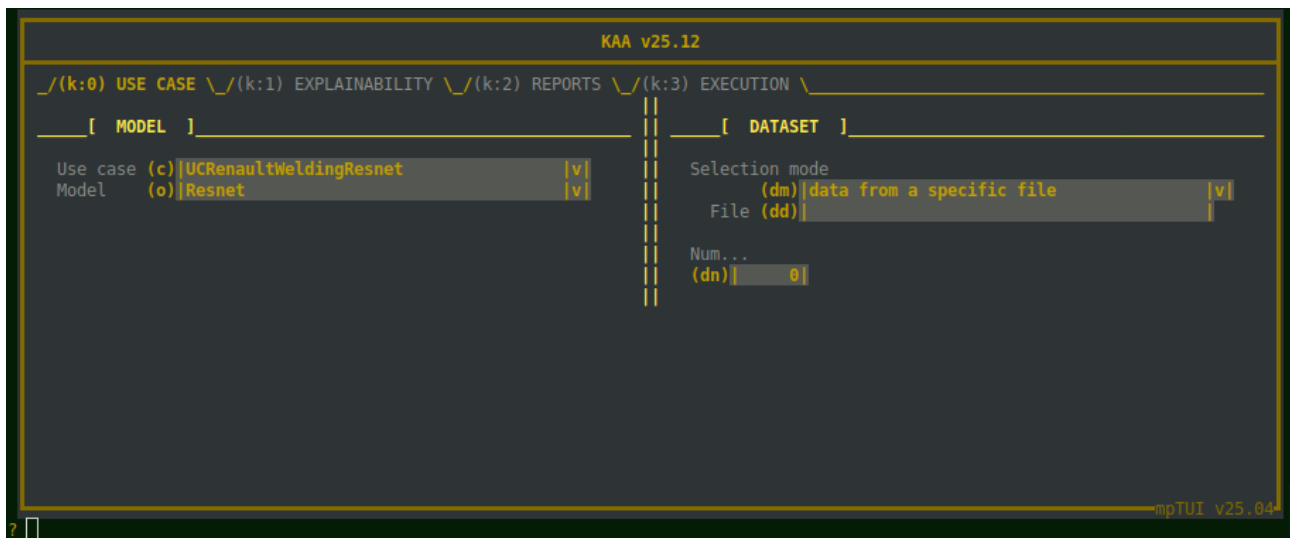
The Kaa interface consists of 4 switchable tabs:

- (k:0) **USE CASE** installing the use case to be treated
- (k:1) **EXPLANABILITY** indicating the methods and metrics to be used and their settings
- (k:2) **REPORTS** to custom reports
- (k:3) **EXECUTION** to control the treatments

Each tab is separated in two columns.

A.2.1 Tab "USE CASE"

This tab contains two columns dedicated to model and dataset configuration.



A.2.1.1 Column "MODEL"

```
-----[ MODEL ]-----  
  
Use case (c) | UC<USECASE_NAME> | v |  
Model (o) | <MODEL_NAME> | v |  
  
Model parameters  
  <param 1> | <value> |  
  <param 2> | <item1> | v |
```

This tab allows you to select the use case, the model and its parameters if necessary. The use case comes from the plugin name retrieved by Kaa, the models come from the associated json file (cf. §C). The use cases present are all active use cases (cf. §B.3.3 to see how to activate or deactivate a use case).

A.2.1.2 Column "DATASET"

```
-----[ DATASET ]-----  
  
Selection mode (dm) | selection of data from dataset | v |  
File (dd) | |
```



Num.Data	Offset	Step
(dn)	(do)	(dp)
2	0	1

This area allows you to specify and set the data selection mode. The *selection mode* allows to designate how the data to be processed will be selected. The possible choice for (dm) are:

(dm:0) data from a specific file : select (dn) data from a list; the option File (dd) appears to select the file name containing the names of the data to be processed (see below).

Since version 24.05, you can use delocalized data. Put a '@' character before the filename of the data ; the file name must be relative to the directory of the file indicated in the editor line dd.

(dm:1) one or more data from config file list : select (dn) data from a list; this list is set in the json configuration file (see §C.3)

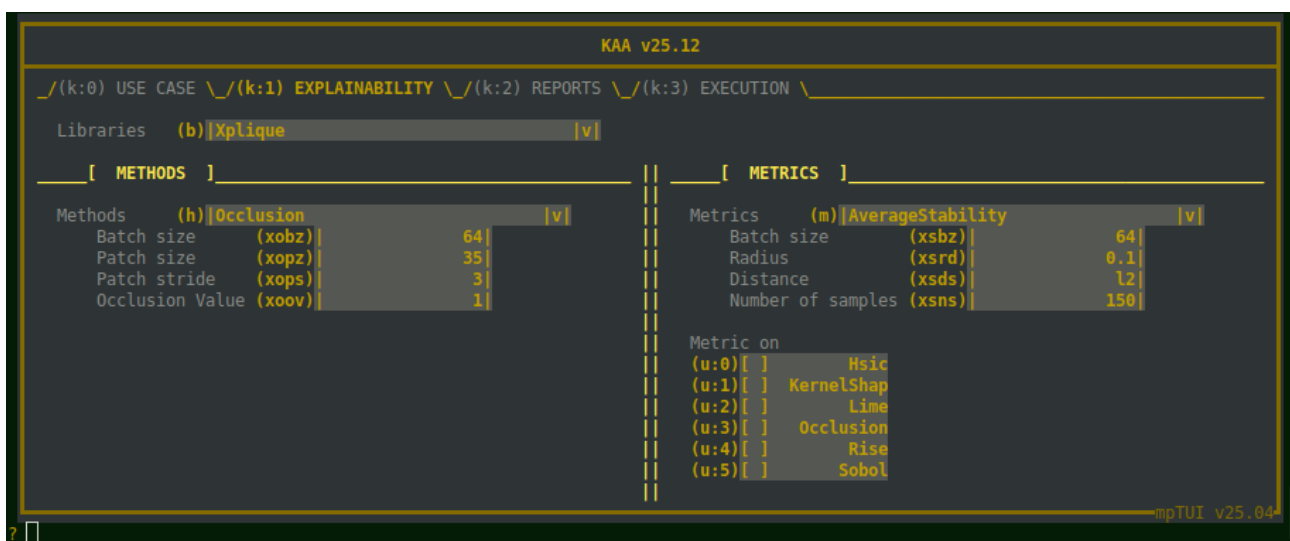
(dm:2) all data from dataset : all data in the dataset path indicated in the json configuration file are selected,

(dm:3) selection of data from dataset : only some data in the dataset path are selected. In this case, the options Offset (do) and Step (dp) appear to adjust the selection:

(dn) data from (do), one each (dp)

A.2.2 Tab "EXPLAINABILITY"

This tab contains two columns dedicated to methods and metrics of XAI libraries.



A.2.2.1 Column "METHODS"

Libraries	(b) <library>	v
-----[METHODS]-----		
Methods	(h) <library>:<method>	v
Param 1	(....)	64
Param 2	(....)	7
.	.	.
Param n	(....)	500 v

The Libraries (b) drop-down list allows access to libraries available in Kaa. Only libraries installed with the correct version are present in the list.



This area allows you to select:

- (h) the library method applicable to the selected use case,
- (...) the method parameters.

A.2.2.2 Column "METRICS"

```

_/(k:0)          \_/(k:1) EXPLANABILITY \_/(k:2)          \_/(k:3)          \_
Libraries      (b)|<library>                                |v|

-----[ METRICS ]-----

Metrics        (m)|<library>:<metric>                                |v|
Param 1  (...)|                                64|
Param 2  (...)|                                7|
Param n  (...)|                                500|v|

Metric on
(u:0)[ ] <library>:<method1>
(u:1)[ ] <library>:<method2>
      . . .
(u:3)[ ] <library>:<methodn>

```

This area allows you to select:

- (m) the library metric applicable to the selected use case,
- (...) the metric parameters,
- (u) the list of methods on which to apply the metric.

A.2.3 Tab "REPORTS"

This tab contains two columns dedicated to the visualization of results and the full report.

```

KAA v25.12
_/(k:0) USE CASE \_/(k:1) EXPLANABILITY \_/(k:2) REPORTS \_/(k:3) EXECUTION \
-----[ VISUALISATION ]-----||-----[ FULL REPORT ]-----
Colormap (tcm)|one colormap in the list|v|Report on
List      (tc)|hot|v|(f:0)[x] Explanations
                                     (f:1)[x] Metrics
Percentile (tp)| 1| Transparency (ta)| 0.6|
Normalise  (tn)|x| Legend (tl)| |
Font size  (fz)| 12|
Plot size
(tw:0)(o)  model size
(tw:1)( )  data size
Image height (inch) (th)| 5|+-|
Quick report: Num.Columns (nc)| 3|+-|
                                     ||
mpTUI v25.04

```

A.2.3.1 Column "VISUALISATION"

```

-----[ VISUALISATION ]-----%
Colormap      (tcm)|one colormap in the list|v|
List          (tc)|hot|v|

```



```

Percentile (tp)|    1|   Transparency (ta)|    0.6|
Normalise (tn)[x]           Legend  (tl)[ ]
Font size (fz)|    12|

Tabular use case
Item explanations (r)|           1-10|

Plot size
(tw:0)(o) model size
(tw:1)( ) data size
Image height (inch) (tc)|    5|+-|

Quick report: Nums.columns (nc)|    3|+-|

```

This tab allows to adjust visualisation of explanation results:

- (tcm)/(tc) Select the colormap for image problems
- (tp) Select the percentile to reduce the explainability histogram range
- (ta) Select the transparency level for the plot of the explainability over the image
- (tn) Choose if the explainability have to be normalised before plotting
- (tl) Choose to add legend
- (r) Select the tabular items to plot (comma separated, '-' for ranges)
- (tw) Select the shape of the plot according to the original data size or the data size as model requirement
- (th) Select the size (in inch) of the png image result
- (nc) Select the number of columns in the generated $\text{\LaTeX}$ quick report.

Remark about colormap:

You can choose the colormap to use for plotting images. You can select a colormap in a list or provide a json file containing the colors (minimum 3). Your json can be of 3 formats : a matplotlib colormap, a list of HEX colors or a list of RGB colors.

```

example:
{
  "colorMap": "viridis",
  "HEXcolors": ["#FFFFFF", "#FAEDED", "#FADBDB"],
  "RGBcolors": [[255, 0, 0], [255, 255, 0], [0, 255, 0]]
}

```

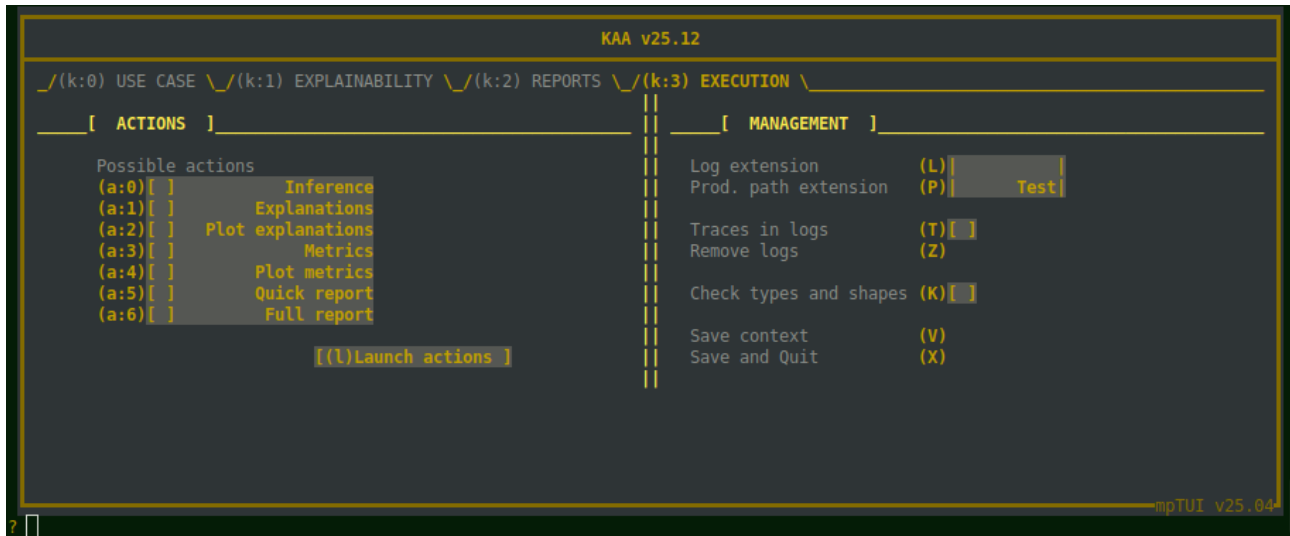
A.2.3.2 Column "FULL REPORT"

This tab allows to adjust full latex reports.

- (f:0) **Explanations** allows to create a latex report containing all the explanations. The report produced is in the directory `./dataProd<EXT>/fullReports`. <EXT> is defined by the menu option (P) see below. You can then choose with the command (fe) if you want the report to be organized by method or by data. You can also choose with the command (nch) how many explanations you want to plot by columns.
- (f:1) **Metrics** allows to create a latex report containing all the explanation metrics. The report produced is in the directory `./dataProd<EXT>/fullReports`. <EXT> is defined by the menu option (P) see below. The metric report is organized by metric. You can also choose with the command (ncq) how many metrics you want to plot by columns.

A.2.4 Tab "EXECUTION"

This tab contains two columns dedicated to the execution of selected processes and the the global configuration of Kaa.



A.2.4.1 Column "ACTIONS"

This tab allows you to select and launch the processing unit or cascade.

- (a:0) Inference allows to run the inference of the use case model on the dataset. The results produced are in the directory `./dataProd<EXT>/dataInference`. `<EXT>` is defined by the menu option (P) see below.
- (a:1) Explanations allows you to launch the explanation action of the use case model by the designated method. The results produced are in the directory `./dataProd<EXT>/dataExplain`.
- (a:2) Plot explanations allows you to launch the action of presenting the results of the explanations on the dataset data. The results produced can be found in the `./dataProd<EXT>/dataPlotExplanations`.
- (a:3) Metrics allows you to start the action of calculating the metrics designated on the explanatory model. The results produced are in the directory `./dataProd<EXT>/dataMetrics`.
- (a:4) Plot metrics allows you to launch the action of presenting the results of the metrics. The results produced can be found in the `./dataProd<EXT>/dataPlotMetrics`.
- (a:5) Quick report write a *contact sheet* report of explanation results and compile it in pdf format if $\text{\LaTeX}$ is available. The Quick report action force the "Visualisation" parameters Legend (tl:0) and "Plot size" as "Model size" (tw:0). If they are as after you choose "Quick report" action, the Quick report will be empty.
- (a:6) Full report write a latex report containing all explanation results or/and metric results and compile it in pdf format if $\text{\LaTeX}$ is available.
The results produced can be found in the `./dataProd<EXT>/fullReports/<UC model>`.

For full description of the file result naming see §A.3.

A.2.4.2 Column "MANAGEMENT"

This "MANAGEMENT" tab adjusts the operation in Kaa:

- (L) Log extension : provides an extension to the generated log file (timestamp if empty),
- (P) Prod. path extension : provides an extension for the root directory of the data produced `./dataProd<EXT>`,
- (T) Trace in logs : traces the processing actions in the log file,
- (Z) Remove logs : remove the generated logs,
- (K) Check types and shapes : during integration process, types and shapes of data can be controlled during actions (see §D),
- (V) Save context : saves the choices to restore them when Kaa is restarted,
- (X) Save and Quit : saves context and leaves Kaa



A.3. Remark about method and metric parameter keys

In "EXPLANABILITY" tabs, To guaranty the uniqueness of the commands, those to access and adjust method and metric parameters are defined by a set of three concatenated keys (cf §A.1):

1. The first key identifies the library,
2. The second key identifies the method or the metric,
3. The third key identifies the parameter.

Example:

Libraries	(b) Xplique	v
[METHODS]		
Methods	(h) Hsic	v
Batch size	(xhbz)	64
Grid size	(xhgz)	7
Number of designs	(xhnd)	500
Perturbation function	(xhfp) inpainting	v

Here for the *Grid size* parameter and its command xhgz, the three keys are:

x for *Xplique*
h for *Hsic*
gz for *Grid size*

Where can I find the results?

As explained in §A.2.4.1, all results are saved in several directory located in a main directory designated by the environment variable RESULT_PROD or the command line option -resultProd (see §B.1.1):

- dataProd<EXT>/dataInference for the inference without explanation mechanism,
- dataProd<EXT>/dataExplanations for the explanation results,
- dataProd<EXT>/dataPlotExplanations for the explanation plots,
- dataProd<EXT>/dataMetrics for the metrics results,
- dataProd<EXT>/dataPlotMetrics for the metric plots.

In each directory, result files are stored in other subdirectories formed by:

- the library name,
- the method name and its parameters: <method>__{<param-key><param-value>}*

The file names are formed with the data filename and some extensions as:

- --i_<value> to specify an item result such as bounding box or table entry
- --c_<value> to specify a class result
- --s_<value> to specify a particular result ('s' for suffix)



Chapter B

Principles of use

B.1. First use

B.1.1 Configuration

The configuration phase consists of defining two environment variables: `USECASE_BASE` and `RESULT_PROD` and ensuring that the paths to the data and models are correctly indicated in the json files of the pluginsUCXAI (cf §C.3).

Since version 24.05, both environment variable can be replaced by the `-b` (`-useCaseBase`) and `-r` (`-resultProd`) option in command line. These options are stronger than environment variables.

B.1.1.1 The environment variables

The two environment variables used by Kaa are:

`USECASE_BASE` refers to the base directory of the use case containing dataset and models,

`RESULT_PROD` refers to the base directory of the results produced by Kaa.

If it is not defined, it is set to the kaa directory.

B.1.1.2 The paths to the data and models

The two elements to control in the json pluginUCXAI file are the dictionary keys:

`dataset` refers to the directory of the dataset (to add to `USECASE_BASE`)

`modelpath` : refers to the directory of the model (to add to `USECASE_BASE`)

B.1.2 Launching

You can go to the kaa directory and launch the script `kaa.sh`.

This script:

- verifies the existence of the variable `USECASE_BASE`,
- set the value of `RESULT_PROD` if it does not exist,
- set the value of `CUDA_VISIBLE_DEVICES` to 0 if it does not exist,
- create the two directories `./logs` and `./kaaJson` if they not exist

The two directories mentioned previously are:

`./logs` to host the log files

`./kaaJson` to host the kaa configuration files

Then it launch the component Kaa: `kaa.py`.

On the first time, it indicates the variable environment values and collects the available plugins:



```
==[KAA prologue]=====
USECASE_BASE:/xxx/confiance/common/UC_Data_Folder
The environment variable 'RESULT_PROD' is not defined outside the script.
RESULT_PROD:/home/ConfianceAI/kaa
CUDA_VISIBLE_DEVICES:0
=====
Collection of plugins from the directory '/home/philippe.dejean/ConfianceAI/kaa/pluginsXAI'
LaTeX presence check for possible report compilation:
/usr/bin/latex
Plugin found : <library name>
library version : <version>
. . .
Plugin found : <library name>
library version : <version>
Collection of plugins from the directory '/home/philippe.dejean/ConfianceAI/kaa/pluginsUCXAI'
Plugin found : UC<usecase>_<library>
. . .
Plugin found : UC<usecase>_<library>
```

And the Kaa's TUI is deployed.

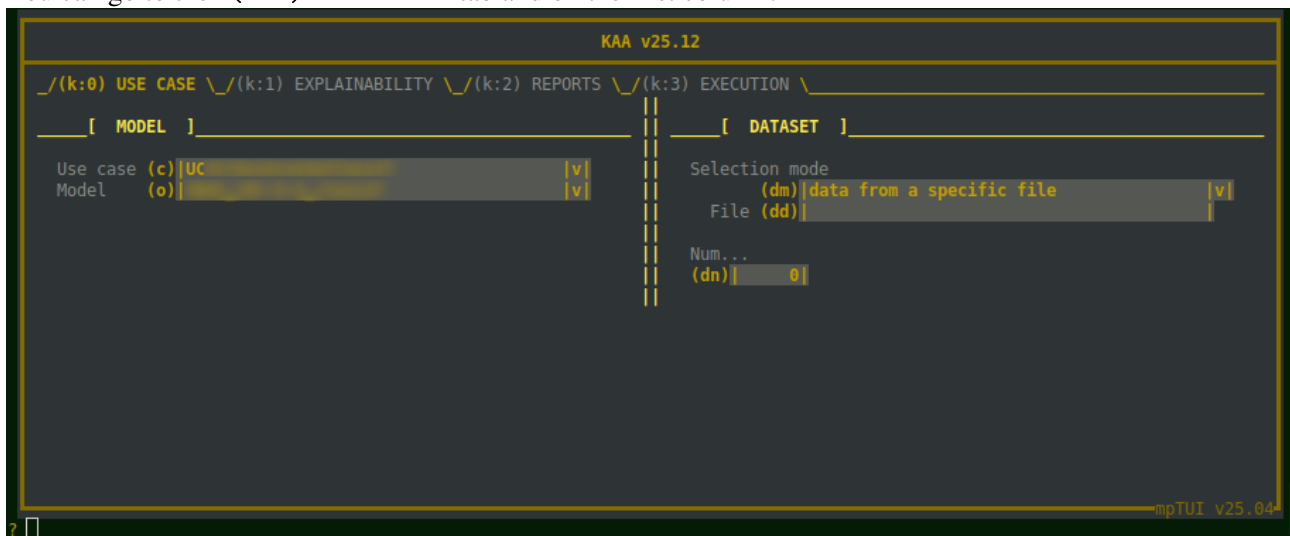
During the first use, the TUI widgets do not have specific and coherent values between them. It is therefore necessary to adjust them all well.

B.2. Kaa quick start

In this section we will measure metrics on explainability methods applied on a resnet50 model classifying welding (Renault use case).

B.2.1 Selection of the use case and the model to test

You can go to the "(k:0) USE CASE" tab and on the first column.



You can drop-down the "Use case" list (command c:v).



KAAs v25.12

/(k:0) USE CASE \/(k:1) EXPLAINABILITY \/(k:2) REPORTS \/(k:3) EXECUTION \

[MODEL]	[DATASET]
Use case (c) UC/ v	Selection mode
(c:0) >UC	(dm) data from a specific file v
(c:1) UC	File (dd)
(c:2) UC	
(c:3) UC	Num...
(c:4) UC	(dn) 0
(c:5) UCrenaultWeldingResnet	
(c:6) UC	
(c:7) UC	
(c:8) UC	
(c:9) UC	
(c:10) UC	
Model (o) v	

mpTUI v25.04

You can select the item "UCrenaultWeldingResnet" (command c:5 or c:'UCrenaultWeldingResnet'). If several models are available, the first item of the "Model" list becomes the current one.

You can drop-down the "Model" list (command o:v) to select the one you want.

KAAs v25.12

/(k:0) USE CASE \/(k:1) EXPLAINABILITY \/(k:2) REPORTS \/(k:3) EXECUTION \

[MODEL]	[DATASET]
Use case (c) UCrenaultWeldingResnet v	Selection mode
Model (o) Resnet v	(dm) data from a specific file v
	File (dd)
	Num...
	(dn) 0

mpTUI v25.04

B.2.2 Selection of the data to treat

You can go to the second column of the same tab.

Drop-down the "Selection mode" list (command dm:v).



KAAs v25.12

/(k:0) USE CASE \/(k:1) EXPLAINABILITY \/(k:2) REPORTS \/(k:3) EXECUTION \

[MODEL]	[DATASET]
Use case (c) UCRenaultWeldingResnet v	Selection mode
Model (o) Resnet v	(dm) data from a specific file v
	(dm:0) >data from a specific file
	(dm:1) one or more data from config file list
	(dm:2) all data from dataset
	(dm:3) selection of data from dataset
	File (dd)
	Num...
	(dn) 0

mpTUI v25.04

You can select the item "one or more data from config file list" (command dm:1). The editable area "File" disappears.

KAAs v25.12

/(k:0) USE CASE \/(k:1) EXPLAINABILITY \/(k:2) REPORTS \/(k:3) EXECUTION \

[MODEL]	[DATASET]
Use case (c) UCRenaultWeldingResnet v	Selection mode
Model (o) Resnet v	(dm) one or more data from config file list v
	Num...
	(dn) 0

mpTUI v25.04

You can select two data from the list (command dn:2).

KAAs v25.12

/(k:0) USE CASE \/(k:1) EXPLAINABILITY \/(k:2) REPORTS \/(k:3) EXECUTION \

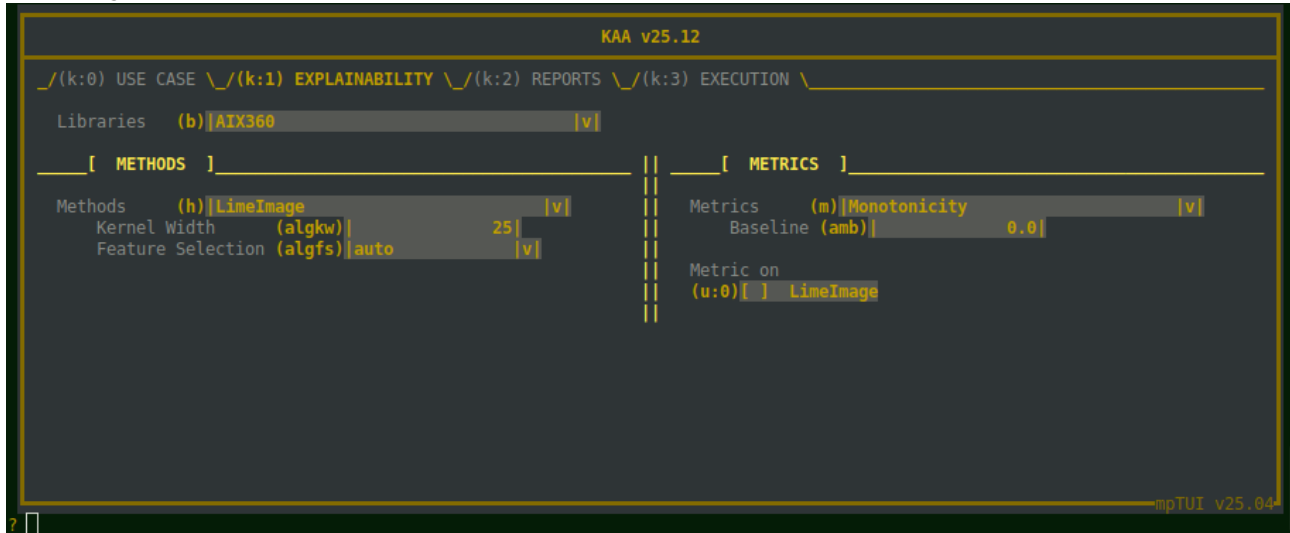
[MODEL]	[DATASET]
Use case (c) UCRenaultWeldingResnet v	Selection mode
Model (o) Resnet v	(dm) one or more data from config file list v
	Num...
	(dn) 2

mpTUI v25.04

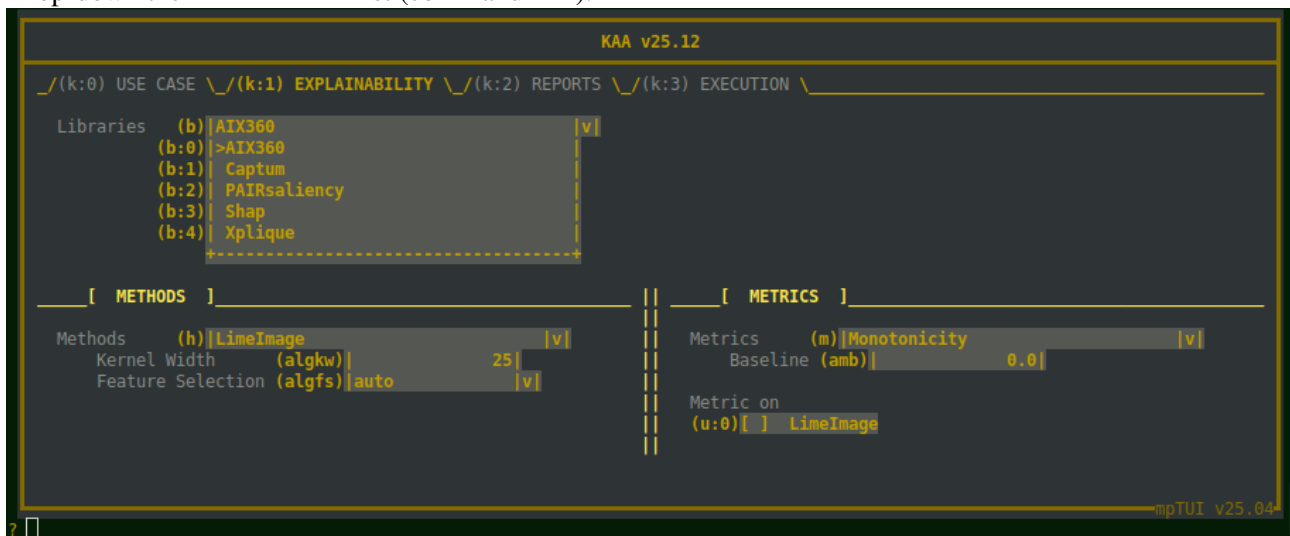


B.2.3 Selection of the explainability library

You can go to the "(k:1) EXPLAINABILITY" tab.



Drop-down the "Libraries" list (command b:v).



You can select the library "Xplique" (command b:4). The first method and metric of this library efficient for this use case are available and set as current elements.



KAAs v25.12

/(k:0) USE CASE \/(k:1) EXPLAINABILITY \/(k:2) REPORTS \/(k:3) EXECUTION \

Libraries (b)|Xplique |v|

[METHODS]				[METRICS]			
Methods	(h) Hsic	v		Metrics	(m) AverageStability	v	
Batch size	(xhbz)	64		Batch size	(xsbz)	64	
Grid size	(xhgz)	7		Radius	(xsrd)	0.1	
Number of designs	(xhnd)	500		Distance	(xsds)	12	
Perturbation function	(xhfp) inpainting	v		Number of samples	(xsns)	150	
				Metric on			
				(u:0)	Hsic		
				(u:1)	KernelShap		
				(u:2)	Lime		
				(u:3)	Occlusion		
				(u:4)	Rise		
				(u:5)	Sobol		

mpTUI v25.04

B.2.4 Selection of the explainability method

You can select the method "Occlusion" (commands h:v and h:3 or h:3 directly). The item number '3' is the same as in Metric on group of checkboxes.

KAAs v25.12

/(k:0) USE CASE \/(k:1) EXPLAINABILITY \/(k:2) REPORTS \/(k:3) EXECUTION \

Libraries (b)|Xplique |v|

[METHODS]				[METRICS]			
Methods	(h) Hsic	v		Metrics	(m) AverageStability	v	
(h:0) >Hsic				Batch size	(xsbz)	64	
(h:1) KernelShap				Radius	(xsrd)	0.1	
(h:2) Lime				Distance	(xsds)	12	
(h:3) Occlusion				Number of samples	(xsns)	150	
(h:4) Rise				Metric on			
(h:5) Sobol				(u:0)	Hsic		
-----+-----				(u:1)	KernelShap		
Batch size	(xhbz)	64		(u:2)	Lime		
Grid size	(xhgz)	7		(u:3)	Occlusion		
Number of designs	(xhnd)	500		(u:4)	Rise		
Perturbation function	(xhfp) inpainting	v		(u:5)	Sobol		

mpTUI v25.04

KAAs v25.12

/(k:0) USE CASE \/(k:1) EXPLAINABILITY \/(k:2) REPORTS \/(k:3) EXECUTION \

Libraries (b)|Xplique |v|

[METHODS]				[METRICS]			
Methods	(h) Occlusion	v		Metrics	(m) AverageStability	v	
Batch size	(xobz)	64		Batch size	(xsbz)	64	
Patch size	(xopz)	3		Radius	(xsrd)	0.1	
Patch stride	(xops)	3		Distance	(xsds)	12	
Occlusion Value	(xooov)	0.5		Number of samples	(xsns)	150	
				Metric on			
				(u:0)	Hsic		
				(u:1)	KernelShap		
				(u:2)	Lime		
				(u:3)	Occlusion		
				(u:4)	Rise		
				(u:5)	Sobol		

mpTUI v25.04



You can change "Patch size" and "Occlusion value" to 35 and 1 respectively (commands xopz:35 and xoov:1)

KAA v25.12

./(k:0) USE CASE \./(k:1) EXPLAINABILITY \./(k:2) REPORTS \./(k:3) EXECUTION \

Libraries (b)|Xplique|v|

[METHODS]

Methods	(h) Occlusion	v
Batch size	(xobz)	64
Patch size	(xopz)	35
Patch stride	(xops)	3
Occlusion Value	(xoov)	1

[METRICS]

Metrics	(m) AverageStability	v
Batch size	(xsbz)	64
Radius	(xsrdr)	0.1
Distance	(xsds)	l2
Number of samples	(xsns)	150

Metric on

(u:0)	Hsic
(u:1)	KernelShap
(u:2)	Lime
(u:3)	Occlusion
(u:4)	Rise
(u:5)	Sobol

mpTUI v25.04

B.2.5 Selection of the report

You can go to the "(k:2) REPORTS" tab and the second column.

KAA v25.12

./(k:0) USE CASE \./(k:1) EXPLAINABILITY \./(k:2) REPORTS \./(k:3) EXECUTION \

[VISUALISATION]

Colormap (tcm)|one colormap in the list|v|

List (tc)|hot|v|

Percentile (tp)|1| Transparency (ta)|0.6|

Normalise (tn)|x| Legend (tl)|

Font size (fz)|12|

Plot size

(tw:0)(o)	model size
(tw:1)()	data size

Image height (inch) (th)|5|+|

Quick report: Num.Columns (nc)|3|+|

[FULL REPORT]

Report on

(f:0) x	Explanations
(f:1) x	Metrics

Report the explanations sorted by (fe)|method|v|

Num. columns for method results(nch)|4|+|

Num. columns for metric results(ncq)|2|+|

mpTUI v25.04

If you want a report on explanations, select it (command f:0) and/or if you want a report on metrics, you can also select it (command f:1).

You can drop-down the "Report the explanations sorted by" list (command fe:v).



KAA v25.12

./(k:0) USE CASE \./(k:1) EXPLAINABILITY \./(k:2) REPORTS \./(k:3) EXECUTION \

[VISUALISATION]

Colormap (tcm) | one colormap in the list | v |
List (tc) | hot | v |

Percentile (tp) | 1 | Transparency (ta) | 0.6 |
Normalise (tn) | x | Legend (tl) | |
Font size (fz) | 12 |

Plot size
(tw:0) | o | model size
(tw:1) | | data size
Image height (inch) (th) | 5 | +- |

Quick report: Num.Columns (nc) | 3 | +- |

[FULL REPORT]

Report on
(f:0) | x | Explanations
(f:1) | x | Metrics

Report the explanations sorted by (fe) | method | v |
(fe:0) | >method |
(fe:1) | data |
+-----+-----+

Num. columns for method results (nch) | 4 | +- |
Num. columns for metric results (ncq) | 2 | +- |

mpTUI v25.04

B.2.6 Set path extension and traces in log

You can go to the "EXECUTION" tab (command k:3).

KAA v25.12

./(k:0) USE CASE \./(k:1) EXPLAINABILITY \./(k:2) REPORTS \./(k:3) EXECUTION \

[ACTIONS]

Possible actions
(a:0) | | Inference
(a:1) | | Explanations
(a:2) | | Plot explanations
(a:3) | | Metrics
(a:4) | | Plot metrics
(a:5) | | Quick report
(a:6) | | Full report

[(L)] Launch actions

[MANAGEMENT]

Log extension (L) | |
Prod. path extension (P) | Test |

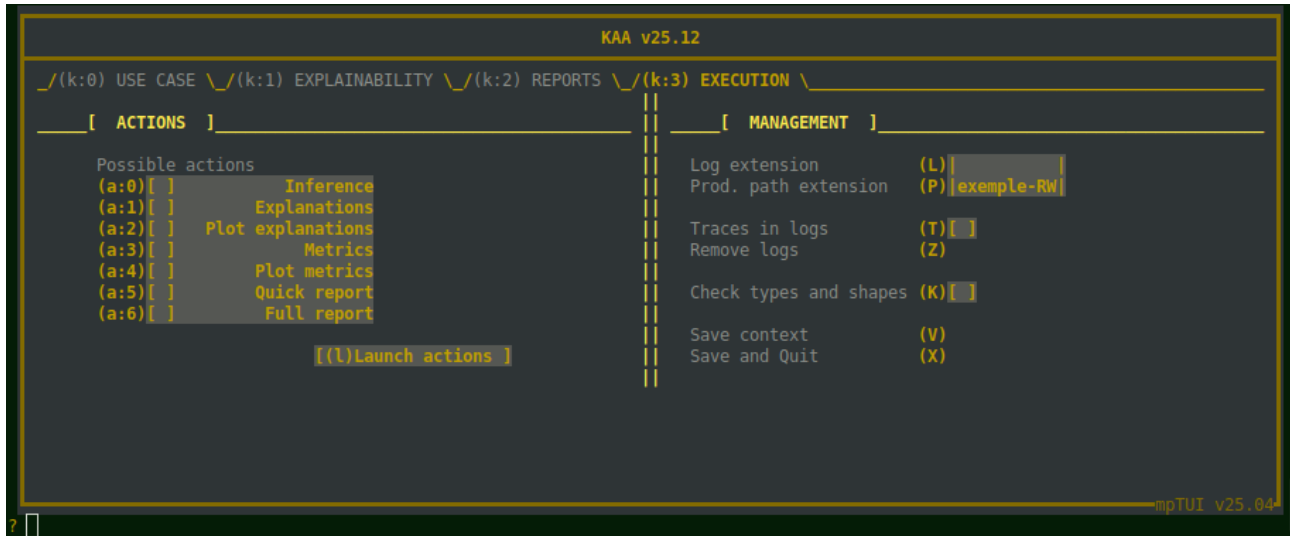
Traces in logs (T) | |
Remove logs (Z) | |

Check types and shapes (K) | |

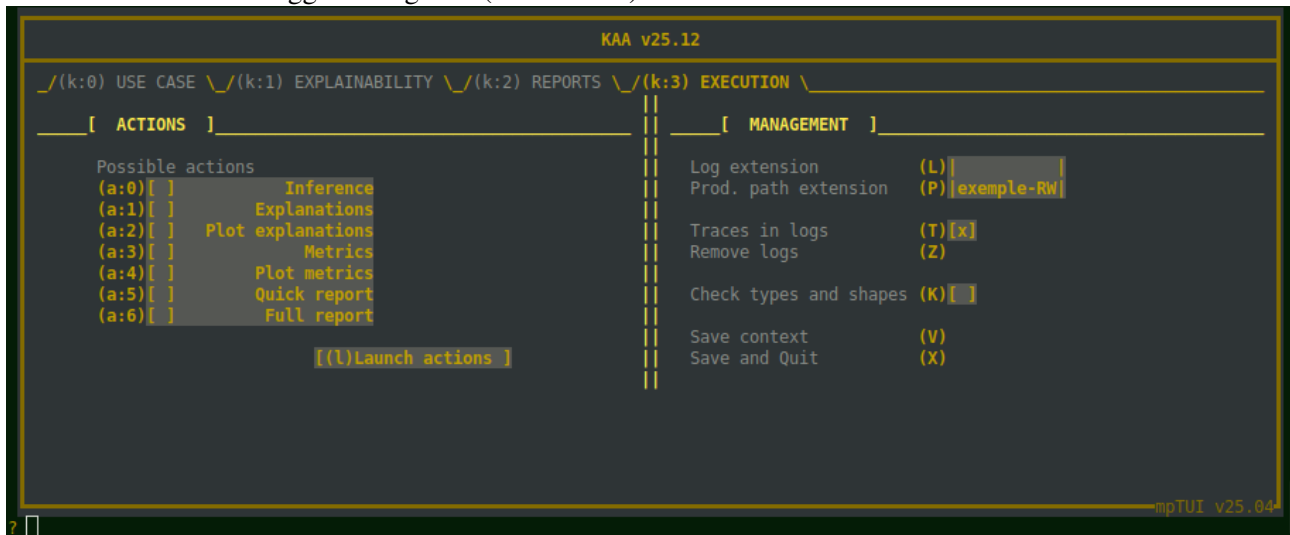
Save context (V) | |
Save and Quit (X) | |

mpTUI v25.04

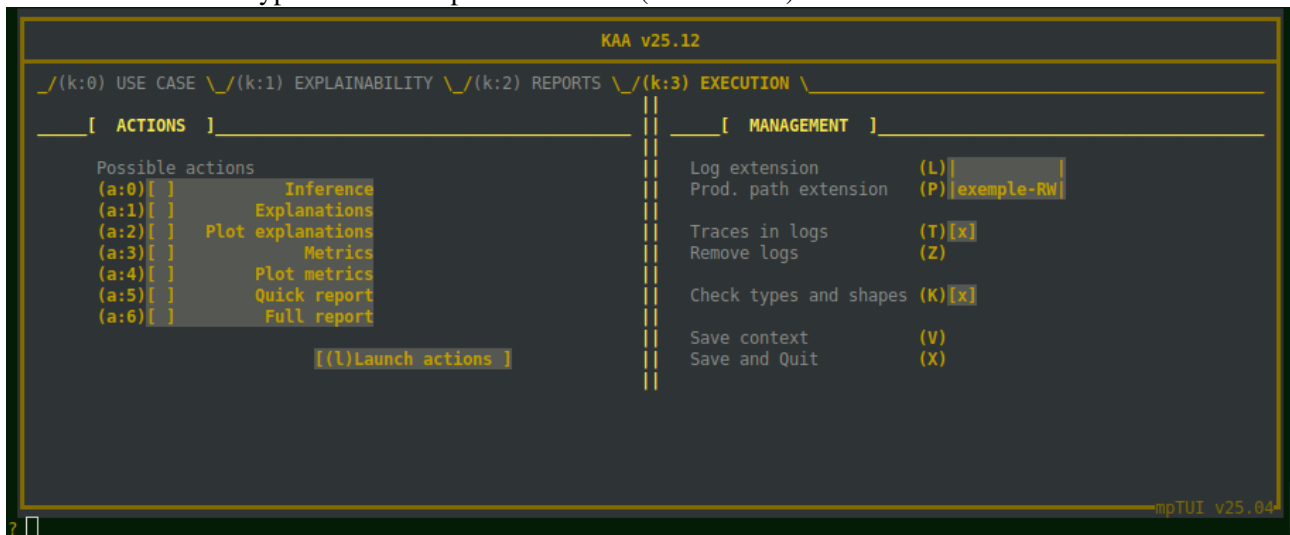
You can set the extension 'exemple-RW' to identify result path (command P:exemple-RW). All results will be saved in the path \$RESULT_PROD\$/dataProdexemple-RW.



You can set the traces logged in log files (command T).



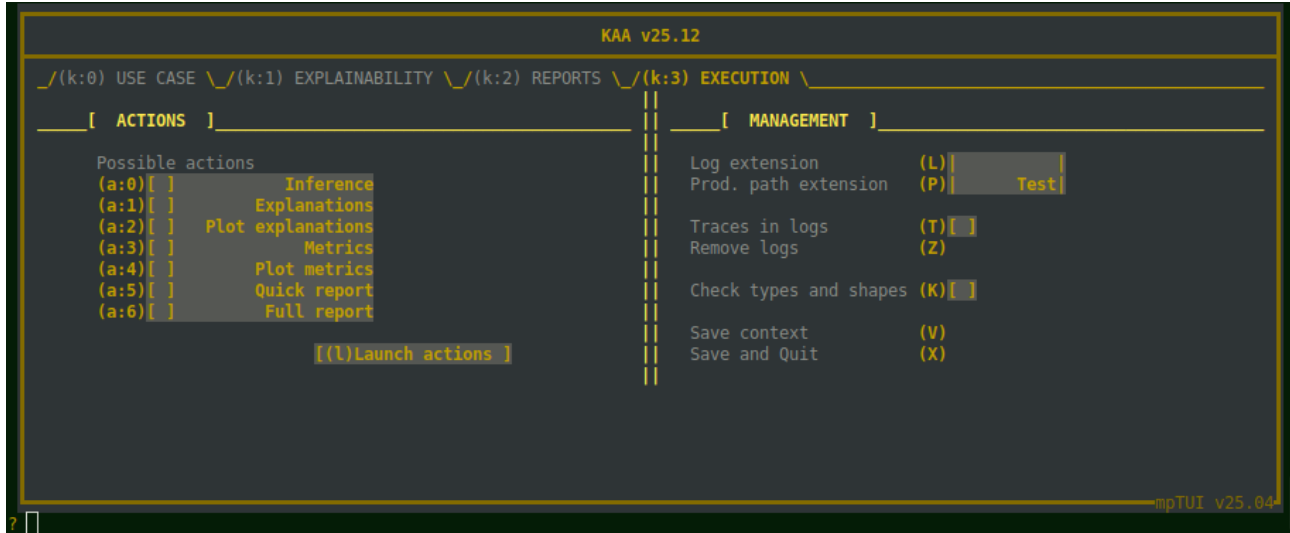
You can check if the types and the shapes are corrects (command K).



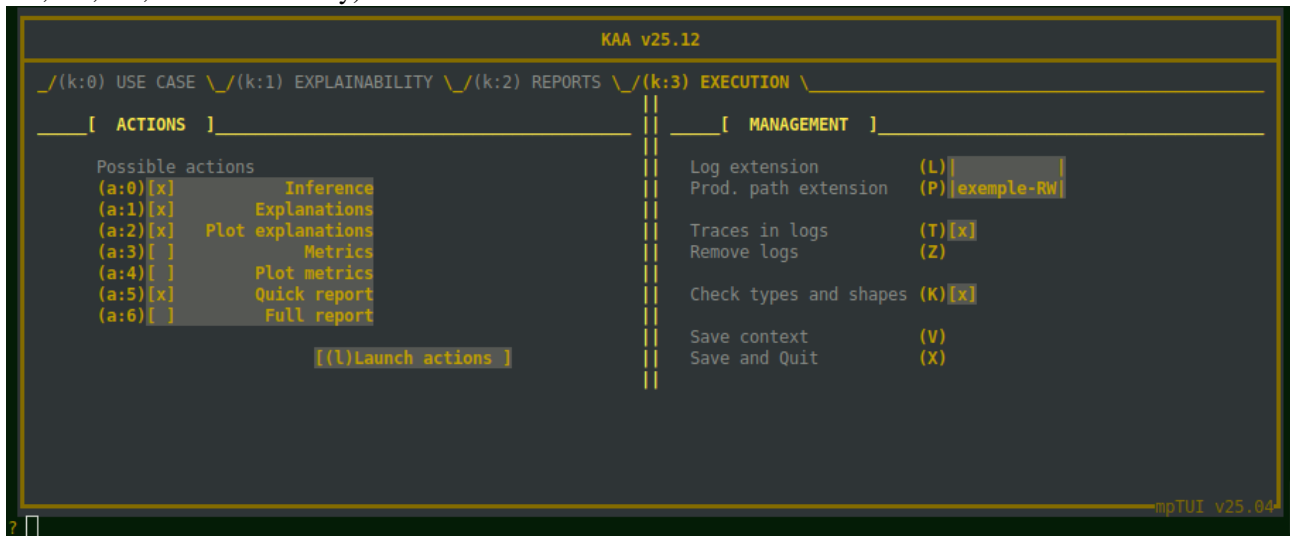


B.2.7 Launch actions

Go to the "(k:3) EXECUTION" tab on the first column.



Select the actions "Inference", "Explanations", "Plot explanations" and "Quick Report"¹ (commands a:0, a:1, a:2, a:5 successively).



You can launch actions and wait... (command l).

Kaa launches the treatments indicating the log file and the global json command file:

- Log file : ./logs/UCRenaultWeldingResnet-Resnet_Xplique_exempleRWr/Occlusion-RWr_o.log
- Json file: /home/.../kaa/kaaJson/kaaexampleRWr.json

B.2.8 Results through the log file

The log file Hsic-RWr_h0 mentions the important elements produced during processing:

```
1 Use case configuration load from 'pluginsUCXAI/UCRenaultWeldingResnet/UCRenaultWeldingResnet.json'
2 Get pNbData data from the list in dataset and associated model dictionary (pNbData = 2)
3 Nb treated data: 2
4 Results saved on subpath: Resnet/Occlusion__bz64_pz35_ps3_ov1
5 Action: Inference
6 .create Model from '/mnt/.../ETAIA/models/submission/challenge_solution/rn50_welding.pth'
```

¹The Quick report action force the "Visualisation" parameters Legend (tl:0) and "Plot size" as "Model size" (tw:0). If they are as after you choose "Quick report" action, the Quick report will be empty.



```
7      .create Data Preprocess Model from '/mnt/.../ETAIA/models/submission/challenge_solution/
      yolov11n_welder.pt'
8      UC_Renault_Welding_Inspection_Resnet -- Resnet
9      .inference
10     - /mnt/.../Full_dataset/labelled/c20/200617-0852_3207236_ - C20_OK.jpg
11 Prediction saved in ./dataProdeXampleRWr/Xplique/dataInference/Resnet/Occlusion__bz64_pz35_ps3_ov1
      /200617-0852_3207236_-C20_OK.json
12     - /mnt/.../Full_dataset/labelled/c20/190923-1320_2927975_ - C20_RETOUCHE.jpg
13 Prediction saved in ./dataProdeXampleRWr/Xplique/dataInference/Resnet/Occlusion__bz64_pz35_ps3_ov1
      /190923-1320_2927975_-C20_RETOUCHE.json
14 Results saved on subpath: Resnet/Occlusion__bz64_pz35_ps3_ov1
15 Action: Explain
16     .get explainer Occlusion
17 ==[Occlusion] =====
18 - Explanation all 2 data (12:41:58)
19 Duration: 49.25530934333801
20     Saved in './dataProdeXampleRWr/Xplique/dataExplanations/Resnet/Occlusion__bz64_pz35_ps3_ov1
      /200617-0852_3207236_-C20_OK.npy'
21     Saved in './dataProdeXampleRWr/Xplique/dataExplanations/Resnet/Occlusion__bz64_pz35_ps3_ov1
      /190923-1320_2927975_-C20_RETOUCHE.npy'
22     Saved in './dataProdeXampleRWr/Xplique/dataExplanations/Resnet/Occlusion__bz64_pz35_ps3_ov1/
      allData.npy'
23 Results saved on subpath: Resnet/Occlusion__bz64_pz35_ps3_ov1
24 Action: Plot explanations
25     .plot Explain #0
26         data: 200617-0852_3207236_ - C20_OK
27         plot with model input data shape
28         saved in: ./dataProdeXampleRWr/Xplique/dataPlotExplanations/Resnet/
      Occlusion__bz64_pz35_ps3_ov1/200617-0852_3207236_-C20_OK--legend
29     .plot Explain #1
30         data: 190923-1320_2927975_ - C20_RETOUCHE
31         plot with model input data shape
32         saved in: ./dataProdeXampleRWr/Xplique/dataPlotExplanations/Resnet/
      Occlusion__bz64_pz35_ps3_ov1/190923-1320_2927975_-C20_RETOUCHE--legend
33 Results saved on subpath: Resnet/Occlusion__bz64_pz35_ps3_ov1
34 Action: Report
35     .Archive of the report generated in /home/.../kaa/dataProdeXampleRWr/Xplique/
      dataPlotExplanations/Resnet/dataProdeXampleRWr/Xplique/dataPlotExplanations/Resnet/
      Occlusion__bz64_pz35_ps3_ov1.tgz
36         contains 19 lines.
37 Quick report available in ./dataProdeXampleRWr/Xplique/dataPlotExplanations/Resnet/
      Occlusion__bz64_pz35_ps3_ov1/contactSheet.pdf
38 Actions end normally.
```

We notice:

- lines 2-3: the data are identified and taken into account.
- line 4: a sub-directory is constructed with the method name and the parameters values.
- line 6: the model is correctly read.
- lines 9-13: the inferences are done and the result filenames are provided.
- lines 17-23: the Occlusion method is applied on data and the result filenames are provided.
- lines 24-33: the plot of explanations are done and the result filenames are provided.
- lines 34-37: the quick report is generated and stored in a tgz file to be easily download. The pdf file is eventually compiled from $\text{\LaTeX}$ files.
- line 38: the run is ok.

B.2.9 Selection of another explainability method

You can repeat the previous steps with the Rise method with `Nb.samples` parameter to 2000 (commands `h:4` and `xrns:2000`)



KAAs v25.12

/(k:0) USE CASE \/(k:1) EXPLAINABILITY \/(k:2) REPORTS \/(k:3) EXECUTION \

Libraries (b)|XpLique |v|

[METHODS]				[METRICS]			
Methods	(h) Rise		v	Metrics	(m) AverageStability		v
Batch size	(xrbz)	64		Batch size	(xsbz)	64	
Number of samples	(xrns)	2000		Radius	(xsrdr)	0.1	
Grid size	(xrgz)	7		Distance	(xsds)	12	
Presence probability	(xrpp)	0.5		Number of samples	(xsns)	150	
				Metric on			
				(u:0)	Hsic		
				(u:1)	KernelShap		
				(u:2)	Lime		
				(u:3)	Occlusion		
				(u:4)	Rise		
				(u:5)	Sobol		

mpTUI v25.04

You can directly launch actions and wait... (command 1).

B.2.10 Selection of explainability metric

You can go to the "(Q) METRICS" sub-tab and select the CausalFidelity-Deletion metric (commands m:v then m:1) and you can adjust the steps parameter to 100 (command xdst:100).

KAAs v25.12

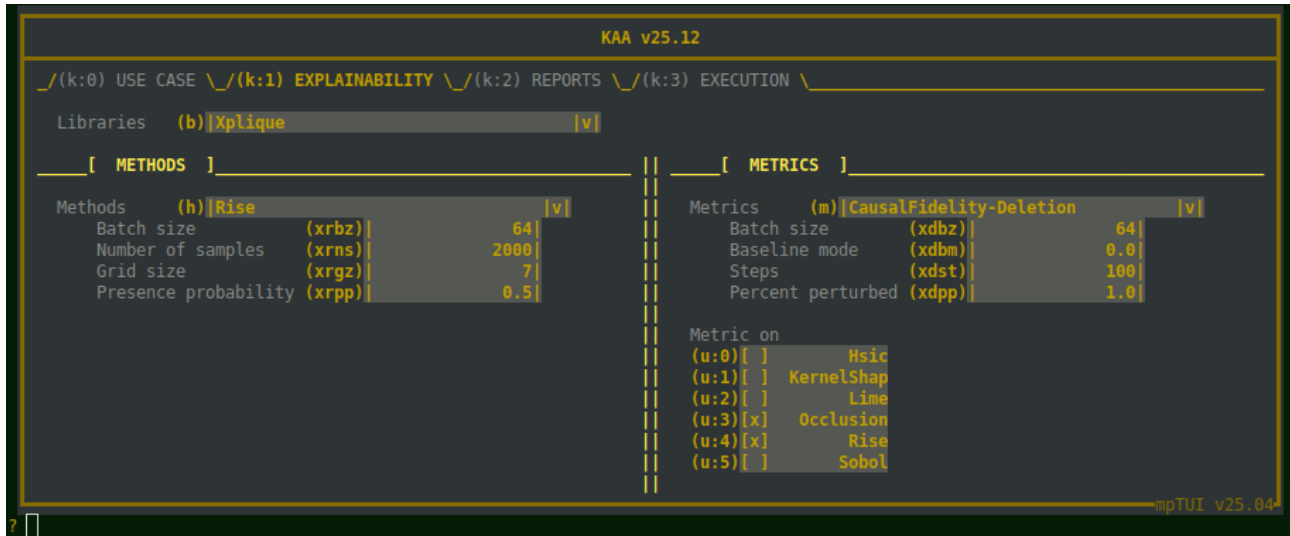
/(k:0) USE CASE \/(k:1) EXPLAINABILITY \/(k:2) REPORTS \/(k:3) EXECUTION \

Libraries (b)|XpLique |v|

[METHODS]				[METRICS]			
Methods	(h) Rise		v	Metrics	(m) CausalFidelity-Deletion		v
Batch size	(xrbz)	64		Batch size	(xdbz)	64	
Number of samples	(xrns)	2000		Baseline mode	(xdbm)	0.0	
Grid size	(xrgz)	7		Steps	(xdst)	100	
Presence probability	(xrpp)	0.5		Percent perturbed	(xdpp)	1.0	
				Metric on			
				(u:0)	Hsic		
				(u:1)	KernelShap		
				(u:2)	Lime		
				(u:3)	Occlusion		
				(u:4)	Rise		
				(u:5)	Sobol		

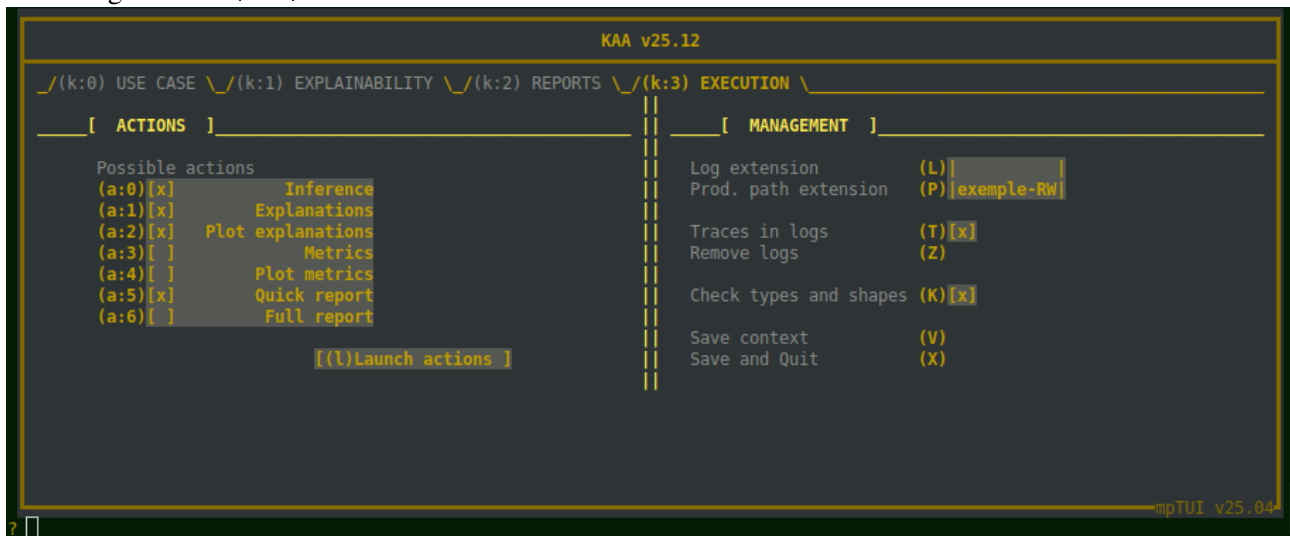
mpTUI v25.04

Then select methods to apply on by checking the corresponding checkboxes (commands u:3 and u:4 for Occlusion and Rise).

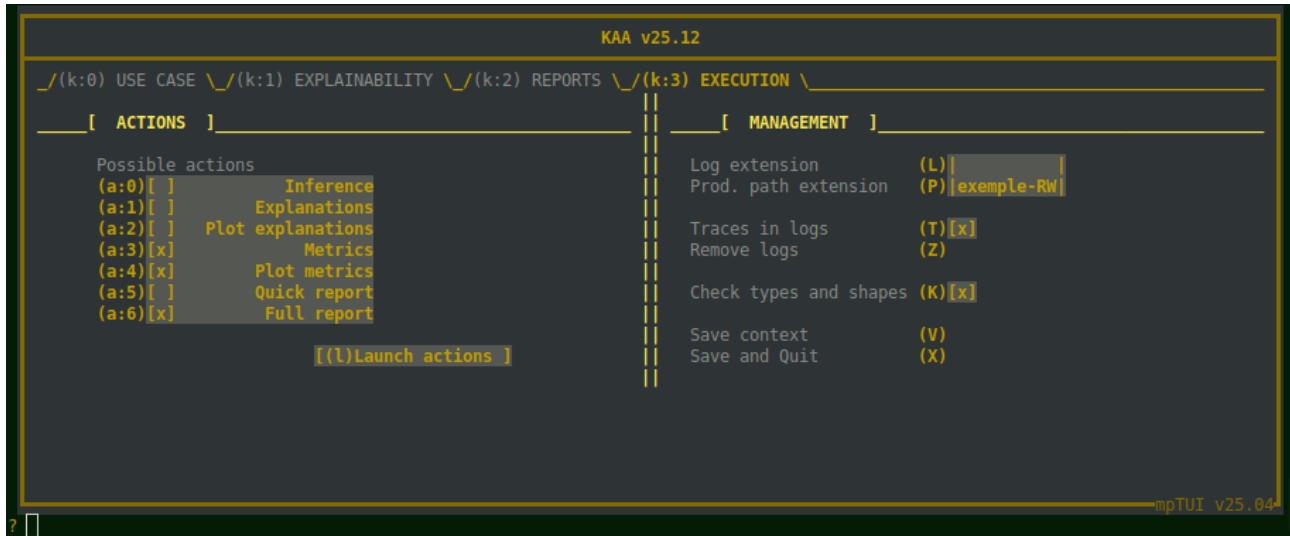


B.2.11 Launch actions

You can go to the "(k:3) EXECUTION" tab on the first column.



You can inverse selection (command a:i) or deselect all and select "Metrics" and "Plot metrics" actions (commands a:d, a:3, a:4).



You can launch actions and wait... (command 1).

Kaa launches the treatments indicating the log file and the global json command file:

Launch actions Metric.PlotM.FullReport

- Log file :

↪ ./logs/UCRenaultWeldingResnet-Resnet_Xplique_exempleRWr/CausalFidelity-Deletion-RWr_m_Metric.log

- Json file: /home/.../kaa/kaaJson/kaaexempleRWr.json

B.2.12 Results through the log file

The log file CausalFidelity-Deletion-RWr_m_Metric.log mentions the important elements produced during processing:

```
1 Use case configuration load from 'pluginsUCXAI/UCRenaultWeldingResnet/UCRenaultWeldingResnet.json'
2 Get pNbData data from the list in dataset and associated model dictionary (pNbData = 2)
3 Nb treated data: 2
4 Results saved on subpath: Resnet/Rise__bz64_ns2000_gz7_pp0-5
5 Action: Inference
6   .create Model from '/mnt/confiance/common/UC_Data_Folder/UC_Renault_Welding_Inspection/ETAIA/
   models/submission/challenge_solution/rn50_welding.pth'
7   .create Data Preprocess Model from '/mnt/confiance/common/UC_Data_Folder/
   UC_Renault_Welding_Inspection/ETAIA/models/submission/challenge_solution/yolov11n_welder.pt'
8   UC_Renault_Welding_Inspection_Resnet -/- Resnet
9   .inference
10  - /mnt/confiance/common/UC_Data_Folder/UC_Renault_Welding_Inspection/Full_dataset/labelled/c20
   /200617-0852_3207236_ - C20_OK.jpg
11 Prediction saved in ./dataProdexamplerWr/Xplique/dataInference/Resnet/Rise__bz64_ns2000_gz7_pp0
   -5/200617-0852_3207236_-C20_OK.json
12 - /mnt/confiance/common/UC_Data_Folder/UC_Renault_Welding_Inspection/Full_dataset/labelled/c20
   /190923-1320_2927975_ - C20_RETOUCHE.jpg
13 Prediction saved in ./dataProdexamplerWr/Xplique/dataInference/Resnet/Rise__bz64_ns2000_gz7_pp0
   -5/190923-1320_2927975_-C20_RETOUCHE.json
14 Action: Metric
15   .get metric Xplique:CausalFidelity-Deletion
16   .Metric on Occlusion method
17     Load explanation from ./dataProdexamplerWr/Xplique/dataExplanations/Resnet/
   Occlusion__bz64_pz35_ps3_ov1/allData.npy
18   .Metric on Rise method
19     Load explanation from ./dataProdexamplerWr/Xplique/dataExplanations/Resnet/
   Rise__bz64_ns2000_gz7_pp0-5/allData.npy
20 Metric CausalFidelity-Deletion
21 occlusion__bz64_pz35_ps3_ov1 auc value is: 0.497
22 rise__bz64_ns2000_gz7_pp0-5 auc value is: 0.168
23   results saved in './dataProdexamplerWr/Xplique/dataMetrics/Resnet/CausalFidelity-
   Deletion__bz64_bm0-0_st100_pp1-0-.json'
24   results saved in './dataProdexamplerWr/Xplique/dataMetrics/Resnet/CausalFidelity-
   Deletion__bz64_bm0-0_st100_pp1-0--s_AUCvalues.json'
25 Results saved on subpath: Resnet/Rise__bz64_ns2000_gz7_pp0-5
```




```

26 Action: Plot metric
27 Metric results loaded from './dataProdexampleRWr/Xplique/dataMetrics/Resnet/CausalFidelity-
    Deletion__bz64_bm0-0_st100_pp1-0.json'
28     saved in: ./dataProdexampleRWr/Xplique/dataPlotMetrics/Resnet/CausalFidelity-
    Deletion__bz64_bm0-0_st100_pp1-0--m_Occlusion__bz64_pz35_ps3_ov1.png
29     saved in: ./dataProdexampleRWr/Xplique/dataPlotMetrics/Resnet/CausalFidelity-
    Deletion__bz64_bm0-0_st100_pp1-0--m_Rise__bz64_ns2000_gz7_pp0-5.png
30     saved in: ./dataProdexampleRWr/Xplique/dataPlotMetrics/Resnet/CausalFidelity-
    Deletion__bz64_bm0-0_st100_pp1-0--g_OR.png
31 Action: Full Report
32   - Inference chapter
33     No data found
34   - Explanations chapter
35     .LaTeX report generated in /home/philippe.dejean/ConfianceAI/kaa/dataProdexampleRWr/fullReports/
    Resnet/UCRenaultWeldingResnet_fullReport__expl-data.tex
36     .Archive of the report generated in /home/philippe.dejean/ConfianceAI/kaa/dataProdexampleRWr/
    fullReports/dataProdexampleRWr/fullReports/Resnet.tgz
37     contains 162 lines.
38 Actions end normally.

```

We notice:

- lines 19-20: the explainability method are identified and taken into account.
- lines 21-24: the metric is computed and score value are provided.
- lines 25-29: the plot of metrics are done and the result filenames are provided.
- line 30: the run is ok.

B.3. Kaa command line

B.3.1 Help option

The command line option `--help` or `-h` displays all the options available:

Key	Extended key	Description
<code>-h</code>	<code>--help</code>	Show this help message and exit
<code>-D</code>	<code>--Debug</code>	Debug mode
<code>-c <file></code>	<code>--command <file></code>	Execute the command file
<code>-a <list></code>	<code>--activate <list></code>	Comma-separated list of use case plugins to activate (default 'All')
<code>-d <list></code>	<code>--deactivate <list></code>	Comma-separated list of use case plugins to deactivate (default '')
<code>-x <path></code>	<code>--pluginXAI <path></code>	Path to add your own pluginsXAI (optional)
<code>-u <path></code>	<code>--pluginUCXAI <path></code>	Path to the pluginsUCXAI (default <Kaa path>/pluginsUCXAI)
<code>-b <path></code>	<code>-useCaseBase <path></code>	Path to the directory containing the use cases (default USECASE_BASE environment variable)
<code>-r <path></code>	<code>-resultProd <path></code>	Path to the directory containing the produced results (default RESULT_PROD environment variable)

B.3.2 Usage of command file

B.3.2.1 Principle

A command file is a text file containing a set of commands from the Kaa's TUI menu. Commands can be on the same line separated by spaces or on separate lines. Commands are read and executed one after the other. Lines beginning with a semicolon are considered comment lines and ignored.

Here is the command file corresponding to the exemple above:

```

;=====

```



```
; RENAULT WELDING methodes
;=====

;-----USE CASE-----
k:0
c:'UCRenaultWeldingResnet
; model
o:'Resnet
; data
dm:1 dn:2

;-----EXPLAINABILITY-----
k:1
; LIBRARY
b:'Xplique

; METHODS
h:'Occlusion
; PARAMETRES
xobz:64 xopz:35 xops:3 xoov:1

;-----REPORTS-----
k:2
tl

;-----EXECUTION-----
k:3

; ACTIONS
; remove all
a:d
; selection
a:0 a:1 a:2 a:5

; MANAGEMENT
; log file
L:RWr_o
; path for results
P:exampleRWr
; trace
T:1

; RUN
l

;=====
X
```

B.3.2.2 Usage

Command files can be enabled via the '!cmd:<filename>' command which will load and execute the specified file. Command files can also be enabled by using them as parameters for starting the kaa.sh.

The syntax is then:

```
./kaa.sh -c <filename>
```

Regardless of the report value of executions in a log file saved in the configuration file, the trace is forced to 'yes' before reading the command file.

The interest of this use is to launch Kaa with the command nohup which allows to launch a script in the background. The logs being recorded, one can then disconnect from the machine while letting the processes take place.

The syntax is then:



```
nohup ./kaa.sh -c <filename> &
```

In this case, it is necessary to check that the command file ends with the X statement to exit and stop the Kaa process.

B.3.3 Activate or deactivate a Use Case

Via the command you will be able to activate or not a Plugin. By default, if you don't provide any parameter in the command, all use cases will be active.

If you provide the parameter -a only the following use cases will be activated. All the use cases must be separated with a "," without any spaces.

The syntax is:

```
./kaa.sh -a <useCaseName1>,<useCaseName2>
```

If you provide the parameter -d the following use cases will be deactivated. All the use cases must be separated with a "," without any spaces.

The syntax is:

```
./kaa.sh -d <useCaseName1>,<useCaseName2>
```

Remark: The deactivate option is priority.

B.3.4 Plugin paths

B.3.4.1 Library plugins

The library plugins are stored in the directory called `pluginsXAI` in the Kaa package. In addition to these ones, the user can use its own plugins by indicating the path of them with the command line option -x.

The syntax is:

```
./kaa.sh -x <path to the path containing personal library plugins>
```

The library plugins in the -x directory are added to the Kaa library plugins.

B.3.4.2 Use case plugins

Some use case plugins are provided as examples. They are stored in the directory called `pluginsUCXAI` in the Kaa package. The user can use its own use case plugins by indicating the path of them with the command line option -u.

The syntax is:

```
./kaa.sh -u <path to the path containing personal use case plugins>
```

The use case plugins in the -u directory replace the Kaa library plugins.



Part II

Integration manual



Introduction

The Kaa platform is built to drive explainability methods and metrics. It hosts a set of plugins:

- `pluginsXAI` : plugins of explainability libraries
- `pluginsUCXAI` : plugins of use cases vs explainability

This part is divided into four chapters:

1. The first is dedicated to the implementation of a new plugin for the integration of methods and metrics of an explainability library.
2. The second is dedicated to the implementation of a new plugin for the integration of a new study called use case by the implementation of available methods and metrics.
3. The third is dedicated to transversal elements: the description of the parameter dictionary transmitted through the plugins and the templates of the code source as provided in the document path of the distribution.
4. The fourth is dedicated to short documentations of the XAI libraries extract from their official repositories.



Chapter A

Library integration

Explainability library plugins are stored in the `pluginsXAI` directory. Since v24.06, a plugin is a folder (whose name is that of the library) which contains a set of three or five python files:

Let `<library_name>` for the name of the library,

- a python file with the library name `<library_name>.py` for the declaration of the library and the main functions,
- a python file with the library name `<library_name>_computeExplanations.py` dedicated to the explanation computation,
- a python file with the library name `<library_name>_plotExplanations.py` dedicated to the explanation plotting,
- optionally a python file with the library name `<library_name>_computeMetrics.py` dedicated to the metric computation,
- optionally a python file with the library name `<library_name>_plotMetrics.py` dedicated to the metric plotting,

Example: with two libraries (Library1 and Library2), the second one have no metric.

```
pluginsXAI
├── Library1
│   ├── Library1.py
│   ├── Library1_computeExplanations.py
│   ├── Library1_plotExplanations.py
│   ├── Library1_computeMetrics.py
│   └── Library1_plotMetrics.py
├── Library2
│   ├── Library2.py
│   ├── Library2_computeExplanations.py
│   └── Library2_plotExplanations.py
└── ...
```

In this section we present main elements to create a new `pluginsXAI`.

A.1. Plugin declaration in the `<library_name>.py` file

The declaration of the plugin is done in the `<library_name>.py` file.

A.1.1 Import libraries and other plugin files

Import all the library that you need and the library and precise its version.



```
from kaasrc.communis import colPrint
import kaasrc.plugin_collection

# -----
# To control the library version
versionPlugin = <library>.version (example: 0.0.1)
version = None
try:
    from <library_name> import __version__
    version = __version__
except Exception as err:
    print("Error:", err)
    colPrint("The library '<library_name>' is not or not properly installed.", "Info")
```

You should also import the 4 others python's files in the plugin.

```
import <library_name>_computeExplanations
import <library_name>_plotExplanations
import <library_name>_computeMetrics
import <library_name>_plotMetrics
```

A.1.2 The class and its initialisation

The plugin consists of a class with the same name as the file. This class inherits the Plugin class from plugin_collection:

```
import kaasrc.plugin_collection
import kaasrc.communis

class <library\_name>(kaasrc.plugin_collection.Plugin):
```

The function `__init__()` is defined as follows:

```
def __init__(self):
    super().__init__()
    self.nom = <library name>
    self.description = "<short.description>"
    self.clef = "<key>"
    self.inputXAIFramework = <class name> or tuple of <class name>
    self.outputXAIFramework = <class name> or tuple of <class name>
    self.methodes = {...}
    self.metriques = {...}
```

It contains the following attributes:

nom : which indicates the name of the library,

description : which provides a description of the library,

clef : which provides the access library key part in the command of the Kaa's TUI. The key must be unique among those of the libraries (c, i, a, p, s, x are already taken),

inputXAIFramework : which provides the type of the input that library methods require.

Example: (eTensor.EagerTensor, eTensor.EagerTensor) or (torch.Tensor, torch.Tensor),

outputXAIFramework : which provides the type of the output returned by library methods.

Example: eTensor.EagerTensor or torch.Tensor or {"Method1": (list, list, dict), "Method2": (list, list, np.ndarray)} if the type depends of the method,

methodes : which provides a description of all usable methods in dictionary form,

metriques : which provides a description of the usable metrics in dictionary form.



A.1.3 Usable methods and metrics

The two dictionaries of usable methods and metrics are built in the same way. They contain entries each corresponding to a method (resp. metric) constructed as follows:

- the entry is the method name (resp. metric),
- value is a tuple (access key, parameter list, method description)

```
"<name>" : ( '<key>', [ <parameters> ] , "<method description>" )
```

Each parameter of the list is a tuple of four or five values.

In the case of parameters with simple values, the tuple consisting of :

1. the parameter label,
2. the parameter key,
3. the parameter's default value,
4. the description of the method for the !i TUI command.

```
(<wording>, "<key>", <default value> , "<parameter description>")
```

In the case of parameters with list of possibles values, the tuple consisting of :

1. the parameter label,
2. the parameter key,
3. the default index of the list of possible values,
4. the list of possible values.
5. the description of the method for the !i TUI command.

```
(<wording>, "<key>", <default index>,  
        {"options":["possible values"]} , "<parameter description>")
```

When parameters are linked to the particular value of other parameters, the fourth element's dictionary contains a "lien" entry defined as follows:

```
{"lien": ( <key> , [<values>] ) }
```

For example, we will have the following dictionary of methods:

```
self.methodes={  
    "method A":('a',[( "parameter 1", "p1", 32, "param. 1 description"),  
                      ("parameter 2", "p2", 800, "param. 2 description"),  
                      ("parameter 3", "p3", 0, {"options":["option 1","option 2"]},  
                                                "param. 3 description"),  
                      ("parameter 4", "p4", 0, {"lien":(<lib.key>ap3,[1])},  
                                                "param. 4 description")  
                ],  
               "method A description"),  
    "method B":('b',[( "parameter 1", "p1", 64, "param. 1 description"),  
                      ("parameter 2", "p2", 100, "param. 2 description")  
                ],  
               "method B description")  
}
```

In the example above, for example, note the "lien" entry, which indicates that the parameter is linked to a particular value of another parameter designated by the complete access <lib.key>ap3 (from the <library key> + <method key> + <parameter key>): parameter 4 is linked to option 2 value of parameter 3. You can have both options and a "lien" for a parameter.



A.1.4 Absence of metrics

If there are no metrics in the library, `self.metrics` is set to `None`.

A.1.5 Method selection for metrics

In the event that a library's metrics can only be applied to a certain number of methods, it is possible to associate these methods with the declared metric. To do this, the triplet (`<key>`, `<list of parameters>`, `<metric.description>`) is augmented by the list of methods and becomes the quadruplet (`<key>`, `<list of parameters>`, `<list of methods>`, `<metric.description>`).

For example, we will have the following dictionary of metrics:

```
self.metrics={
    "metric C":('c',[("parameter 1", "p1", 32,"description of the parameter 1"),("parameter 2", "p2"
    , 800,"description of the parameter 2")], ["Method 1"], "description of the metric C"),
    "metric D":('d',[("parameter 1", "p1", 32,"description of the parameter 1"),("parameter 2", "p2"
    , 800,"description of the parameter 2")], "description of the metric C")
}
```

A.2. Plugin methods

Several methods are defined in the plugin. These methods will be used by the pluginsUCXAI plugins. These methods have different signatures depending on the explicability libraries, but we should find the methods for:

- the explanation calculation: `XAI_computeExplanations()`,
- the plot of the explanations: `XAI_plotExplanations()`,
- the metric calculation: `XAI_computeMetrics()` (optional, if metrics exist in library),
- the plot of the metrics: `XAI_plotMetrics()` (optional, if metrics exist in library),
- the writing of the quick report: `XAI_writeReport()`

The pseudo codes of the main code block of these methods are given below as examples. One of the parameters of these functions is the dictionary of parameters `pDictParams` read from the *json* file generated by Kaa when actions are launched. The contents of this dictionary are described in §E.1.

A.2.1 `XAI_computeExplanations()`

This method calculates explanations for a selected data set. The API of this method is:

```
#-----
## Compute explanations
# @param pDictParams : parameter dictionary
# @param pExplainClasses : Boolean to indicate if classes have to be treated
# @return parameter dictionary
def XAI_computeExplanations(self,pDictParams,pExplainClasses=True)
```

A pseudo code can be:

```
def XAI_computeExplanations(self,pDictParams,pExplainClasses=True):
    # shortcuts
    pDataType = pDictParams['datatype']
    pModelType = pDictParams['modeltype']

    # recording XAI framework for transmission in case of specific previous modifications
    pDictParams['inputXAIframework'] = self.inputXAIframework
    pDictParams['outputXAIframework'] = self.outputXAIframework

    if pDataType == "tabular":
```



```
pDictParams = <library_name>_computeExplanations.computeExplanationsTabText (pDictParams,
pExplainClasses)
elif pDataType == "image":
    if pModelType == "detection":
        pDictParams = <library_name>_computeExplanations.computeExplanationsObjDetection (
pDictParams)
    elif pModelType == "segmentation":
        pDictParams = <library_name>_computeExplanations.computeExplanationsObjSegmentation (
pDictParams)
    else:
        pDictParams = <library_name>_computeExplanations.computeExplanationsImage (pDictParams)
elif pDataType == "text":
    pDictParams = <library_name>_computeExplanations.computeExplanationsTexts (pDictParams)
return pDictParams
```

This method uses two functions in the class functions:

- `_getExplainer(self, pAiModel, pMethod, pMethodParam, ...)` which returns the *explainer* object corresponding to the `pMethod` method instantiated with the `pMethodParam` parameters,
- `_saveExplanations(self, pDictParams, pExplanation, pResultSavedOn, ...)` which saves explanations.

A.2.2 XAI_plotExplanations()

This method plots explanations. It is dedicated to the type of data processed and the explanation results. The API of this method is:

```
#-----
## Plot explanations
# @param pDictParams : parameter dictionary
# @param pClass : class to treat
# @param pItem : item to treat
# @param pFctExplainToPlot : function to apply to explanation before plotting
# @param pFctDataToPlot : function to apply to data before plotting
def XAI_plotExplanations (self, pDictParams, pClass=None, pItem=None, pFctExplainToPlot=None,
pFctDataToPlot=None)
```

A pseudo code for local explanations can be:

```
def XAI_plotExplanations (self, pDictParams, pClass=None, pItem=None, pFctExplainToPlot=None,
pFctDataToPlot=None)
# shortcuts
pDataType = pDictParams['datatype']
pModelType = pDictParams['modeltype']

# recording XAI framework for transmission in case of specific previous modifications
pDictParams['inputXAIframework'] = self.inputXAIframework
pDictParams['outputXAIframework'] = self.outputXAIframework

# Tracé
if pDataType == "tabular":
    pDictParams = <library_name>_plotExplanations.plotExplanationsTabText (pDictParams,
pFctExplainToPlot, pFctDataToPlot)
elif pDataType == "image":
    if pModelType == "detection":
        pDictParams = <library_name>_plotExplanations.plotExplanationsObjDetection (pDictParams,
pFctExplainToPlot, pFctDataToPlot)
    elif pModelType == "segmentation":
        pDictParams = <library_name>_plotExplanations.plotExplanationsObjSegmentation (
pDictParams, pFctExplainToPlot, pFctDataToPlot)
    else:
        pDictParams = <library_name>_plotExplanations.plotExplanationsImage (pDictParams,
pFctExplainToPlot, pFctDataToPlot)
elif pDataType == "text":
```



```
pDictParams = <library_name>_plotExplanations.plotExplanationsTexts(pDictParams,  
pFctExplainToPlot, pFctDataToPlot)
```

This method uses a function in the class functions:

- `_loadExplanations(self, pDictParams, pExplanation, pResultSavedOn, ...)` which loads explanations.

A.2.3 XAI_computeMetrics()

This method calculates metrics for a set of calculated explanations. One of the parameters is the dictionary of `pDictParams` parameters read from the *json* file generated by Kaa when the actions are launched. The API of this method is:

```
#-----  
## Compute metrics  
# @param pDictParams : parameter dictionary  
def XAI_computeMetrics(self, pDictParams):
```

A pseudo code can be:

```
def XAI_computeMetrics(self, pDictParams)  
    # shortcuts  
    pDataType = pDictParams['datatype']  
    pModelType = pDictParams['modeltype']  
    xData, yPred = pDictParams['xy']  
  
    # recording XAI framework for transmission in case of specific previous modifications  
    pDictParams['inputXAIframework'] = self.inputXAIframework  
    pDictParams['outputXAIframework'] = self.outputXAIframework  
  
    # convert data and prediction to XAI framework  
    xData, yPred = kaasrc.communis.convertToXAIframework(pDictParams, self.inputXAIframework)  
    pDictParams['xy'] = xData, yPred  
  
    if pDataType == "tabular":  
        pDictParams = <library_name>_computeMetrics.computeMetricsTabText(pDictParams)  
    elif pDataType == "image":  
        if pModelType == "detection":  
            pDictParams = <library_name>_computeMetrics.computeMetricsObjDetection(pDictParams)  
        elif pModelType == "segmentation":  
            pDictParams = <library_name>_computeMetrics.computeMetricsObjSegmentation(pDictParams)  
        else:  
            pDictParams = <library_name>_computeMetrics.computeMetricsImage(pDictParams)  
    elif pDataType == "text":  
        pDictParams = <library_name>_computeMetrics.computeMetricsTexts(pDictParams)
```

This method uses three class functions:

- `_getMetricEvaluator(self, pAiModel, pXinput, pYpred, pMetric, pMetricParam)` which returns the object `metricEvaluator` corresponding to the metric `pMetric` instantiated with the parameters allowing to measure the explanations of the model `pAiModel` applied on the inputs `pXinput` providing the predictions `pYpred`,
- `_getExplainer(self, pAiModel, pMethod, pMethodParam)` which returns the *explainer* object corresponding to the `pMethod` method instantiated with the `pMethodParam` parameters,
- `_loadExplanations(self, pDictParams, ficExplanation, pFullData, ...)` that reads the explanations and where `pFullData` indicate the number of explanations saved.

A.2.4 XAI_plotMetrics()

This method plots explanations. It is dedicated to the type of data processed and the measurement results. The API of this method is:



```
#-----  
## Plot metric results  
# @param pDictParams : parameter dictionary  
def XAI_plotMetrics(self, pDictParams):
```

A pseudo code can be:

```
def XAI_plotMetrics(self, pDictParams):  
    # shortcuts  
    pDataType = pDictParams['datatype']  
  
    # Tracé  
    if pDataType == "tabular":  
        pDictParams = <library_name>_plotMetrics.plotMetricsTabText (pDictParams)  
    elif pDataType == "image":  
        pDictParams = <library_name>_plotMetrics.plotMetricsImage (pDictParams)  
    elif pDataType == "text":  
        pDictParams = <library_name>_plotMetrics.plotMetricsTexts (pDictParams)
```

A.2.5 XAI_writeReport()

This method plots explanations. It is dedicated to the type of data processed and the measurement results.
The API of this method is:

```
#-----  
## Write report function : generate a latex report of plots and other elements  
# @param pDictParams : parameter dictionary  
def XAI_writeReport(self, pDictParams):
```

A pseudo code can be:

```
def XAI_writeReport(self, pDictParams):  
    # shortcuts  
    pDataType = pDictParams['datatype']  
    pModelType = pDictParams['modeltype']  
  
    if pDataType == "tabular":  
        kaasrc.communs.writeReportTabText (pDictParams, pClasses=pDictParams['classes'], pSuffixes=["  
", "mean"])  
    elif pDataType == "image":  
        if pModelType == "detection":  
            kaasrc.communs.writeReportObjDetection (pDictParams)  
        elif pModelType == "segmentation":  
            kaasrc.communs.writeReportObjSegmentation (pDictParams)  
        else:  
            kaasrc.communs.writeReportImage (pDictParams)  
    elif pDataType == "text":  
        kaasrc.communs.writeReportTabText (pDictParams, pClasses=pDictParams['classes'], pSuffixes=["  
", "mean"])
```

This method uses the Kaa functions writeReportxxx() in kaasrc package

A.2.6 Compute Explanations methods in the <library_name>_computeExplanations.py file

A.2.6.1 Import libraries

Import all the methods of the library

```
import os, time, json, datetime  
import numpy as np
```



```
from kaasrc.communs import colPrint
import kaasrc.communs
import kaasrc.controles

try:
    from <library_name> import MethodA, MethodB
except Exception as err:
    print("Error:", err)
    colPrint("The library '<library_name>' is not or not properly installed.", "Info")
```

A.2.6.2 Functions in the file

First of all, you can create a function `getExplainer()` which create an instance of the explainability method. Here is the pseudo-code of this function:

```
# -----
## Create an instance of the explainability method
# @param pDictParams : parameter dictionary
# @param pIndex : in case of PartitionTabular method: index table to explain
# @return method instance
def getExplainer(pDictParams):
    # shortcuts
    pMethod = pDictParams['method']
    pMethodParam = pDictParams[pMethod]
    pClasses = pDictParams['classes']

    _, aiModel = pDictParams['aiModel']

    if pMethod == "Method A":
        param1 = int(pMethodParam["p1"])
        param2 = int(pMethodParam["p2"])
        param3 = str(pMethodParam["p3"])
        explainer = MethodA(aiModel.useCase, p1=param1, p2=param2, p3=param3)
    elif pMethod == "Method B":
        ...
    return explainer
```

This file contains also all the functions required to compute explanations. There are one function by type of data accepted by the library. For example, if your library can handle tabular data, texts, image classification, image detections and image segmentations, you will have these five functions :

- `computeExplanationsTabText(pDictParams, pExplainClasses)`,
- `computeExplanationsObjDetection(pDictParams)`,
- `computeExplanationsObjSegmentation(pDictParams)`,
- `computeExplanationsImage(pDictParams)`,
- `computeExplanationsTexts(pDictParams)`

These functions calculate explanations for a selected data set.

The API of this method is:

```
# -----
## Compute explanation
# @param pDictParams : parameter dictionary
def computeExplanationsXX(self, pDictParams)
```

A pseudo code can be:

```
def computeExplanationsXX(self, pDictParams):
    # Shortcuts
    _, aiModel = pDictParams['aiModel']
```




```
xData,yPred = pDictParams['xy']
pDataProd = pDictParams['dataProd']
pReprtProd = pDictParams['reprtProd']
pNbData = pDictParams['nbData']
pDataPathList = pDictParams['dataPathList']
inputXAIFramework = pDictParams['inputXAIFramework']
outputXAIFramework = pDictParams['outputXAIFramework']

# control function for XAI_computeExplanations inputs
pParamSpecif = {}
kaasrc.controles.controlXAI_computeExplanationsInput(pDictParams, inputXAIFramework, xData,
yPred, aiModel.useCase, pNbData, __file__, pParamSpecif)

...
# Creation of the explainer instance
explainer=getExplainer(pDictParams)

...
# Launch explanations
print("==[%s] ===== "%pMethod, flush=True)
start = time.time()
explanations = []
for index in range(pNbData):
    explanations=explainer(xData,yPred,...)
    explanations.append(explanation)
end = time.time()
print("Duration:", end - start)

# control function for XAI_computeExplanations outputs
pParamSpecif = {}
kaasrc.controles.controlXAI_computeExplanationsOutput(pDictParams, outputXAIFramework,
explanations, aiModel.useCase, pNbData, __file__, pParamSpecif)
....
# Backup to disk
resultSavedOn=os.path.join(pDataProd,"dataExplain",pReprtProd)
_saveExplanations(pDictParams,explanations,resultSavedOn,...)
```

This method uses two functions:

- `getExplainer(pDictParams)` describes previously which returns the *explainer* object corresponding to the `pMethod` method instantiated with the `pMethodParam` parameters,
- `_saveExplanations(pDictParams,pExplanation,pResultSavedOn,...)` which saves explanations.

Then, to finish writing this file, you just have to define the `_saveExplanations()` function. Here is the pseudo-code of this function:

```
# -----
## Save explanation on file
# @param pDictParams : parameter dictionary
# @param pExplanation : explanation
# @param pResultSavedOn : path to save on
# @param pIndex : index of the treated data
# @param pExtension : extension for filename to identify explanation
# @param pMinMax : min/max vales to save (unused)
def _saveExplanations(pDictParams, pExplanation, pResultSavedOn, pIndex=None, pExtension="", pMinMax
=True):
    # shortcuts
    pDataList = pDictParams['dataList']

    if pIndex is not None:
        ficData = os.path.basename(pDataList[pIndex])
        dirName = os.path.dirname(pDataList[pIndex])
        ficExplanation = kaasrc.communs.createDirName(pResultSavedOn, os.path.join(dirName, '%s-Expl
%s.json'%(ficData, pExtension)))
        if pMinMax:
            minMaxJson = {}
            minMaxJson["min"]=str(np.min(pExplanation))
            minMaxJson["max"]=str(np.max(pExplanation))
```




```
        with open(ficExplanation, "w", encoding="utf-8") as f:
            json.dump(minMaxJson, f, indent=4)
        ficExplanation = ficExplanation.replace('.json', '.npz')
    else:
        ficExplanation = kaasrc.communis.noSpace(pResultSavedOn, 'Full-Expl%s.npz'%pExtension)

    np.save(ficExplanation, pExplanation)
    print("    Saved in '%s'"%ficExplanation, flush=True)
```

So you also need to write the `loadExplanations()` function to read the explanation saved with the previous function. Here is the pseudo-code of this function:

```
# -----
## Load explanation from file
# @param pDictParams : parameter dictionary
# @param pFicExplanation : explanation filename to load
# @param pNumData : number of data in explanations to load (to control)
# @param pFullData : are all data loaded? (True in metric computation process)
# @param pNumClasses : number of class of the use case
# @return explanations loaded
def loadExplanations(pDictParams, pFicExplanation, pNumData=None, pFullData=None, pNumClasses=None):
    # shortcuts
    _, aiModel = pDictParams['aiModel']
    outputXAIFramework = pDictParams['outputXAIFramework']

    if not os.path.exists(pFicExplanation):
        colPrint("The file containing the explanation '%s' does not exists."%pFicExplanation, "Info"
        )
        return None
    explanations = np.load(pFicExplanation, allow_pickle=True)

    # check explanations format
    pParamSpecif = {}
    kaasrc.controles.control_loadExplanationsOutput(pDictParams, outputXAIFramework, explanations,
    aiModel.useCase, __file__, pParamSpecif)

    return explanations
```

A.2.7 Plot Explanations methods in the `<library_name>_plotExplanations.py` file

A.2.7.1 Import libraries

Import all the methods of the library

```
import os, json
import numpy as np
import matplotlib.pyplot as plt
from kaasrc.communis import colPrint
import kaasrc.controles

try:
    from <library_name> import plot_function
except Exception as err:
    print("Error:", err)
    colPrint("The library '<library_name>' is not or not properly installed.", "Info")

import <library_name>_computeExplanations
```

A.2.7.2 Functions in the file

This file contains all the functions required to plot explanations. There are one function by type of data accepted by the library. For example, if your library can handle tabular data, texts, image classification, image detections



and image segmentations, you will have these five functions :

- `plotExplanationsTabText(pDictParams, pFctExplainToPlot, pFctDataToPlot)`,
- `plotExplanationsObjDetection(pDictParams, pFctExplainToPlot, pFctDataToPlot)`,
- `plotExplanationsObjSegmentation(pDictParams, pFctExplainToPlot, pFctDataToPlot)`,
- `plotExplanationsImage(pDictParams, pFctExplainToPlot, pFctDataToPlot)`,
- `plotExplanationsTexts(pDictParams, pFctExplainToPlot, pFctDataToPlot)`

These functions calculate explanations for a selected data set.

The API of this method is:

```
#-----  
## Plot explanations  
# @param pDictParams : parameter dictionary  
# @param pFctExplainToPlot : function to apply to explanation before plotting  
# @param pFctDataToPlot : function to apply to data before plotting  
def plotExplanationsXX(pDictParams, pFctExplainToPlot=None, pFctDataToPlot=None)
```

A pseudo code for local explanations can be:

```
def <library_name>_plotExplanations.plotExplanationsXX(pDictParams, pFctExplainToPlot=None,  
pFctDataToPlot=None):  
    # shortcuts  
    pNbData=pDictParams['nbData']  
    pDataProd=pDictParams['dataProd']  
    pReperProd=pDictParams['reperProd']  
    pDataList = pDictParams['dataList']  
    pData,_=pDictParams['xy']  
    outputXAIFramework = pDictParams['outputXAIFramework']  
  
    ...  
    # Data loop  
    for index in range(pNbData):  
        dataName = pDataList[index]  
  
        # Reading the data explanation  
        ficExplanation=kaasrc.communs.noSpace(os.path.join(pDataProd, "dataExplain", pReperProd,  
dirName), '%s-Expl.npy'%os.path.basename(dataName))  
        oneExplanation = <library_name>_computeExplanations.loadExplanations(pDictParams,  
ficExplanation)  
  
        # Application of a function for the explanation plot  
        if pFctExplainToPlot is not None:  
            oneExplanation=pFctExplainToPlot(pDictParams, oneExplanation, index)  
        else:  
            oneExplanation = kaasrc.communs.fctExplanationToPlotResized(pDictParams, oneExplanation,  
index)  
  
        data=pData[index]  
        # Application of a function to plot data  
        if pFctDataToPlot is not None:  
            data=pFctDataToPlot(data)  
  
        pParamSpecif = {}  
        kaasrc.controles.control_plotExplanationInput(pDictParams, outputXAIFramework, pPlotDataSize  
, oneExplanation, data, aiModel.useCase, __file__, pParamSpecif)  
  
        # Plot function  
        # Function in kaasrc  
        kaasrc.communs.plotExplanationOnImage(pDictParams, resultSavedOn, oneExplanation, data,  
dataName, index)  
        # Function define in this file  
        custom_function(resultSavedOn, oneExplanation, ...)  
        # Function imported from the library directly  
        plot_function(resultSavedOn, oneExplanation, ...)  
  
    ...
```



This method uses the function `<library_name>_computeExplanations.loadExplanations()` of the `<library_name>_computeExplanations.py` file. This function has already been described previously.

A.2.8 Compute Metrics methods in the `<library_name>_computeMetrics.py` file

A.2.8.1 Import libraries

Import all the methods of the library

```
import os, json
import numpy as np
from kaasrc.communs import colPrint
import kaasrc.controles

try:
    from <library_name> import MetricC
except Exception as err:
    print("Error:", err)
    colPrint("The library '<library_name>' is not or not properly installed.", "Info")

import <library_name>_computeExplanations
```

A.2.8.2 Functions in the file

First of all, you can create a function `_getMetricEvaluator()` which create an instance of the explainability metric.

Here is the pseudo-code of this function:

```
# -----
## Create an instance of the explainability metric
# @param pAiModel : model instance
# @param pData : data prepared for inference
# @param pPred : inference result
# @param pMetric : metric used for measurement
# @param pMetricParam : metric params used for measurement
# @param pOperator : specific operator
# @param pActivation : activation parameter
# @return method instance
def _getMetricEvaluator(pAiModel, pData, pPred, pMetric, pMetricParam, pOperator, pActivation):

    if pMetric == "Metric C":
        param1 = int(pMetricParam["p1"])
        param2 = int(pMetricParam["p2"])
        param3 = str(pMetricParam["p3"])
        metricEvaluator = MetricC(pAiModel.useCase, pData, pPred, p1=param1, p2=param2, p3=param3,
            operator=pOperator, activation=pActivation)
    elif pMetric == "Method D":
        ...
    return metricEvaluator
```

This file contains also all the functions required to computeMetrics. There are one function by type of data accepted by the library. For example, if your library can handle tabular data, texts, image classification, image detections and image segmentations, you will have these five functions :

- `computeMetricsTabText(pDictParams)`,
- `computeMetricsObjDetection(pDictParams)`,
- `computeMetricsObjSegmentation(pDictParams)`,
- `computeMetricsImage(pDictParams)`,
- `computeMetricsTexts(pDictParams)`



These functions calculate explanations for a selected data set.

One of the parameters is the dictionary of pDictParams parameters read from the *json* file generated by Kaa when the actions are launched.

The API of this method is:

```
#-----  
## Compute metrics  
# @param pDictParams : parameter dictionary  
def computeMetricsXX(self, pDictParams)
```

A pseudo code can be:

```
def computeMetricsXX(self, pDictParams):  
    # shortcuts  
    _, aiModel = pDictParams['aiModel']  
    pData, yPred = pDictParams['xy']  
    pMetrique = pDictParams['metric']  
    pMetricParam = pDictParams[pMetrique]  
    pNbData = pDictParams['nbData']  
    pCode = pDictParams['code']  
    pDataProd = pDictParams['dataProd']  
    pSuffixMetric = pDictParams["suffixMetric"]  
  
    ...  
    # Evaluator instance creation  
    metricEvaluator = _getMetricEvaluator(aiModel, pData, yPred, pMetrique, pMetricParam, ...)  
  
    ...  
    # creation of explainer and explanation dictionaries  
    dicoExplainers = {}  
    dicoExplanations = {}  
    pMethods4metric = pDictParams['methods4metric'].split(',')  
  
    for methode in pMethods4metric:  
        pMethodParam = pDictParams[methode]  
  
        dicoExplainers[methode] = <library_name>_computeExplanations.getExplainer(pDictParams)  
  
        ...  
        repertProd = os.path.join(pCode, "%s_%s"%(methode, pDictParams[methode]['suffixMethod']))  
        ficExplanation = kaasrc.communs.noSpace(os.path.join(pDataProd, "dataExplain", repertProd, 'Full-  
Expl.npy'))  
  
        dicoExplanations[methode] = <library_name>_computeExplanations.loadExplanations(pDictParams,  
        ficExplanation, ...)  
  
    # Launching measures  
    fichProd = os.path.join(pDataProd, "dataMetrics", pCode, '%s_%s'%(pMetrique, pSuffixMetric))  
  
    if metric == <nom de la metrique>:  
        score = <fonction>(pMetrique, metricEvaluator, pMethods4metric, dicoExplainers, dicoExplanations,  
        fichProd)  
  
    ...  
    return score
```

This method uses three class functions:

- `_getMetricEvaluator(pAiModel, pXinput, pYPred, pMetric, pMetricParam)` which returns the object `metricEvaluator` corresponding to the metric `pMetric` instantiated with the parameters allowing to measure the explanations of the model `pAiModel` applied on the inputs `pXinput` providing the predictions `pYPred`,
- `getExplainer(pDictParams)` which returns the *explainer* object corresponding to the `pMethod` method instantiated with the `pMethodParam` parameters which are contain in `pDictParams`,
- `loadExplanations(pDictParams, pResultSavedOn, ...)` who reads the explanations.



A.2.9 Plot Metrics methods in the <library_name>_plotMetrics.py file

A.2.9.1 Import libraries

Import all the methods of the library

```
import os, json
import kaasrc.graphMatplot

try:
    from <library_name> import plot_function
except Exception as err:
    print("Error:", err)
    TUI.TUIprint("The library '<library_name>' is not or not properly installed.", "Info")
```

A.2.9.2 Functions in the file

This file contains all the functions required to plot explanations. There are one function by type of data accepted by the library. For example, if your library can handle tabular data, texts, image classification, image detections and image segmentations, you will have these 5 functions :

- plotMetricsTabText(pDictParams),
- plotMetricsObjDetection(pDictParams),
- plotMetricsObjSegmentation(pDictParams),
- plotMetricsImage(pDictParams),
- plotMetricsTexts(pDictParams)

These functions calculate explanations for a selected data set.

The API of this method is:

```
#-----
## Plot metric results
# @param pDictParams : parameter dictionary
def plotMetricsXX(self, pDictParams)
```

No pseudo code can be supplied, due to the specificity of each metric in each library.



Chapter B

Existing pluginsXAI

The chapter above present the principles to create new pluginsXAI. Here, we present the specificities of existing plugins pluginsXAI to respect to implement plugins pluginsUCXAI.

For each pluginsXAI, we will detail the five main APIs.

Some libraries need specific function to work; they will be explained. A short description of the methods and metrics implemented in these library is available in chapter §F. A table of links to read more about them is available in chapter §F.6.2.6.

B.1. AIX360

This library is very complex due to different framework for the methods.

B.1.1 Main methods

The five method signatures are:

```
# -----  
## Compute explanations  
# @param pDictParams : parameter dictionary  
# @return parameter dictionary  
def XAI_computeExplanations(self, pDictParams):
```

```
# -----  
## Launch the explanations plotting  
# @param pDictParams : parameter dictionary  
# @param pFctExplainToPlot : function to apply to explanation before plotting  
# @param pFctDataToPlot : function to apply to data before plotting  
def XAI_plotExplanations(self, pDictParams, pFctExplainToPlot=None, pFctDataToPlot=None)
```

```
# -----  
## Compute metric  
# @param pDictParams : parameter dictionary  
# @return score  
def XAI_computeMetrics(self, pDictParams)
```

```
# -----  
## Plot metric results  
# @param pDictParams : parameter dictionary  
def XAI_plotMetrics(self, pDictParams)
```



```
# -----  
## Write report function : generate a latex report of plots and other elements  
# @param pDictParams : parameter dictionary  
def XAI_writeReport(self,pDictParams)
```

B.1.2 Specifical methods and functions

B.1.2.1 Same training prediction function

The XAI360 library uses a *same training prediction* function to retrieve train data with same prediction. This function has to be implemented in the python file UC<usecase>_AIX360.py of each pluginsUCXAI using AIX360 *protodash* method.

The template of this function is:

```
# -----  
## AIX360_sameTrainingPrediction function for AIX360 protodash function  
# @param pResultats : data to treat  
# @param pInputs : data in the training dataset  
# @param pEntry : index of treated prediction  
# @return inference results  
def AIX360_sameTrainingPrediction(pResultats,pInputs,pEntry)  
    ...  
    return data, prediction, example_good_prediction, index_of_good_prediction
```

This function have to be add to the global parameter dictionary in each UCXAI method (cf §C.5) using *protodash* function as follow:

```
pDictParams['sameTrainingPrediction']=AIX360_sameTrainingPrediction
```

B.1.2.2 Model method: predic_proba()

The XAI360 library uses the predict_proba() function for the models to be studied. This function, if it does not already exist, must be implemented in the UC_Model class of the pluginsUCXAI.

The API of this function is:

```
# -----  
## predict_proba function for AIX360 metrics  
# @param pData : input data  
# @result probability  
def predict_proba(self,pData)
```

B.1.2.3 Model method: megaPixels() and reduceTo1Band()

On image problems, some methods of XAI360 library uses a function to aggregate pixels into regions. This function must be implemented in the UC_Model class of the pluginsUCXAI. A function providing several methods is available in the kaasrc/communs.py python file. The megaPixels() function can call as follows:

```
# -----  
## Function to construct mega-pixels  
# @param pImage : image to treat
```




```
# @return map of mega-pixels
def megaPixels(self, pImage):
    return kaasrc.communis.megaPixels(pImage, self.megaPixelsParametres)
```

The megaPixelsParametres attribut is define in the `__init__()` function of the UC_Model class as follows, where reduceTo1Band is a method to reduce the multi-band data to a single band to compute watershed algorithm:

```
self.megaPixelsParametres = { "seuil":0,
                              "pretraitement":"none",
                              "noyau":3,
                              "nTo1band":self.reduceTo1Band }
```

B.1.2.4 Model method: `predict_from_text()`

The XAI360 library uses the `predict_from_text()` function when you use Lime method and you have a textual use case. This function, if it does not already exist, must be implemented in the UC_Model class of the pluginsUCXAI. In fact, the call function needs to take tokenized data as inputs when you work with texts. The API of this function is:

```
# -----
## predict_from_text function for AIX360 Lime method
# @param pData : datas/samples to be processed in text form
# @result inference results
def predict_from_text(self, pData)
```

B.2. Alibi

The Alibi library has not been completely studied. It has only been used on an Opinion mining use case, a special case of NLP.

Only these three methods are available:

```
# -----
## Compute explanations
# @param pDictParams : parameter dictionary
def XAI_computeExplanations(self, pDictParams)
```

```
# -----
## Launch the explanations plotting
# @param pDictParams : parameter dictionary
# @param pFctExplainToPlot : function to apply to explanation before plotting
# @param pFctDataToPlot : function to apply to data before plotting
def XAI_plotExplanations(self, pDictParams, pFctExplainToPlot=None, pFctDataToPlot=None)
```

```
# -----
## Write report function : generate a latex report of plots and other elements
# @param pDictParams : parameter dictionary
def XAI_writeReport(self, pDictParams)
```



B.2.1 Specifial methods and functions

Too complex to generalize actually.

B.3. Captum

The Captum library does not propose mature metrics.

Only these three methods are available:

```
# -----  
## Compute explanations  
# @param pDictParams : parameter dictionary  
def XAI_computeExplanations(self, pDictParams)
```

```
# -----  
## Plot of explanations  
# @param pDictParams : parameter dictionary  
# @param pFctExplainToPlot : function to apply to explanation before plotting  
# @param pFctDataToPlot : function to apply to data before plotting  
def XAI_plotExplanations(self, pDictParams, pFctExplainToPlot=None, pFctDataToPlot=None):
```

```
# -----  
## Write report function : generate a latex report of plots and other elements  
# @param pDictParams : parameter dictionary  
def XAI_writeReport(self, pDictParams)
```

B.3.1 Specifial methods and functions

B.3.1.1 Model method: megaPixels() and reduceTo1Band()

As for XAI360 library, on image problems, some methods of Captum library uses a function to aggregate pixels into regions.

Cf §B.1.2.3 for details.

B.3.1.2 Model method: predict_torchtensor()

The Captum library uses the `predict_torchtensor()` function when you have a textual use case. This function, if it does not already exist, must be implemented in the `UC_Model` class of the `pluginsUCXAI`. In fact, the call function needs to take a dictionary containing tokenized data as inputs when you work with texts (`{'inputs_ids':, 'attention_mask'}`). This additionnal function takes as input an array containing takenized data (`[inputs_ids_value, attention_mask_value]`).

The API of this function is:

```
# -----  
## predict_torchtensor function for Captum methods  
# @param pData : data to treat in the format [input, mask]  
# @result inference results  
def predict_torchtensor(self, pData)
```



B.4. PAIR saliency

The PAIR saliency library does not propose metrics. Only these three methods are available:

```
# -----  
## Compute explanation  
# @param pDictParams : parameter dictionary  
# @param pExplainClasses : boolean to identify if classes have to be explained independently  
# @return parameter dictionary  
def XAI_computeExplanations(self, pDictParams, pExplainClasses=True)
```

```
# -----  
## Plot explanations  
# @param pDictParams : parameter dictionary  
# @param pFctExplainToPlot : function to apply to explanation before plotting  
# @param pFctDataToPlot : function to apply to data before plotting  
def XAI_plotExplanations(self, pDictParams, pFctExplainToPlot=None, pFctDataToPlot=None)
```

```
# -----  
## Write report function : generate a latex report of plots and other elements  
# @param pDictParams : parameter dictionary  
def XAI_writeReport(self, pDictParams)
```

B.4.1 Special methods and functions

B.4.1.1 Call Model Function To Gradient function

Some PAIRSaliency methods need to access certain layers of the model to calculate a gradient. To indicate these gradient layers, the `callModelFunctionToGradient` entry of the global parameter dictionary is positioned in each function using PAIRSaliency methods.

The value -a dictionary- of this new parameter varies according to the method to be used:

For the *GradCam* method:

```
"callModelFunctionToGradient" : { 'CONVOLUTION_LAYER_VALUES': ... ,  
                                  'CONVOLUTION_OUTPUT_GRADIENTS': ... }
```

For the *Occlusion* method:

```
"callModelFunctionToGradient" : { 'OUTPUT_LAYER_VALUES': ... }
```

For the other methods:

```
"callModelFunctionToGradient" : { 'INPUT_OUTPUT_GRADIENTS': ... }
```

See Pairsaliency documentation for more details.

B.5. Shap

The PAIR saliency library does not propose metrics. Only the three methods are available:

```
# -----  
## Compute explanations  
# @param pDictParams : parameter dictionary  
def XAI_computeExplanations(self, pDictParams)
```



```
# -----  
## Plot explanations  
# @param pDictParams : parameter dictionary  
# @param pFctExplainToPlot : function to apply to explanation before plotting  
# @param pFctDataToPlot : function to apply to data before plotting  
def XAI_plotExplanations(self, pDictParams, pFctExplainToPlot=None, pFctDataToPlot=None)
```

```
# -----  
## Write report function : generate a latex report of plots and other elements  
# @param pDictParams : parameter dictionary  
def XAI_writeReport(self, pDictParams)
```

B.5.1 Specifical methods and functions

B.5.1.1 Model method: predict_from_textlist()

The SHAP library uses the `predict_from_textlist()` function when you have a textual use case. This function, if it does not already exist, must be implemented in the `UC_Model` class of the `pluginsUCXAI`. In fact, the call function needs to take tokenized data as inputs when you work with texts. This additionnal input wil take a list of text datas as input.

The API of this function is:

```
# -----  
## predict_from_textlist function for SHAP  
# @param pData : list of datas/samples to be processed in text form  
# @result list of inference results  
def predict_from_text(self, pData)
```

B.6. Xplique

The five methods are:

```
# -----  
## Compute explanation  
# @param pDictParams : parameter dictionary  
# @param pExplainClasses : boolean to identify if classes have to be explained independently  
# @return parameter dictionary  
def XAI_computeExplanations(self, pDictParams, pExplainClasses=True)
```

```
# -----  
## Plot explanations  
# @param pDictParams : parameter dictionary  
# @param pClass : class to treat  
# @param pItem : item to treat  
# @param pFctExplainToPlot : function to apply to explanation before plotting  
# @param pFctDataToPlot : function to apply to data before plotting  
def XAI_plotExplanations(self, pDictParams, pClass=None, pItem=None, pFctExplainToPlot=None, pFctDataToPlot=None)
```

```
# -----  
## Compute metrics  
# @param pDictParams : parameter dictionary
```



```
def XAI_computeMetrics(self,pDictParams)
```

```
# -----  
## Plot metric results  
# @param pDictParams : parameter dictionary  
def XAI_plotMetrics(self,pDictParams)
```

```
# -----  
## Write report function : generate a latex report of plots and other elements  
# @param pDictParams : parameter dictionary  
def XAI_writeReport(self,pDictParams)
```

B.6.1 Specifial methods and functions

B.6.1.1 Model method: predic_proba()

Unlike AIX360, some Xplique library methods are prevented by the presence of the `predict_proba` method in the `UC_Model` class. This method must therefore be removed when calculating explanations.

This must be done in the `UCXAI` method (cf §C.5) creating the `UC_Model` instance (`UCXAI_createModel()`) as follows:

```
if hasattr(self.UC.UC_Model,'predict_proba'): delattr(self.UC.UC_Model,'predict_proba')
```

B.6.1.2 Model method: megaPixels() and reduceTo1Band()

As for XAI360 library, on image problems, some methods of Captum library uses a function to aggregate pixels into regions.

Cf §B.1.2.3 for details.



Chapter C

Use case integration

Use case plugins handled by an explainability library are stored in the directory `pluginsUCXAI\<UC name>`. They consist of:

- a python file with the use case name `<nameUC>.py` grouping together the specific elements of the use case,
- a python file with the use case name `UC<nameUC>.py` for the generic elements for the integration of the use case,
- a json file with the use case name `UC<nameUC>.json` gathering information on the models and data processed,
- python files with the use case name and explainability library `UC<nameUC>_<nameLib>.py` consolidating library processing functions on the use case.

Example :

```
pluginsUCXAI
├── UCnameUC1
│   ├── nameUC1.py
│   ├── UCnameUC1.py
│   ├── UCnameUC1.json
│   ├── UCnameUC2_nameLib1.py
│   └── UCnameUC2_nameLib2.py
├── UCnameUC2
│   ├── nameUC2.py
│   ├── UCnameUC2.py
│   ├── UCnameUC2.json
│   ├── UCnameUC2_nameLib1.py
│   └── UCnameUC2_nameLib3.py
└── ...
```

These files are described in the following sections.

C.1. Use case definition in `<nameUC>.py`

This file is dedicated to the functionalities and specificities of the use case.

C.1.1 Specific functions

You'll need to find functions for:

- load and prepare data for model inference
- load and prepare model



- compute inference
- convert data from a framework to another
- convert prediction from a framework to another

C.1.2 Model class and methods

The object containing these functions is instantiated with the class called <nameUC>. This class has several methods to implement the required functions:

- `__init__()`: to initiate the instance of the class
- `fctPrepareModel()`: to compute data inference
- `fctPrepareData()`: to load data and prepare it as a model input
- `__call__()`: to the calculation of model inference on data distinguishing target inference from explanation inferences
- `convertDataToFramework()`: to convert data from a framework to another
- `convertPredictionToFramework()`: to convert prediction from a framework to another

Other methods can be found as:

- `predict_proba()`: cf §B.1.2.2

The pseudo codes for these methods are given below as examples:

C.1.2.1 `__init__()`

This method initiates the instance of the class. After specific initialisation of attributs, it call the method `fctPrepareModel()` that instantiate the model.

The pseudo code is:

```
#-----
## Initialisation function
# @param pCuda,pDevice : from CUDA detection
# @param pModelFilePath : model file path
# @param pXAIframework : library implementation framework
def __init__(self,pCuda,pDevice,pModelFilePath,pXAIframework):

    ## @brief use case identifier
    self.id = "<name of the use case>"
    ## @brief use case name
    self.name = "<text to describe the use case>"
    ## @brief number of image channels (0 si no image)
    self.numImageChannels = <number of bands>
    ## @brief number of channels in a data for model (0 si no image)
    self.numDataChannels = <number of bands>
    ## @brief input features
    self.features = None
    ## @brief number of classes
    self.numClasses = <number of classes>
    ## @brief flag to treat prediction as target of explainability one
    self.inferenceCible = True
    ## @brief inference results
    self.resPredictionsCibles = []
    ## @brief size of data for model inference
    self.inputModelSize = [<height>,<width>]
    self.outputModelSize = [<height>,<width>]
    ## @brief model framework
    self.inputModelFramework = <framework type>
    self.outputModelFramework = <framework type>
    ## @brief methode framework
    self.XAIframework=pXAIframework
    ## @brief cuda and device
    self.cuda = pCuda
    self.device = pDevice
    ## @brief model
    self.model = self.fctPrepareModel(pModelFilePath)
```




C.1.2.2 fctPrepareModel()

This method reads and instantiates the model from the file passed as parameter. It is a very use case specific method that corresponds to the first functionality.

A simple pseudo code is:

```
#-----  
## Initialisation function  
# @param pModelFilePath : model file path  
# @return the model loaded  
def fctPrepareModel(self,pModelFilePath):  
    ...  
    model = <specific code>  
    ...  
    return model
```

C.1.2.3 fctPrepareData()

This method is implemented to load data and prepare it as a model input. It corresponds to the second use case specific functionality.

A simple pseudo code is:

```
#-----  
## Function to prepare data from file to model input  
# @param pDataPath : data file path  
# @return tuple (data for model , its size)  
def fctPrepareData(self,pDataPath):  
    # load data  
    ...  
  
    # preprocess data  
    dataPreprocessed = ...  
  
    dataSize = dataPreprocessed. ...  
  
    return dataPreprocessed, datasize
```

C.1.2.4 __call__()

This method is implemented as the call function to the calculation of model inference on a single data. It corresponds to the encapsulation of the third functionality and prepare and store inference according to the step target inference vs explainability.

A simple pseudo code is:

```
#-----  
## Call function  
# @param pData : data to treat  
# @return inference result  
def __call__(self,pData):  
    # launch inference  
    inference = self.fctInferenceModel(pData)  
  
    ...  
  
    if self.inferenceCible:  
        self.resPredictionsCibles.append(inference)  
    else:
```



```
self.resPredictionsSamples.append(inference)
...

return inference
```

C.1.2.5 convertDataToFramework()

This function convert the preprocessed data from its framework to a specific one. A simple pseudo code is:

```
# -----
## convert data to dedicated framework
# @param pXData : data to convert
# @param pFramework : XAI framework
# @result converted data
def convertDataToFramework(self, pXData, pFramework):

    if pFramework == <framework>:
        pXData = ...
    elif pFramework == <framework>:
        pXData = ...
    ...
    else:
        TUI.TUIprint("The method 'convertDataToFramework' of the class dedicated to %s does not
        implement the conversion process from %s to %s"%(self.id, type(pXData), str(pFramework)), "Error")
    return pXData
```

C.1.2.6 convertPredictionToFramework()

This function convert the prediction from its framework to a specific one. A simple pseudo code is:

```
# -----
## convert prediction to dedicated framework
# @param pYPred : prediction
# @param pFramework : XAI framework
# @result converted prediction
def convertPredictionToFramework(self, pYPred, pFramework):
    if pFramework == <framework>:
        pYPred = ...
    elif pFramework == <framework>:
        pYPred = ...
    ...
    else:
        TUI.TUIprint("The method 'convertPredictionToFramework' of the class dedicated to %s does
        not implement the conversion process from %s to %s"%(self.id, type(pYPred), str(pFramework)), "
        Error")
    return pYPred
```

C.2. Use case definition in UC<nameUC>.py

Several functions and the class encapsulating the model are defined in the UC<nameUC>.py python file. This file is dedicated to the functionalities and specificities of the use case. A *Quick howto integration* guide is available in §C.6.

C.2.1 Specific functions

These functions will be called by the methods in UC<nameUC>_<nameLib>.py python file.

You'll need to find functions for:

- UC_createModel(): model creation



- UC_computeInference(): the calculation of model inference on data
- UC_ImageDataToPlot(): (optional) prepare the image for the plot. Imposed parameters
- UC_ExplanationToPlot(): (optional) prepare the explanation for the plot. Imposed parameters
- getModelParams(): (optional) get all the parameter options related to the model

The pseudo codes for these functions are given below as examples:

C.2.1.1 UC_createModel()

This method constructs the model file name from the json elements and creates the instance of the UC_Model1. class. A simple pseudo code is:

```
#-----  
## Create an instance of the model  
# @param pDictParams : parameter dictionary  
# @return the instance of the model  
def UC_createModel(pDictParams):  
    # Shortcuts  
    pUseCaseBase = pDictParams['useCaseBase']  
    pModelpath = pDictParams['modelpath']  
    pModelfile = pDictParams['modelfile']  
    pXAIframework=pDictParams['pluginXAI'].inputXAIframework  
    useCaseModelParams=pDictParams['useCaseModelParams']  
  
    # get model file  
    ficModel=os.path.join(pUseCaseBase,pModelpath,pModelfile)  
  
    # create model instance from 'model' parameter  
    aiModel=UC_Model(ficModel,pXAIframework,useCaseModelParams)  
  
    return aiModel
```

C.2.1.2 UC_computeInference()

This method loads the data to be processed, and performs the inferences. It then checks the inference results against the ground truth and returns the data, the inferences and the list of erroneously inferred data. A simple pseudo code is:

```
#-----  
## Compute the data inference with model  
# @param pAiModel : model instance  
# @param pDictParams : parameter dictionary  
# @return data, prediction, list of inference error against truth  
def UC_computeInference(pAiModel,pDictParams):  
    # Shortcuts  
    pNbData = pDictParams['nbData']  
    pClasses = pDictParams['classes']  
    pDataPathList = pDictParams['dataPathList']  
    pDataProd = pDictParams['dataProd']  
    pReperProd = pDictParams['reperProd']  
    pDataList=pDictParams['dataList']  
  
    # List of data with erroneous inference with respect to the ground truth.  
    listInferError=[]  
  
    # Test/Prediction data  
    xData=[]  
    xDataSize=[]  
    yPred=[]  
    resultSavedOn=os.path.join(pDataProd,"dataInference",pReperProd)  
  
    # Loop on data  
    for i in range(pNbData):  
        # data preparation  
        dataPreprocessed,dataSize=pAiModel.useCase.fctPrepareData(pDataPathList[i])
```



```
# Checkpoint 1: control function for fctPrepareData outputs
kaasrc.controles.controlFctPrepareDataOutput (pDictParams, dataPreprocessed, pAiModel.useCase,
__file__)

# set inference mode
pAiModel.setInferenceCible(True)

# Checkpoint 2: control function for Inference input
kaasrc.controles.controlInferenceInput (pDictParams, dataPreprocessed, pAiModel.useCase,
__file__)

# inference
prediction=pAiModel (dataPreprocessed)

# Checkpoint 3: control function for Inference outputs
kaasrc.controles.controlInferenceOutput (pDictParams, prediction, pAiModel.useCase, __file__)

# prediction backup with inference vs. truth check
isInferenceKO=pAiModel.savePrediction(i, pDataList, prediction, resultSavedOn, pClasses, dataSize
)

if len(isInferenceKO)>0:
    print("        ...inference error against truth: %d items."%len(isInferenceKO), flush=
True)
    listInferError.append((pDataPathList[i], isInferenceKO))

    xData.append(dataPreprocessed)
    xDataSize.append(dataSize)
    yPred.append(prediction)

# Saving the list of erroneously inferred data
if len(listInferError)>0:
    ficListInferError=os.path.join(resultSavedOn, "..", "listInferError.json")
    f=open(ficListInferError, "w")
    json.dump(listInferError, f, indent=2)
    f.close()

# transform xData and yPred to be conform to the format
...

# control function for all Inference outputs
kaasrc.controles.controlUC_computeInferenceOutput (pDictParams, xData, yPred, pAiModel.useCase,
pNbData, __file__)

return xData, xDataSize, yPred, listInferError
```

C.2.1.3 UC_ImageDataToPlot()

This method prepares the image for the plot. This function is optional and can be called in the function UCXAI_plotExplanations() of UC<nameUC>_<nameLib>.py python file. A simple pseudo code is:

```
#-----
## Prepare the image for the plot. Imposed parameters.
# @param pDictParams : parameter dictionary
# @param pPathImage : image path file
# @return image read and normalized
def UC_ImageDataToPlot (pDictParams, pPathImage):
    # Load image
    image = ... (pPathImage)
    # Normalisation
    ...
    # Resize according plot shape (tw TUI command)
    image=kaasrc.communs.fctDataToPlotResized (pDictParams, image)

    return image
```



C.2.1.4 UC_ExplanationToPlot()

This method prepares the explanation for the plot. This function is optional and can be called in the function UCXAI_plotExplanations() of UC<nameUC>_<nameLib>.py python file. A simple pseudo code is:

```
# -----  
## Prepare the explanation for the plot. Imposed parameters.  
# @param pDictParams : parameter dictionary  
# @param pExplanation : the explanation  
# @param pIndex : the data index in dataList (optionnel)  
# @param pItem : the item in tabular data, os the class (optionnel)  
# @return explanation to plot  
def UC_ExplanationToPlot(pDictParams,pExplanation,pIndex,pItem=""):  
    # Transformation  
    ...  
    # Resize according plot shape (tw TUI command)  
    pExplanation=kaasrc.communis.fctExplanationToPlotResized(pDictParams,pExplanation,pIndex)  
  
    return pExplanation
```

C.2.1.5 getModelParams()

This method gets all the parameter options related to the model thanks to the function define in <nameUC>.py. This function is optional and can be called in the initialisation of the MenuTUI in kaaTUIapplication.py python file. A simple pseudo code is:

```
# -----  
## Get all the parameter options related to the model  
# @return list of options that can be interpreted by the TUI  
def getModelParams():  
    return <nameUC>.getModelParams()
```

C.2.2 Model class and methods

The model is instanciased with the class UC_Model. This class has several methods dedicated to manage the model and dataset of the use case.

You'll need to find methods for:

- __init__(): to initiate the instance of the class
- setInferenceCible(): to set the boolean attribute inferenceCible
- savePrediction(): to save target prediction on disk

Other methods can be found as:

- megaPixels() and reduceTo1Band(): respectively to aggregate pixels into regions for *Lime* method like and to reduce multi bands data to 1 band for image data type cf §B.1.2.3

The pseudo codes for these methods are given below as examples:

C.2.2.1 __init__()

This method initiates the instance of the class. After specific initialisation of attributs, it initialize the use case with the class <UCname> in <UCname>.py.

The pseudo code is:

```
# -----  
## Initialisation function  
# @param pModelFilePath : model file path  
# @param pXAIframework : model format  
def __init__(self,pModelFilePath,pXAIframework):
```



```
## @brief megaPixels function parameters
self.megaPixelsParametres={"seuil":0,"pretraitement":"none",
                           "noyau":3,"nTolband":self.reduceTolBand}

## @brief is cuda? and device
self.cuda, self.device = kaasrc.communs.selDeviceForTorch()

## @brief use case initialisation
self.useCase=<UCName>.<UCName>(self.cuda, self.device, pModelFilePath, pXAIframework)
```

C.2.2.2 setInferenceCible()

This method is provide to set the boolean attribute inferenceCible.
The simple pseudo code is:

```
# -----
## Function to set inference: target vs explanation
# @param pCible : boolean
def setInferenceCible(self,pCible):
    self.inferenceCible=pCible
```

C.2.2.3 savePrediction()

This method is implemented to save target prediction on disk and control the inference correctness with ground truth if available. How to save inference on disk depends on the data itself and its use for visualizing explicability results.

This method returns two elements:

- a boolean to indicate the inference correctness,
- the prediction vector itself

A simple pseudo code is:

```
# -----
## Function to save inference and to control inference correctness
# @param pIndex : index of data to treat
# @param pDataPath : filename of data treated
# @param pYpred : model result
# @param pResultSavedOn : directory name to save result
# @param pClassesModele : prediction classes
# @param pDataSize : size of the data loaded
# @return list of item with inference vs truth KO
def savePrediction(self,pIndex,pDataPath,pYpred,pResultSavedOn,pClassesModele,pDataSize):

    # Get complete, brut prediction
    prediction = pYpred.numpy().tolist()

    # Cible of the inference prediction
    predictionCible = self.useCase.resPredictionsCibles[pIndex].numpy().tolist()

    # Index of the class in the prediction
    indexClasse = [predictionCible[i].index(1.0) for i in range(len(predictionCible))]

    # Name of predicted classes
    classesPredites = [pClassesModele[ind] for ind in indexClasse]

    # Scores of predictions
    score = [float(prediction[i][indexClasse[i]]) for i in range(len(prediction))]

    # truth
    verite = [int(x) for x in self.useCase.label[pIndex]]

    # control inference vs truth
    inferenceVSverite = [bool(verite[i]==indexClasse[i]) for i in range(len(verite))]
```



```
# list of items with an error
inferenceKO = [int(i+1) for i in range(len(verite)) if not inferenceVSverite[i]]

# save prediction on the disk
ficPrediction,_ = os.path.splitext(kaasrc.communs.createDirName(pResultSavedOn,pDataPath[pIndex]))
ficPrediction+="json"
print("Prediction saved in %s"%ficPrediction,flush=True)
dictJson={"classesModele":pClassesModele,
          "verite":verite,
          "prediction":prediction,
          "classe":classesPredites,
          "score":score,
          "inferenceVSverite":inferenceVSverite,
          "taille":[int(n) for n in pDataSize]}
f=open(ficPrediction,"w")
json.dump(dictJson, f, indent=2)
f.close()

return inferenceKO
```

C.3. Data files declaration in UC<nameUC>.json

Dataset files and model files are declared in the json file UC<nameUC>.json. This file consists of a dictionary model with an entry for the data file extension "dataExtension" and the following entries for each model:

```
"<model name>": {
  "classes"      : [<list of class labels>],
  "dataset"      : "<dataset path>",
  "datatype"     : "<type of dataset/use case>",
  "dataTrainPath": "<path to dataset of training>", // (optional, required for example based
  methods)
  "modelpath"    : "<model path>",
  "modelfile"    : "<model filename>",
  "truthlabelpath": "<path to the folder containing truth label files>", // (optional, usefull on
  object detection use case)
  "modeltype"    : "<type of model problem/use case>",
  "data"         : [<list of selected data>] }
```

For more details:

"classes" is the list of the name of the different classes,

"dataset" is the complement for dataset path added to USECASE_BASE environment variable or -b command line option value,

"datatype" is one of the elements "image","tabular","text",

"dataTrainPath" is the complement added to USECASE_BASE environment variable to indicate the data train set for the example-based explainability methods,

"modelpath" is the complement for model path added to USECASE_BASE environment variable or -b command line option value,

"modelfile" is the model filename in USECASE_BASE/<modelpath> (USECASE_BASE or -b command line option value),

"truthlabelpath" is the complement for truth label folder path added to USECASE_BASE environment variable",

"modeltype" is one of the elements "classification", "regression", "detection", "segmentation",

The entry "data" can take several forms according the "modeltype" and "datatype" values. In general, this entry consists of a simple list of filename without the "dataExtension". Other cases are possible:

- "datatype": "image" and "modeltype": "detection"

In case of object detection in images, the entry "data" is a list of elements as follows:



```
data: [ [ <filename>, <list of bounding box to treat> ]* ]
```

The <list of bounding box to treat> can be empty (all the bounding boxes found by the model are explained) or a list of bounding box numbers.

- "datatype": "image" and "modeltype": "segmentation"

In case of object segmentation in images, the entry "data" is a list of elements as follows:

```
data: [ [ <filename>, [ [<class>,<list of regions to treat> ]* | [ <class> ]* ] ] ]
```

The class is the class reference to explain.

The <list of regions to treat> can be empty (all the regions of the class found by the model are explained) or a list of region numbers.

If class is the only element of the list, we will explain the class instead of the regions.

This list of data can be declare in a simple text file and loaded with the dd command in Kaa's TUI. The three forms of that text file are:

- simple list : each data filename is on a line
- object detection : each data filename is on a line and the bounding box numbers to treat are separated with commas (no region number to explain all bounding baxes),
filename,number,number or only filename
- object segmentation: each data filename is on a line and the class / regions numbers to treat are separated with commas and semicolons: filename,class1:number1;number2,class2,class3:number1

C.4. Plugin declaration in UC<nameUC>_<nameLib>.py

A plugin is declared in the python file UC<nameUC>_<nameLib>.py.

C.4.1 The class and its initialization

The plugin consists of a class with the same name as the file. This class inherits the "Plugin" class from "kaasrc.plugin_collection":

```
import kaasrc.plugin_collection
import kaasrc.communs

import UC<nameUC>

class UC<nameUC>_<namelib>(kaasrc.plugin_collection.Plugin):
```

La fonction `__init__()` est définie comme suit :

```
def __init__(self):
    super().__init__()
    self.nom = "UC<nameUC>"
    self.description = "<short.description>"
    self.modeles = [<list of available models>]
    self.bibliotheque = "<library name>"
    self.methodes = [<list of library efficient methods>]
    self.metriques = [<list of library efficient metrics>] or None
    self.UC = UC<nameUC>
    self.resPredictionsCibles = None
```

It contains the attributes :

nom : which indicates the name of the use case,

description : provides a description of the use case,

models : provides a list of usable models,

library : indicates the name of the library,

methodes : provides a list of usable methods,



`metrics` : provides the list of usable metrics.
`UC` : provides the name of the use case.
`resPredictionsCibles` : which will contain the target predictions.

C.5. Plugin methods

Several methods are defined in the plugin. These methods will be used by Kaa. They therefore have imposed signatures.

You'll need to find the methods for :

- `UCXAI_createModel()`: to create the model class instance
- `UCXAI_computeInference()`: to the calculation of model inference on data
- `UCXAI_computeExplanations()`: to the calculation of explanations
- `UCXAI_plotExplanations()`: to plot explanations
- `UCXAI_writeReport()`: to create quick report in the form of a contact sheet

and for libraries containing metrics, the methods for :

- `UCXAI_computeMetrics()`: for calculating metrics
- `UCXAI_plotMetrics()`: to plot metrics

Depending on the specific use case, the implementation of these methods may contain *ad-hoc* elements. In most cases, these functions can be implemented as follows:

C.5.1 `UCXAI_createModel()`

This method generally uses the appropriate function in the `UC<nameUC>.py` python file. The pseudo code can simply be:

```
# -----  
## Create an instance of the model  
# @param pDictParams : parameter dictionary  
# @return the instance of the model  
# @remark Call directly UC_createModel() of UC model  
def UCXAI_createModel(self, pDictParams)  
    return self.UC.UC_createModel(pDictParams)
```

C.5.2 `UCXAI_computeInference()`

This method generally uses the appropriate function in the `UC<nameUC>.py` python file. The pseudo code can simply be:

```
# -----  
## Compute the data inference with model  
# @param pAiModel : model instance  
# @param pDictParams : parameter dictionary  
# @return data, prediction, list of inference error against truth  
# @remark Call directly UC_computeInference() of UC model  
def UCXAI_computeInference(self, pAiModel, pDictParams):  
    return self.UC.UC_computeInference(pAiModel, pDictParams)
```

C.5.3 `UCXAI_computeExplanations()`

This method generally uses the appropriate function in the `pluginsXAI`. The pseudo code can simply be:

```
# -----  
## Launch the explanations computation  
# @param pDictParams : parameter dictionary
```



```
# @remark Call directly XAI_computeExplanations() of XAI plugin
def UCXAI_computeExplanations(self, pDictParams):
    pDictParams['pluginXAI'].XAI_computeExplanations(pDictParams)
```

C.5.4 UCXAI_plotExplanations()

This method generally uses the appropriate function in the pluginsXAI. The pseudo code can simply be:

```
# -----
## Launch the explanations plotting
# @param pDictParams : parameter dictionary
# @remark Call directly XAI_plotExplanations() of XAI plugin
def UCXAI_plotExplanations(self, pDictParams):
    pDictParams['pluginXAI'].XAI_plotExplanations(pDictParams)
```

The XAI_plotExplanations function has two other optional parameter dedicated for:

1. pFctExplainToPlot function to tranform the explanation before plotting (resize, rotation, etc).
A "Foo" function is available in the kaasrc/communs.py file for test.
2. pFctDataToPlot function to transform input data to plot it with the explanation.

C.5.5 UCXAI_computeMetrics()

This method generally uses the appropriate function in the pluginsXAI. The pseudo code can simply be:

```
# -----
## Launch the metrics computation
# @param pDictParams : parameter dictionary
# @remark Call directly XAI_computeMetrics() of XAI plugin
def UCXAI_computeMetrics(self, pDictParams):
    pDictParams['pluginXAI'].XAI_computeMetrics(pDictParams)
```

C.5.6 UCXAI_plotMetrics()

This method generally uses the appropriate function in the pluginsXAI. The pseudo code can simply be:

```
# -----
## Launch the metrics plotting
# @param pDictParams : parameter dictionary
# @remark Call directly XAI_plotMetrics() of XAI plugin
def UCXAI_plotMetrics(self, pDictParams):
    pDictParams['pluginXAI'].XAI_plotMetrics(pDictParams)
```

C.5.7 UCXAI_writeReport()

This method generally uses the appropriate function in the pluginsXAI. The pseudo code can simply be:

```
# -----
## Launch the report writing as contact sheet
# @param pDictParams : parameter dictionary
# @remark Call directly XAI_writeReport() of XAI plugin
def UCXAI_writeReport(self, pDictParams):
    pDictParams['pluginXAI'].XAI_writeReport(pDictParams)
```



C.6. Quick howto integration

A quick method to integrate a new use case in Kaa is to identify three functionalities:

1. model instantiation,
2. data preparation as model input,
3. inference

C.6.1 Model instantiation

Each use case, each model is specific. Generally, the model is stored in a file declared in `UC<nameIC>.json` (cf §C.3).

The filename is then:

```
$USECASE_BASE$/<"modelpath" entry>/<"modelfile" entry>.
```

This functionality must read the model from the file, define its weights and parameters and provide an operational instance.

It is implemented in `fctLoadModel()` method of the class `UC_Model`.

C.6.2 Data preparation as model input

As for models, each use case, each dataset is specific. Generally, the data are stored in files declared in `UC<nameIC>.json` (cf §C.3).

The filenames are then:

```
$USECASE_BASE$/<"dataset" entry>/<"data" from list>.<"dataExtension" entry>.
```

This functionality must read the a single data from the file, prepare it as model input provide and return it.

It is implemented in `fctPrepareData()` method of the class `UC_Model`.

C.6.3 Inference

This fonctionnality must implement the data inference by the model.

It is implemented in `fctInferenceModel()` method of the class `UC_Model`.



Chapter D

Checkpoints

From the version v24.05, ten checkpoints have been set up to control data type and data shape. They can be activated with the TUI command K.

These checkpoints are set to help developer to easily integrate its plugin in the KAA framework. They are implemented in the `kaasrc/controles.py` file.

The results the these checkpoints are displayed as following:

- Positive check example.

Indicates the checkpoint number and the function, the "type" check status with the obtained value and the "shape" check status with the obtained value and its expected format:

```
=====
Control (1) fctPrepareData outputs -- AIX360:Lime
TYPE: OK - <class 'tensorflow.python.framework.ops.EagerTensor'> - (inputModelFramework)
SHAPE: OK - (1, 224, 224, 3) - (1,H,W,B)
=====
```

- Negative check example.

As previously, indicate the checkpoint number and the function, the "type" check status with the obtained value and the "shape" check status with the obtained value and its expected format. In case of KO status, it indicates the expected and the obtained value, the checked function and the file in which the check is done:

```
=====
Control (5') XAI_computeMetrics inputs -- AIX360
TYPE: KO
expected <class 'tensorflow.python.framework.ops.EagerTensor'> (inputXAIframework)
obtained <class 'numpy.ndarray'>
Checked by 'controlXAI_computeExplanationsInput()' in file:
/home/xxx.yyy/kaa/pluginsXAI/AIX360.py
SHAPE: OK - (2, 2) - (n,C)
=====
```

D.1. PluginUCXAI checkpoints

In case of pluginUCXAI, four checkpoints control the inference computation:



D.1.1 FctPrepareDataOutput checkpoint

It verifies the type and the shape of the data returned by the function `fctPrepareData` according to the `inputModelFramework` value declared in the use case class.

The function call is :

```
controlFctPrepareDataOutput (pDictParams, dataPreprocessed, pAiModel.useCase, __file__)
```

D.1.2 controlInferenceInput checkpoint

It verifies the type and the shape of the data before inference computation according to the `inputModelFramework` value declared in the use case class. Its often the same check than the previous one.

The function call is :

```
controlInferenceInput (pDictParams, dataPreprocessed, pAiModel.useCase, __file__)
```

D.1.3 controlInferenceOutput checkpoint

It verifies the type and the shape of the prediction inference according to the `outputModelFramework` value declared in the use case class.

The function call is :

```
controlInferenceOutput (pDictParams, prediction, pAiModel.useCase, __file__)
```

D.1.4 UC_computeInferenceOutput checkpoint

It verifies the type and the shape of the structure containing all the prediction according to the `outputModelFramework` value declared in the use case class.

The function call is :

```
controlUC_computeInferenceOutput (pDictParams, xData, yPred, pAiModel.useCase, pNbData, __file__)
```

D.2. PluginXAI checkpoints

In case of pluginXAI, six checkpoints control the following computation steps:

D.2.1 XAI_computeExplanationsInput checkpoint

In the `XAI_computeExplanations` function it verifies the type and the shape of the data and the prediction according to the `inputXAIframework` value declared in the pluginXAI class before computing the explanations.

The function call is :

```
controlXAI_computeExplanationsInput (pDictParams, self.inputXAIframework, xData, yPred, aiModel.  
useCase, pNbData, __file__, pParamSpecif)
```



D.2.2 XAI_computeExplanationsOutput checkpoint

Still in function `XAI_computeExplanations`, this checkpoint verifies the type and the shape of the explanation data according to the `outputXAIframework` value declared in the `pluginXAI` class.

The function call is :

```
controlXAI_computeExplanationsOutput (pDictParams, self.outputXAIFramework, explanations, aiModel.  
useCase, nbExplanations, __file__)
```

D.2.3 _loadExplanationsOutput checkpoint

In the functions that plot explanation or compute metrics, this checkpoint verifies the type and the shape of the explanation after reading them from disk.

The function call is :

```
control_loadExplanationsOutput (pDictParams, self.outputXAIFramework, explanations, aiModel.useCase,  
__file__, pParamSpecif)
```

D.2.4 pFctExplainToPlotOutput checkpoint

In the function that plot explanation, this checkpoint verifies the type and the shape of the explanation after its transformation by a "*Explain To Plot*" function. This checkpoint is activate if such a function exists.

The function call is :

```
controlpFctExplainToPlotOutput (pDictParams, self.outputXAIFramework, oneExplanation, aiModel.  
useCase, __file__, pParamSpecif)
```

D.2.5 _plotExplanationInput checkpoint

In the function that plot explanation, this checkpoint verifies the type and the shape of both the explanation and the data according to the shape of the plotting requirement.

The function call is :

```
control_plotExplanationInput (pDictParams, self.outputXAIFramework, pExplanPlot, oneExplanation, data  
, aiModel.useCase, __file__, pParamSpecif)
```

D.2.6 XAI_computeMetricsInput checkpoint

This checkpoint is dedicated to the verification of the type and the shape of the initial data and the inference result before computing metrics.

The function call is :

```
controlXAI_computeMetricsInput (pDictParams, self.inputXAIFramework, xData, yPred, aiModel.useCase,  
pNbData, __file__, pParamSpecif)
```




Chapter E

Documentation

E.1. Global parameter dictionary

This dictionary is built up from user commands via the Kaa user interface and the json files of the use case plugins. It is saved on disk in a json file in the kaaJson directory for each treatment or sequence of treatments. The elements of this dictionary are used in the various functions and methods encountered, and are identified by the variable pDictParams.

The entries in this dictionary are as follows:

dataProd	contains the base directory in which the results will be stored
legender	boolean value positioning legend on method or metric plots
percentile	percentile for graph plotting (cf. matplotlib)
alpha	transparency of image data for drawing explanations
normalise	normalization of explanation values when tracing
fontSize	font size for legend in plot results
cmap	look up table of explanation plots
heightPlot	size (in inch) of the plot image result
nbCollatex	number of columns in <i>planche contact</i> report
items	text or tabular data items to be processed
plotDataSize	boolean (0/1) to indicate the reference size of the result (data or input model)
useCase	use case name
library	explainability library name
useCaseBase	data and model access directory (environment variable USECASE_BASE or -b command line option value)
code	model short name
dataListMode	data collection method
dataListFichier	list of data files to be processed
nbData	number of data to be processed
offset	offset selection of data to be processed
stepData	selection step value for data to be processed
cmapListMode	mode to get cmap (by list or file)
cmapListColor	index of cmap in list
controls	boolean to activate/deactivate checkpoints
fullReport	chapter organization of benchmark report



<code>nbColFullLatexMethod</code>	number of columns in method chapter of benchmark report
<code>nbColFullLatexMetric</code>	number of columns in metric chapter of benchmark report
<code>fullReportM</code>	kind of full report
<code>method</code>	name of the method to be used
<code>metric</code>	name of the metric to be used
<code>suffixMetric</code>	path suffix built according to the parameter values to distinguish results
<code>methods4metric</code>	comma-separated string of method names to be processed by the metric
<code><method>*</code>	dictionary of method keys and parameter values
<code><metric>*</code>	and suffix <code>suffixMethod</code> built from these elements to distinguish results
<code>aides</code>	dictionary of metric keys and parameter values
	dictionary of descriptions for the !i TUI action

E.2. Templates

Templates of plugins are available in the two sub-directories with the structure as follow:

Example_PluginsUCXAI

```
-library_name-
├── -library_name-.py
├── -library_name-_computeExplanations.py
├── -library_name-_plotExplanations.py
├── -library_name-_computeMetrics.py
└── -library_name-_plotMetrics.py
```

Example_PluginsXAI

```
UC-nameUC-
├── -nameUC-.py
├── UC-nameUC-.py
├── UC-nameUC-.json
├── UC-nameUC-_nameLib1.py
├── ...
└── UC-nameUC-_nameLibn.py
```



Chapter F

Explainability toolboxes

F.1. AIX360

AIX360 is IBM's explainability library, documentation can be found [here](#). This library contains many tools that are applicable to different places in an AI project. These tools are applicable to both tabular data, images and texts. Some tools are adapted to regression models and others to classification models. In general the methods are model agnostic. We listed all the available methods in the following section and the resume of the documentation provided for each one. All codes are available on this [github](#).

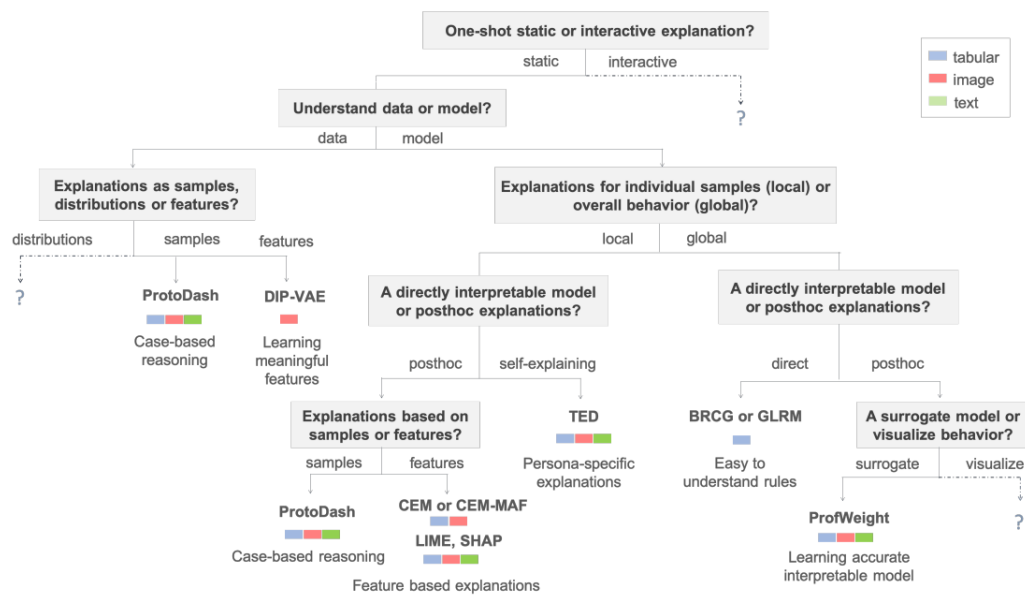


Figure F.1: Applicability of AIX360 tools

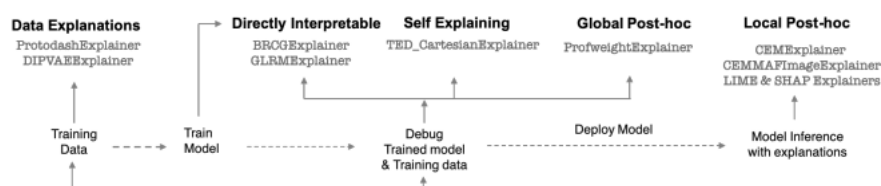


Figure F.2: AIX360 tools on an AI project chain



F.1.1 The library

All papers are listed on the website [here](#) and [here](#)

Protodash

Protodash is an explainability method that aims to better understand the data and the model locally. It is a method that is applied to classification models and allows a better understanding of them. Indeed, this method returns the closest data to the studied example and having the same classification as this one. The influence of the parameters on the classification can be thus deduced locally. This method is intended for end users.

For more information you can read [this paper](#).

DIPVAE (Disentangled Inferred Prior Variational Autoencoder Explainer)

DIPVAE is an explainability method that also aims at better understanding the data. This method is intended for end users. This method is based on the training of an auto-encoder which reduces the size of the data and then decompress it. This method highlights the main characteristics of the data. However this method has the disadvantage of being applicable only to images.

For more information you can read [this paper](#).

LIME (Local Interpretable Model-Agnostic Explanations)

LIME is an explainability method that aims to provide a local explanation of the model after training. This method is intended for end users. It is a method that applies to both classification and regression models. It highlights the characteristics of the data that influence the classification. To do this, this method varies the input data. This method is similar to SHAP but has the advantage of being faster.

To validate or not the explanation given by LIME, these metrics can be used:

- monotonicity,
- faithfulness.

For more information you can read [this paper](#).

SHAP (Shapley Additive explanation)

SHAP is a method similar to LIME. Indeed it also aims to provide a local explanation of the model after training. This method is also intended for end users. It is a method that applies to both classification and regression models. It highlights the characteristics of the data that influence the classification. To do this, this method calculates all global permutations and deduces a local explanation. It is this difference that makes SHAP slower to run than LIME.

For more information you can read [this paper](#).

CEM (Contrastive Explainable Method)

CEM is an explainability method which also aims at providing a local explanation of the model after its training. This method is intended for data engineers. It is a method that applies to classification models. This method provides two different types of explanation:

- Pertinent Positives (PP): The Pertinent Positives explanation finds the features that are necessary for the model to predict the same output class as the predicted class. For example, this includes the important pixels of an image or the features having a high feature weight.
- Pertinent Negatives (PN): The Pertinent Negatives explanation finds the features that should be minimally and sufficiently absent from an instance whilst maintaining the original output class.

For more information you can read [this paper](#) and [this paper](#).

TED (Teaching Explanations for Decisions)

TED is an explainability method that aims to provide a local explanation of the model through explanations given in the model's input. This method is intended for end users. The principle of TED is to train the model to return both a prediction and the corresponding explanation. To apply TED, it is therefore necessary to have an augmented dataset, i.e. a dataset where each data item contains input features, an output decision and an explanation of this decision. The advantage of TED is that it produces explanations that are directly understandable



by end users.

For more information you can read [this paper](#).

BRCGE (Boolean Rule Column Generation Explainer)

BRCGE is an explainability method that aims to provide a direct and global explanation of the model. This method is intended for data scientists. This method applies only to binary tabular data and classification models. The principle of BRCGE is to generate rules that allow to understand the classifications of the model. The rules can be of two types:

- CNF : rules in normal conjunctive form (AND-of-ORs);
- DNF : rules in normal disjunctive form (OR-of-ANDs).

To validate or not the explanation given by BRCGE, these metrics can be used:

- the training accuracy ;
- the test accuracy ;
- the test precision ;
- the test recall ;
- the test F1 score.

For more information you can read [this paper](#).

Linear Rule Regression

Linear Rule Regression is a method quite similar to BRCGE. Indeed, it also aims at providing a direct and global explanation of the model. This method is also intended for data scientists. This method is only applicable to binary tabular data and regression models. The principle of Linear Rule Regression is to generate linear rules that approximate the regression of the model.

To validate or not the explanation given by Linear Rule Regression, these metrics can be used:

- the training R^2 score ;
- the test R^2 score ;
- the test explained variance ;
- the test mean absolute error ;
- the test maximum error.

For more information you can read [this paper](#)

Logistic Rule Regression

Logistic Rule Regression is a method quite similar to BRCGE and Linear Rule Regression. Indeed, it also aims at providing a direct and global explanation of the model. This method is also intended for data scientists. This method is only applicable to binary tabular data and classification models. The principle of Logistic Rule Regression is to generate logistic rules that allow to understand the classifications of the model.

To validate or not the explanation given by Logistic Rule Regression, these metrics can be used:

- the training accuracy ;
- the test accuracy ;
- the test precision ;
- the test recall ;
- the test F1 score.

For more information you can read [this paper](#)

Profweight Explainer

Profweight Explainer is a method that aims to provide a global explanation of the model. This method is intended for data scientists. This method works by substituting the complex model by a simpler model of low complexity. This method applies to all types of models. The principle of Profweight Explainer is to train a simple model by using the training on a more complex model so that the simple model has better performances. As the model is simple, it can be interpreted and associate the explanations obtained with the more complex model.



To work, the algorithm needs to retrieve the weights of some layers of the complex model. For that the complex model must have a ".meta" format.

For more information you can read this paper

F.1.2 The metrics

F.1.2.1 Metrics of AIX360

Faithfulness

The Faithfulness metric applies only to LIME and corresponds to the opposite of the Pearson Product-Moment correlation. Faithfulness is therefore between -1 and 1 with 1 corresponding to a negative correlation, -1 to a positive correlation and 0 to no correlation.

$$\text{Faithfulness} = -\frac{\sigma_{xy}}{(\sigma_x + \sigma_y)}$$

with $\sigma_{xy} = \sqrt{E(xy) - E(x)E(y)}$ and $\sigma_x = \sqrt{E(x^2) - E(x)^2}$

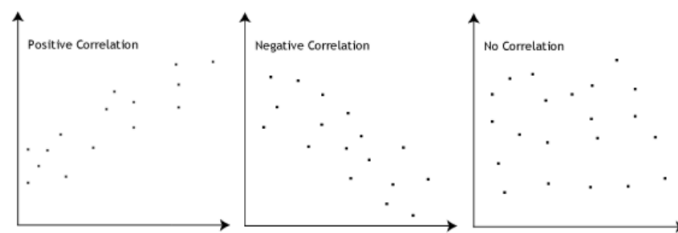


Figure F.3: Examples of Pearson correlations

Here is the code corresponding to its calculation.

```
def faithfulness_metric(model, x, coefs, base):
    #find predicted class
    pred_class = np.argmax(model.predict_proba(x.reshape(1,-1)), axis=1)[0]

    #find indexes of coefficients in decreasing order of value
    ar = np.argsort(-coefs) #argsort returns indexes of values sorted in increasing order; so do it for negated array
    pred_probs = np.zeros(x.shape[0])
    for ind in np.nditer(ar):
        x_copy = x.copy() #makes a copy of x so as not to alter it
        x_copy[ind] = base[ind] #replaces in position ind of the copy of x its value by a 0 because base is a null vector
        x_copy_pr = model.predict_proba(x_copy.reshape(1,-1)) #retrieves the probabilities obtained in the output of the model for the copy of x
        pred_probs[ind] = x_copy_pr[0][pred_class] #adds in position ind the probability obtained for the pred_class

    #the opposite of the Pearson Product-Moment correlation
    return -np.corrcoef(coefs, pred_probs)[0,1] #coefs corresponds to the coefficients calculated by LIME for the pred_class
```

Figure F.4: Code of the faithfulness metric

This metric seems to be equivalent to the metric "MuFidelity" of Xplique.

Monotonicity

The monotonicity metric also applies only to LIME. This method consists in progressively adding the values of x to a null vector, then looking if the probability of predicting the correct class with it is increasing.



```
def monotonicity_metric(model, x, coefs, base):
    #find predicted class
    pred_class = np.argmax(model.predict_proba(x.reshape(1,-1)), axis=1)[0]
    x_copy = base.copy() #makes a copy of base which is a null vector

    #find indexes of coefficients in increasing order of value
    ar = np.argsort(coefs)
    pred_probs = np.zeros(x.shape[0])
    for ind in np.nditer(ar):
        x_copy[ind] = x[ind] #replaces in position ind of the copy of x its 0 by the value of x
        x_copy_pr = model.predict_proba(x_copy.reshape(1,-1)) #retrieves the probabilities obtained in the output of the model for the copy of x
        pred_probs[ind] = x_copy_pr[0][pred_class] #adds in position ind the probability obtained for the pred_class

    #returns 1 if the pred_probs vector is increasing and 0 otherwise
    return np.all(np.diff(pred_probs[ar]) >= 0)
```

Figure F.5: Code of the monotonicity metric

This metric seems to be equivalent to the metric "Insertion" of Xplique.

F.1.2.2 Metrics of external libraries

		Actual	
		Positive	Negative
Predicted	Positive	True Positive	False Positive
	Negative	False Negative	True Negative

Figure F.6: True Positive, True Negative, False Positive, False Negative

Accuracy

$$Accuracy = \frac{TP+TN}{TP+FP+FN+TN}$$

Precision

$$Precision = \frac{TP}{TP+FP}$$

Recall

$$Recall = \frac{TP}{TP+FN}$$

F1 score

$$F1_score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

R^2 score

$$R^2_score = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \text{ avec } \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$$

Explained Variance

$$Explained_Variance = 1 - \frac{Var\{y - \hat{y}\}}{Var\{y\}}$$

Mean absolute error

$$MAE = \frac{1}{n_{samples}} \sum_{i=0}^{n_{samples}-1} |y_i - \hat{y}_i|$$

Max error

$$Max_Error = \max(|y_i - \hat{y}_i|)$$



F.2. Alibi

F.2.1 Description

Alibi is an open source Python library aimed at machine learning model inspection and interpretation. The focus of the library is to provide high-quality implementations of black-box, white-box, local and global explanation methods for classification and regression models.

F.2.2 The explanation methods

All methods work for classification and on tabular data.

Method	Models	Explan.	Regression	Text	Images	Categorical features	Train set required
ALE	BB	global	✓			✓	
Partial Dependence	BB WB	global	✓			✓	
PD Variance	BB WB	global	✓			✓	
Permutation Importance	BB	global	✓			✓	
Anchors	BB	local		✓	✓	✓	For tab.
CEM	BB* TFK	local			✓		Opt.
Counterfactuals	BB* TFK	local			✓		
Prototype Counterfactuals	BB* TFK	local			✓	✓	Opt.
Counterfactuals with RL	BB	local			✓	✓	✓
Integrated Gradients	TFK	local	✓	✓	✓	✓	Opt.
Kernel SHAP	BB	local - global	✓			✓	✓
Tree SHAP	WB	local - global	✓			✓	Opt.
Similarity explanations	WB	local	✓	✓	✓	✓	✓

Keys:

- BB - black-box (only require a prediction function)
- BB* - black-box but assume model is differentiable
- WB - requires white-box model access. There may be limitations on models supported
- TFK - TensorFlow models via the Keras API
- Local - instance specific explanation
- Global - explains the model with respect to a set of instances

We will only focus on the methods Anchors, Integrated Gradients and Similarity explanations because they can be applied on text.

Anchors computes local post-hoc explanations. This methods replaces input feature by another value to see which features are necessary to obtain the prediction.

Integrated Gradient as described previously uses the gradients of the model to obtain local post-hoc explanations.

Similarity explanations method computes local post-hoc explanations based on examples. This methods returns data which are similar to the data we are trying to explain. This method is quite similar to the method *Protodash* of AIX360.



F.3. Captum

F.3.1 Description

Captum is Meta's explainability library. This library contains many tools that are applicable to Pytorch models that classifies images or texts. These tools are intended to datascientists and end users. The Captum ¹ library contains 3 types of methods:

- Primary Attribution: which shows from which pixels or words the final classification comes from,
- Layer Attribution: which shows the impact of each layer of the network on the final classification,
- Neuron Attribution: which shows the impact of each neuron of a layer on the final classification.

Each type of method contains respectively 14, 10 and 8 different methods.

In this benchmark we focus on primary attribution methods, because we want to study the influence of inputs (data) on the output (prediction/classification...) and not understand the behavior of internal mechanisms of deep network such as layers and neurons.

6 methods are model agnostic, black box methods for explanation : Feature Ablation, Feature Permutation, LIME, Occlusion, Shapley Value Sampling, Kernel SHAP. Mainly used for local explanations, they can be used for global explanation when applied to the whole dataset but usually not because of high computing cost. DeepLIFT, DeepLIFT SHAP, Guided Backpropagation and Deconvolution and Guided GradCam are backpropagation-based methods they are model specific (neuron network). Integrated gradient, Gradient SHAP, Saliency and InputXGradient are gradient-based methods are model agnostic but you need access to the gradient of your model.

Black Box and model agnostic methods can be applied to any data : tabular, text and image with more or less efficiency as described in the benchmark for each use cases.

The output of explanation are numerical lists or matrices, but the library does not propose any visualisation.

F.3.2 Primary attribution

For more information and original papers please refer to the documentation of the library : [here](https://captum.ai/docs/algorithms). The following section just resumes the information available in the documentation.

- **Integrated Gradients** : represents the integral of gradients with respect to inputs along the path from a given baseline to input. The integral can be approximated using a Riemann Sum or Gauss Legendre quadrature rule.
- **Gradient SHAP** : is a gradient method to compute SHAP values, which are based on Shapley values. Gradient SHAP adds Gaussian noise to each input sample multiple times, selects a random point along the path between baseline and input, and computes the gradient of outputs with respect to those selected random points. The final SHAP values represent the expected value of gradients * (inputs - baselines).
- **DeepLIFT** : is a back-propagation based approach that attributes a change to inputs based on the differences between the inputs and corresponding references (or baselines) for non-linear activations. As such, DeepLIFT seeks to explain the difference in the output from reference in terms of the difference in inputs from reference. DeepLIFT uses the concept of multipliers to "blame" specific neurons for the difference in output.
- **DeepLIFT SHAP** : is a method extending DeepLIFT to approximate SHAP values, which are based on Shapley values. DeepLIFT SHAP takes a distribution of baselines and computes the DeepLIFT attribution for each input-baseline pair and averages the resulting attributions per input example.
- **Saliency** : is a simple approach for computing input attribution, returning the gradient of the output with respect to the input. This approach can be understood as taking a first-order Taylor expansion of the network at the input, and the gradients are simply the coefficients of each feature in the linear representation of the model. The absolute value of these coefficients can be taken to represent feature importance.
- **Input X Gradient** : is an extension of the saliency approach, taking the gradients of the output with

¹<https://captum.ai/docs/algorithms>



respect to the input and multiplying by the input feature values. One intuition for this approach considers a linear model; the gradients are simply the coefficients of each input, and the product of the input with a coefficient corresponds to the total contribution of the feature to the linear model's output.

- **Guided Backpropagation and Deconvolution** : compute the gradient of the target output with respect to the input, but backpropagation of ReLU functions is overridden so that only non-negative gradients are backpropagated. In guided backpropagation, the ReLU function is applied to the input gradients, and in deconvolution, the ReLU function is applied to the output gradients and directly backpropagated. Both approaches were proposed in the context of a convolutional network and are generally used for convolutional networks, although they can be applied generically.
- **Guided GradCAM** : computes the element-wise product of guided backpropagation attributions with upsampled (layer) GradCAM attributions. GradCAM attributions are computed with respect to a given layer, and attributions are upsampled to match the input size. This approach is designed for convolutional neural networks. The chosen layer is often the last convolutional layer in the network, but any layer that is spatially aligned with the input can be provided.
- **Feature Ablation** : is a perturbation based approach to compute attribution, involving replacing each input feature with a given baseline / reference value (e.g. 0), and computing the difference in output. Input features can also be grouped and ablated together rather than individually. This can be used in a variety of applications. For example, for images, one can group an entire segment or region and ablate it together, measuring the importance of the segment (feature group).
- **Feature Permutation** : is a perturbation based approach which takes each feature individually, randomly permutes the feature values within a batch and computes the change in output (or loss) as a result of this modification. Like feature ablation, input features can also be grouped and shuffled together rather than individually. Note that unlike other algorithms in Captum, this algorithm only provides meaningful attributions when provided with a batch of multiple input examples, as opposed to other algorithms, where a single example is sufficient.
- **Occlusion** : is a perturbation based approach to compute attribution, involving replacing each contiguous rectangular region with a given baseline / reference, and computing the difference in output. For features located in multiple regions (hyperrectangles), the corresponding output differences are averaged to compute the attribution for that feature. Occlusion is most useful in cases such as images, where pixels in a contiguous rectangular region are likely to be highly correlated.
- **Shapley Value Sampling** : is an attribution method based on a concept from cooperative game theory. This method involves taking each permutation of the input features and adding them one-by-one to a given baseline. The output difference after adding each feature corresponds to its contribution, and these differences are averaged over all permutations to obtain the attribution.
- **Lime** : is an interpretability method that trains an interpretable surrogate model by sampling data points around a specified input example and using model evaluations at these points to train a simpler interpretable 'surrogate' model, such as a linear model. There are 2 implementations of this method: LimeBase and Lime. LimeBase provides a generic framework to train a surrogate interpretable model. This differs from most API of other attribution methods, since the method returns a representation of the interpretable model (e.g. coefficients of the linear model). On the other hand, Lime provides a more specific implementation than LimeBase in order to expose a consistent API with other perturbation-based algorithms.
- **KernelSHAP** : is a method that uses the LIME framework to compute Shapley Values. Setting the loss function, weighting kernel and regularization terms appropriately in the LIME framework allows theoretically obtaining Shapley Values more efficiently than directly computing Shapley Values.

F.3.3 Layer attribution

- **Layer Conductance** : combines the neuron activation with the partial derivatives of both the neuron with respect to the input and the output with respect to the neuron to build a more complete picture of neuron importance.
- **Internal Influence** : approximates the integral of gradients with respect to a particular layer along the



path from a baseline input to the given input. This method is similar to applying integrated gradients, integrating the gradient with respect to the layer (rather than the input).

- **Layer Activation** : is a simple approach for computing layer attribution, returning the activation of each neuron in the identified layer.
- **Layer Gradient X Activation** : is the analog of the Input X Gradient method for hidden layers in a network. It element-wise multiplies the layer's activation with the gradients of the target output with respect to the given layer.
- **GradCAM** : is a layer attribution method designed for convolutional neural networks, and is usually applied to the last convolutional layer. GradCAM computes the gradients of the target output with respect to the given layer, averages for each output channel (dimension 2 of output), and multiplies the average gradient for each channel by the layer activations. The results are summed over all channels and a ReLU is applied to the output, returning only non-negative attributions.
- **Layer Integrated Gradients** : represents the integral of gradients with respect to the layer inputs / outputs along the straight-line path from the layer activations at the given baseline to the layer activation at the input.
- **Layer GradientSHAP** : is the analog of GradientSHAP for a particular layer. Layer GradientSHAP adds Gaussian noise to each input sample multiple times, selects a random point along the path between baseline and input, and computes the gradient of the output with respect to the identified layer. The final SHAP values approximate the expected value of gradients * (layer activation of inputs - layer activation of baselines).
- **Layer DeepLIFT** : is the analog of the DeepLIFT method for hidden layers in a network.
- **Layer DeepLIFT SHAP** : is the analog of DeepLIFT SHAP for a particular layer. Layer DeepLIFT SHAP takes a distribution of baselines and computes the Layer DeepLIFT attribution for each input-baseline pair and averages the resulting attributions per input example.
- **Layer Feature Ablation** : is the analog of feature ablation for an identified layer input or output. It is a perturbation based approach to compute attribution, involving replacing each value in the identified layer with a given baseline / reference value (e.g. 0), and computing the difference in output. Values within the layer can also be grouped and ablated together rather than individually.

F.3.4 Neuron attribution

- **Neuron Conductance** : combines the neuron activation with the partial derivatives of both the neuron with respect to the input and the output with respect to the neuron to build a more complete picture of neuron importance.
- **Neuron Gradient** : is the analog of the saliency method for a particular neuron in a network. It simply computes the gradient of the neuron output with respect to the model input. Like Saliency, this approach can be understood as taking a first-order Taylor expansion of the neuron's output at the given input, and the gradients correspond to the coefficients of each feature in the linear representation of the model.
- **Neuron Integrated Gradients** : approximates the integral of input gradients with respect to a particular neuron along the path from a baseline input to the given input. This method is equivalent to applying integrated gradients considering the output to be simply the output of the identified neuron.
- **Neuron Guided Backpropagation and Deconvolution** : are the analogs of guided backpropagation and deconvolution for a particular neuron.
- **Neuron GradientSHAP** : is the analog of GradientSHAP for a particular neuron. Neuron GradientSHAP adds Gaussian noise to each input sample multiple times, selects a random point along the path between baseline and input, and computes the gradient of the target neuron with respect to each selected random points. The final SHAP values approximate the expected value of gradients * (inputs - baselines).
- **Neuron DeepLIFT** : is the analog of the DeepLIFT method for a particular neuron.
- **Neuron DeepLIFT SHAP** : is the analog of DeepLIFT SHAP for a particular neuron. Neuron DeepLIFT SHAP takes a distribution of baselines and computes the Neuron DeepLIFT attribution for each input-baseline pair and averages the resulting attributions per input example.
- **Neuron Feature Ablation** : is the analog of feature ablation for a particular neuron. It is a perturbation



based approach to compute attribution, involving replacing each input feature with a given baseline / reference value (e.g. 0), and computing the difference in the target neuron's value. Input features can also be grouped and ablated together rather than individually. This can be used in a variety of applications. For example, for images, one can group an entire segment or region and ablate it together, measuring the importance of the segment (feature group).

F.3.5 "Noise Tunnel" function

The Captum library also contains a "Noise Tunnel" function which allows to add a Gaussian noise to our input. The supported types for noise tunnel are:

- Smoothgrad: The mean of the sampled attributions is returned. This approximates smoothing the given attribution method with a Gaussian Kernel
- Smoothgrad Squared: The mean of the squared sample attributions is returned.
- Vargrad: The variance of the sample attributions is returned.

F.3.6 Metrics

Finally the Captum library contains 2 metrics:

- **Infidelity** : measures the mean squared error between model explanations in the magnitudes of input perturbations and predictor function's changes to those input perturbations.
- **Sensitivity** : measures the degree of explanation changes to subtle input perturbations using Monte Carlo sampling-based approximation.



F.4. PAIR Saliency

F.4.1 Description

This library is developed by Google PAIR (People + AI Research).

This library is composed of attribution methods based on saliency map explanations, and has two subpackages:

- the first subpackage uses a generic *call_model_function* which can be used with any ML framework.
- the second subpackage accepts input/output tensors directly, and sets up the necessary graph operations for each method.

All codes are available on github.

N.B.: the attribution methods can be applied to different format models because the gradient function is external and one can treat their specificity. All of these methods are for local explanation.

F.4.2 The attribution methods

These attribution methods can be used to get saliency mask but these techniques can be augmented with the SmoothGrad technique by denoising the saliency masks obtained using these sensitivity techniques.

F.4.2.1 (Vanilla) Gradients and SmoothGrad

This method computes the standard model gradients w.r.t. the features but can also compute its gradients averages ; the method of averaging the (Vanilla) gradients is equivalent to SmoothGrad. SmoothGrad often reduces the noise from irrelevant noisy regions. (paper,paper)

F.4.2.2 Grad-CAM

The approach of Grad-CAM is a generalization of Class Activation Maps (CAM), a technique for identifying discriminative regions, by using the gradient information flowing into a model layer. This method increases the accuracy and the complexity w.r.t. the CAM approach. (paper)

F.4.2.3 Occlusion

The occlusion sensitivity is a method for understanding which features are the most important for a deep network's classification. When occluding an important feature, a strong drop in activity can be seen in the feature map.

F.4.2.4 Integrated Gradients

Integrated Gradients combines the implementation of gradients along a baseline and with the sensitivity. Integrated Gradients has become a popular interpretability technique due to its broad applicability to any differentiable model, ease of implementation, and computational efficiency relative to alternative approaches. (paper)

F.4.2.5 Guided IG (Guided Integrated Gradients)

Guided IG improves upon Integrated Gradients by introducing the idea of an Adaptive Path Method and can take advantage of the knowledge of the model to dynamically construct a path that meets a desirable objective. By computing integration along a different path than Integrated Gradients, high gradient areas are avoided which often leads to an overall reduction in irrelevant attributions. (paper)

F.4.2.6 Blur IG (Blur Integrated Gradients)

BlurIG extends the Integrated Gradients technique. Blur Integrated Gradient relies on Gaussians and Laplacians of the Gaussian to construct human intelligible explanations. (paper)



F.4.2.7 XRAI (eXplanation with Ranked Area Integrals)

This method attributes regions instead of individual pixels. It does it by aggregating pixel level attributions within regions to find image areas that positively or negatively impact the prediction. As a result, the XRAI attributions are often much easier to interpret than pixel-level saliency. (paper)

F.4.3 The metrics

F.4.3.1 PIC (Performance Information Curve)

Performance Information Curves (PICs) are curves of a function of the calculated entropy for each input. PIC provides a good view of the attribution quality.



F.5. SHAP

F.5.1 Description

Shap is an explainability library whose methods are unified by SHAP ([link to the documentation here](#)). SHAP (SHapley Additive exPlanations) is a widely used approach from cooperative game theory. It is a black box, model agnostic approach mainly used for local explanation but applied to the whole dataset it can be used as a global method of explanation (at a heavy computing cost). The local explanations of the methods of the library Shap are related to those of the method SHAP. These methods can be applied to any type of data in input : tabular, text and image.

Most of the algorithms and visualizations come from the University of Washington and Microsoft Research.

F.5.2 The explanation methods

Shapley values are the contributions of feature values. The goal of all methods is to highlight the impact of the features in the models predictions, therefore through Shapley values.

As SHAP is a black-box model method, these methods can be adapted to many models such as deep learning models, tree-based models and NLP models. There is also a great compatibility with many model format: TensorFlow, Keras, PyTorch, XGBoost, scikit-learn, ... The available data format for Shap are images, tabular data and text data.

Among all methods in the library Shap, one is selected for this benchmark report: the **Partition** method. As the impact of the features of the model predictions is desired, it is almost computationnally unextractable, thus the impact is approximated. However, Partition has one specific advantage through the Owen values: this method always assigns to groups of correlated features the credit that set of features would have had if treated as a group.

The other methods (Kernel, Sampling, Tree, Deep, Gradient, Linear, Permutation and Additive) have not been developed here ; their documentation is located on this [github](#).

For more information, the associated papers are :

- For general use of SHAP NeurIPS paper
- For TreeExplainer you can read Nature Machine Intelligence paper
- For GPU TreeExplainer you can read this paper
- For visualizations and medical applications you can read Nature Biomedical Engineering paper

F.5.3 The metrics

The metrics written in this library are similar to the causal fidelity metrics from the Xplique library (deletion and insertion, see F.6). They measure the decrease or the increase of the prediction score when removing progressively the features, which are given by the explainer:

- the most important and positive features
- the most important and negative features
- the most important features in absolute value



F.6. Xplique

The library is composed of several modules: the Attributions Methods module, the Metrics module, the Feature Visualization module, and the Concepts module (concept extraction from a model). Some methods are white-box and/or black-box attribution models, whereas the metrics are more dedicated to data formats. The code can be found on [github](#).

We focus on the attribution methods and metrics. There are 16 methods implemented to date, either black box and model agnostic methods (LIME, KernelSHAP, Occlusion, Rise,...) or gradient/backpropagation based. A lot of tutorials in the documentation are either for images or tabular dataset, we will test these methods for text.

F.6.1 The attribution methods

Attribution methods are interpretability and explainability methods highlighting features or feature pattern that influences a neural network's prediction. For more information see the [documentation](#) here.

Attributions only depend on one method, however, if several explanations are completely different or incoherent between methods, There is a strong probability that the model is bad.

The attribution methods can be divided in two types:

- white-box attribution methods that are based on computation of gradients and back-propagation,
- black-box explanation methods that are based on perturbation of one or some features.

F.6.1.1 White-box attribution methods

F.6.1.1.1 DeconvNet

The DeconvNet method aims to reconstruct the input of each layer from its output, showing what input pattern caused a given activation in the feature maps.

F.6.1.1.2 Guided Backpropagation

Guided backpropagation is a variant to the DeconvNet method. The differences between these methods are the gradients of ReLU functions. This method reduces its complexity and its accuracy w.r.t. the DeconvNet approach.

F.6.1.1.3 Saliency

The Saliency method generalizes the DeconvNet method.

F.6.1.1.4 Gradient x Input

Gradient x Input computes the element-wise product between the saliency maps and the input to improve the sharpness of the attribution maps.

F.6.1.1.5 Integrated gradients

Integrated Gradients combines the implementation of gradients along with the sensitivity (e.g. of the technique Layer-wise Relevance Propagation). Due to its implementation, the method results are better than the above methods, in most of the cases.

F.6.1.1.6 Smoothgrad

The SmoothGrad method consists in adding some Gaussian noise and averaging saliency maps. Adding some noise to the samples reduces the noise to the explanation.



F.6.1.1.7 SquareGrad

The SquareGrad method is similar to the SmoothGrad method. The difference between SquareGrad and SmoothGrad is the way of averaging the noisy samples: SmoothGrad averages the values of the noisy samples, SquareGrad averages the standard deviation values of the noisy samples.

F.6.1.1.8 VarGrad

The VarGrad method is similar to the SmoothGrad method. The difference between VarGrad and SmoothGrad is the way of averaging the noisy samples: SmoothGrad averages the values of the samples, VarGrad averages the values in function of their variance.

F.6.1.1.9 Grad-CAM

The approach of Grad-CAM is a generalization of Class Activation Maps (CAM), a technique for identifying discriminative regions, by using the gradient information flowing into a model layer. This method increases the accuracy and the complexity w.r.t. the CAM approach.

F.6.1.1.10 Grad-CAM++

The approach of Grad-CAM++ is a generalization of Grad-CAM and increases the precision and complexity w.r.t. the Grad-CAM method.

F.6.1.2 Black-box explanation methods

F.6.1.2.1 LIME

The LIME method interprets individual model predictions (of any classifier or regressor) in a faithful way, by approximating it locally with an interpretable model around a given prediction.

F.6.1.2.2 KernelSHAP

The KernelSHAP method is a combination of linear LIME method and Shapley values (feature importances for linear models). This method increases the accuracy and decreases the simulation time w.r.t. Lime and SHAP.

F.6.1.2.3 Occlusion

The occlusion sensitivity is a method for understanding which features are the most important for a deep network's classification. When occluding an important feature, a strong drop in activity can be seen in the feature map. The comparison between Occlusion and the other methods depends on the parameter values. Generally, when the performance is better, the simulation time increases.

F.6.1.2.4 RISE

The RISE method generates an importance map indicating how salient each feature is for the model's prediction. This method estimates importance empirically by probing the model with randomly masked versions of the input image and obtaining the corresponding outputs. RISE performs competitively in terms of the human-based metric. The comparison between RISE and the other methods depends on the parameter values. Generally, when the performance is better, the simulation time increases.



F.6.1.2.5 HSIC

The Hsic attribution method explains a neural network's prediction for a given input image by assessing the dependence between the output and patches of the input. Thanks to the sample efficiency of HSIC Estimator, this black box method requires fewer forward passes to produce relevant explanations.

F.6.1.2.6 Sobol

The Sobol attribution method is an attribution method grounded in Sensitivity Analysis. Beyond modeling the individual contributions of image regions, Sobol indices provide efficient way to capture higher-order interactions between image regions and their contributions to a neural network's prediction through the lens of variance.

F.6.2 The explanation metrics

F.6.2.1 MuFidelity

The muFidelity metric represents the faithfulness for explainers. This metric corresponds to the Pearson's correlation coefficient between the predicted logits of each modified test point and the average explanation for only the subset of features. Faithfulness increases with the increasing of subset size.

F.6.2.2 Deletion

The Deletion metric is a fidelity metric, it measures the decrease of the prediction score when removing progressively the most important features, which are given by the explainer.

F.6.2.3 Insertion

The Insertion metric is a fidelity metric, it measures the increase in the prediction score when adding progressively the most important features, which are given by the explainer.

F.6.2.4 Average Stability

The Average Stability metric ensure that close inputs with similar predictions yields similar explanations.

F.6.2.5 MeGe

MeGe means Mean Generalizability and is a metric for representativity of explanations. MeGe gives an overview of the generalization of your explanations: the explanations provided by datasets, and the others.

F.6.2.6 ReCo

ReCo means Relative Consistency and is a metric for consistency of explanations. ReCo is the consistency score of the explanations by ensuring that contradictory predictions lead to different explanations.



Table of links

AIX360	doc.	http://aix360.mybluemix.net
	code	https://github.com/Trusted-AI/AIX360
	paper	http://aix360.mybluemix.net/resources#overview
		http://aix360.mybluemix.net/resources#papers
		https://arxiv.org/pdf/1707.01212.pdf
		https://openreview.net/pdf?id=H1kG7GZAW
		https://arxiv.org/pdf/1602.04938.pdf
		https://proceedings.neurips.cc/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf
		https://papers.nips.cc/paper/2018/file/c5ff2543b53f4cc0ad3819a36752467b-Paper.pdf
		https://arxiv.org/pdf/1905.12698.pdf
	paper	https://dl.acm.org/doi/10.1145/3306618.3314273
		https://papers.nips.cc/paper/2018/file/743394beff4b1282ba735e5e3723ed74-Paper.pdf
		http://proceedings.mlr.press/v97/wei19a.html
		http://proceedings.mlr.press/v97/wei19a.html
		https://papers.nips.cc/paper/2018/file/972cda1e62b72640cb7ac702714a115f-Paper.pdf
Alibi	doc.	https://docs.seldon.io/projects/alibi/en/stable
	code	https://github.com/SeldonIO/alibi/tree/master
Captum	doc.	https://captum.ai/docs/attribution_algorithms
PairSaliency	code	https://github.com/PAIR-code/saliency
	paper	https://arxiv.org/abs/1312.6034
		https://arxiv.org/abs/1706.03825
		https://arxiv.org/abs/1610.02391
		https://arxiv.org/abs/1703.01365
		https://arxiv.org/abs/2106.09788
Shap	doc.	https://arxiv.org/abs/2004.03383
		https://arxiv.org/abs/1906.02825
	paper	https://shap.readthedocs.io/en/latest
		https://github.com/slundberg/shap
		https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html
Xplique	based on	https://www.nature.com/articles/s42256-019-0138-9
		https://arxiv.org/abs/2010.13972
		https://www.nature.com/articles/s41551-018-0304-0
		https://github.com/deel-ai/xplique
		https://deel-ai.github.io/xplique
		https://arxiv.org/abs/1311.2901
		https://arxiv.org/abs/1610.02391
Xplique	based on	https://arxiv.org/abs/1710.11063
		https://arxiv.org/abs/1704.02685
		https://arxiv.org/abs/1412.6806
		https://arxiv.org/abs/1412.6806



based on <https://arxiv.org/abs/1703.01365>
based on <https://arxiv.org/abs/1705.07874>
based on <https://arxiv.org/abs/1602.04938>
based on <https://arxiv.org/abs/1311.2901>
based on <https://arxiv.org/abs/1806.07421>
based on <https://arxiv.org/abs/1312.6034>
based on <https://arxiv.org/abs/1706.03825>
based on <https://arxiv.org/abs/1806.10758>
based on <https://arxiv.org/abs/1810.03292>
based on <https://arxiv.org/abs/2111.04138>
based on <https://arxiv.org/abs/2206.06219>
based on <https://arxiv.org/abs/2005.00631>
based on <https://arxiv.org/abs/1806.07421>
based on <https://arxiv.org/abs/1806.07421>
based on <https://arxiv.org/abs/2005.00631>
based on <https://arxiv.org/abs/2009.04521>
based on <https://arxiv.org/abs/2009.04521>
based on <https://arxiv.org/pdf/1711.11279.pdf>
based on <https://arxiv.org/pdf/1711.11279.pdf>
based on <https://distill.pub/2017/feature-visualization>
